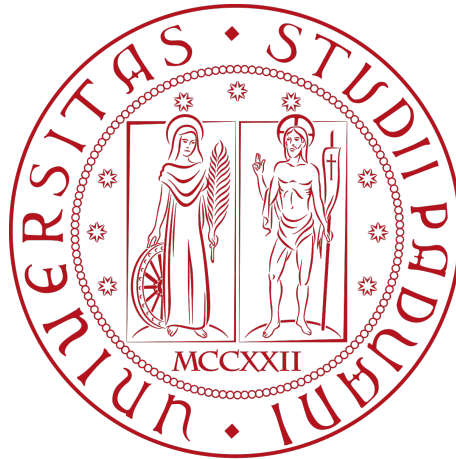


Università degli Studi di Padova
Dipartimento di Scienze Statistiche

Corso di Laurea Triennale in
Statistica per le Tecnologie e le Scienze



RELAZIONE FINALE

**Pacchetti Python per la distribuzione
Normale Asimmetrica:
analisi comparativa con R
e proposta di un approccio integrato**

Relatore: Prof. Antonio Canale
Dipartimento di Scienze Statistiche

Laureando: Mattia Casetta
Matricola N. 2015821

Anno Accademico 2024/2025

Indice

Introduzione	6
1 La Distribuzione Normale Asimmetrica	9
1.1 La diffusione	10
1.2 Aspetti teorici	11
1.2.1 Definizione e proprietà della Normale Asimmetrica	11
1.2.2 t Asimmetrica	14
1.2.3 Estensioni multivariate	16
1.2.4 Normale Asimmetrica Unificata (SUN)	18
1.3 Aspetti computazionali	20
1.3.1 Simulazione di dati	21
1.3.2 Stima dei parametri	23
2 Implementazioni software	27
2.1 La situazione generale	28
2.2 Stato dell'arte in R: pacchetto <i>sn</i>	28
2.3 Alternative Python	31
2.3.1 SciPy	31
2.3.2 Skewt-scipy	32
2.3.3 Multivariate skew distribution	33
2.4 Analisi della situazione generale	33
3 Analisi e confronti dei codici: R vs Python	35
3.1 Introduzione generale	36
3.2 Normale Asimmetrica univariata	36
3.2.1 Descrizione delle analisi	36
3.2.2 Risultati	37
3.3 Normale Asimmetrica multivariata	41

3.3.1	Descrizione delle analisi	41
3.3.2	Risultati	42
3.4	t Asimmetrica univariata	45
3.4.1	Descrizione delle analisi	45
3.4.2	Risultati	45
3.5	t Asimmetrica multivariata	50
3.5.1	Descrizione delle analisi	50
3.5.2	Risultati	51
3.6	Analisi generale dei risultati	56
4	Proposta di un pacchetto integrato	57
4.1	Proposta di integrazione	58
4.2	Schemi di struttura dei codici	58
4.2.1	Funzione di densità	58
4.2.2	Funzione di ripartizione	59
4.3	Considerazioni finali	60
	Conclusioni	64
	Appendice	67
	Codici Capitolo 3	68
	Normale Asimmetrica univariata	68
	Normale Asimmetrica multivariata	71
	t Asimmetrica univariata	77
	t Asimmetrica multivariata	82
	Codici Capitolo 4 - Implementazione SUN	87
	Funzione dsun (Python)	87
	Funzione psun (Python)	90
	Bibliografia	93
	Ringraziamenti	97

Introduzione

Le distribuzioni asimmetriche nascono dall'esigenza di avere dei modelli capaci di adattarsi meglio ai fenomeni reali: non solo per determinare la legge di dipendenza, ma anche per ricercare le cause delle variazioni e delle modificazioni. Il tutto, preservando proprietà e strutture matematiche accettabili.

Lo scopo di questo elaborato è osservare lo stato attuale della diffusione delle distribuzioni Normale Asimmetrica (SN), t Asimmetrica (ST) e Normale Asimmetrica Unificata (SUN) nel linguaggio Python. Attualmente non esiste un pacchetto pronto all'uso che includa i codici di base relativi a queste distribuzioni, né in ambito univariato né multivariato; in particolare, la SUN risulta quasi del tutto assente sia nei pacchetti che nei repository attualmente disponibili. Come punto di riferimento, stato dell'arte, è stato considerato il pacchetto *sn* di R, sviluppato dal Professor Adelchi Azzalini, dal quale sono state selezionate alcune funzioni fondamentali che un pacchetto Python dovrebbe includere per costruire una base solida nell'analisi dei dati con le tre distribuzioni in esame. L'approccio seguito prevede l'integrazione di diversi codici open source; laddove non fossero disponibili, come nel caso della SUN, se ne propone una versione di base appositamente sviluppata. L'obiettivo è arrivare a definire una collezione di codici in grado di costruire la base per un pacchetto Python che, con il tempo, possa essere ampliato fino a diventare comparabile con *sn* di R.

L'elaborato si propone come strumento a carattere pragmatico, con un'attenzione prevalente agli aspetti di programmazione. La parte teorica non viene trascurata, ma è trattata in maniera essenziale e limitata agli argomenti strettamente rilevanti, senza eccessivi approfondimenti. Le formulazioni considerate fanno esclusivo riferimento agli studi di Azzalini (1985) e ai successivi sviluppi (Azzalini & Capitanio, 2014).

Il Capitolo 1 introduce i concetti teorici alla base delle distribuzioni analizzate, illustrandone gli aspetti generali sia in ambito univariato che multivariato. Sono inoltre presenti alcuni elementi computazionali di base, con particolare attenzione alle differenze tra le distribuzioni più che alla sola trattazione teorica.

Il Capitolo 2 descrive i pacchetti e i repository selezionati per questo elaborato. Per il pacchetto *sn* di R vengono presentati gli aspetti generali, per poi soffermarsi sulle funzioni fondamentali selezionate per ciascuna distribuzione. Per quanto riguarda Python, sono analizzati pacchetti e repository tra i più utilizzati e facilmente integrabili. In chiusura viene discussa e riassunta la situazione dei pacchetti e repository Python rispetto a *sn*.

Il Capitolo 3 è dedicato all'utilizzo pratico delle distribuzioni. Per *sn* sono

riportate analisi con risultati noti, in modo da disporre di una base di confronto per i codici di Python. Successivamente, le stesse analisi vengono replicate in Python impiegando tutti i pacchetti e repository illustrati nel Capitolo 2. L'obiettivo è valutarne principalmente l'usabilità in termini di integrazione, l'affidabilità e la correttezza dei risultati.

Il Capitolo 4 si concentra specificatamente sulla Normale Asimmetrica Unificata. A seguito di varie ricerche è emersa la quasi totale assenza di codice Python disponibile, se non per casi particolari o utilizzi in backend. In questo capitolo si propone una collezione di codici che possa costituire una base da integrare in un pacchetto Python dedicato.

Nell'Appendice sono riportati i codici, in formato sintetizzato, utilizzati nei capitoli. I codici completi sono visionabili su GitHub: github.com/mattiacasetta/skew_distribution_python_tests.git.

Capitolo 1

La Distribuzione Normale Asimmetrica

1.1 La diffusione

I primi contributi a introdurre formulazioni asimmetriche risalgono alla fine dell'Ottocento, con i lavori di Edgeworth (1886) e Pearson (1893, 1895). Una prima famiglia di distribuzioni asimmetriche, ottenuta come perturbazione di distribuzioni simmetriche, fu presentata da Fernando de Helguero al Congresso Internazionale dei Matematici a Roma nel 1908. Gli studi moderni sulle distribuzioni asimmetriche si fondano sull'articolo di Azzalini (1985) che, riprendendo il lavoro di F. de Helguero (1909), elabora la Normale Asimmetrica e, negli anni successivi, ne estende la formulazione al caso multivariato (Azzalini & Dalla Valle, 1996). Nei primi anni Duemila, per rispondere a crescenti esigenze di modellazione e flessibilità, sono state introdotte la *t* Asimmetrica (Azzalini & Capitanio, 2003) e la Normale Asimmetrica Unificata (Arellano-Valle & Azzalini, 2006).

Negli ultimi anni queste distribuzioni hanno avuto diversi sviluppi con applicazioni in diversi ambiti: la Normale Asimmetrica per fenomeni con lievi asimmetrie, come quelli biologici; la *t* Asimmetrica per fenomeni con lievi asimmetrie e code pesanti, ad esempio nell'analisi di eventi meteorologici estremi o di fenomeni finanziari; la Normale Asimmetrica Unificata, utile nei contesti bayesiani (Durante, 2019).

L'introduzione di queste distribuzioni ha consentito sia di migliorare molto la modellazione statistica quando utilizzata come verosimiglianza, che di facilitare l'inferenza bayesiana quando utilizzata come distribuzione a priori / a posteriori per modelli probit; tuttavia, l'inclusione dei parametri che regolano l'asimmetria, complicando la struttura della distribuzione, ha portato a problematiche relative all'identificazione dei parametri, alla complessità computazionale e alla creazione di sistemi stabili ed efficienti.

A oggi, l'impiego di tali distribuzioni non è molto diffuso in ambito industriale, probabilmente a causa della disponibilità limitata di soluzioni software pronte all'uso. Come riportato da Stackoverflow [11], uno dei linguaggi di programmazione più utilizzati negli ultimi anni è Python; tuttavia, al momento non esistono pacchetti Python che trattino le distribuzioni asimmetriche in modo sistematico e paragonabile al pacchetto *sn* di R. Ciò ne limita l'utilizzo e, di conseguenza, l'ulteriore sviluppo di queste distribuzioni nelle applicazioni industriali, inducendo spesso a ricorrere alla classica distribuzione Normale e ad accontentarsi di un adattamento, a volte, solo parziale dei modelli ai dati.

Nei paragrafi successivi verranno definite le distribuzioni Normale Asimmetrica,

t Asimmetrica e Normale Asimmetrica Unificata, anche nel contesto multivariato, facendo riferimento al libro dal titolo THE SKEW-NORMAL AND RELATED FAMILIES (Azzalini & Capitanio, 2014).

1.2 Aspetti teorici

1.2.1 Definizione e proprietà della Normale Asimmetrica

Per introdurre la distribuzione Normale Asimmetrica (SN) è utile richiamare un importante risultato teorico, de Helguero (1909).

Proposizione 1.1 *Sia f_0 una funzione di densità unidimensionale simmetrica in 0, G_0 una funzione di ripartizione continua e simmetrica rispetto a 0 con $G_0(-y) = 1 - G_0(y)$ e $w(x)$ una funzione reale dispari, $w(-x) = -w(x)$, su $\mathbb{R}$. Allora, la funzione*

$$f(x) = 2 f_0(x) G_0\{w(x)\}, \quad x \in \mathbb{R} \quad (1.1)$$

è una funzione di densità.

La proposizione fornisce un criterio generale per costruire una distribuzione asimmetrica a partire da una densità simmetrica, tramite l'applicazione di una perturbazione. In particolare, sostituendo in (1.1) $f_0 = \varphi$ e $G_0 = \Phi$, ossia la densità e la funzione di ripartizione di una distribuzione $N(0, 1)$ e ponendo $w(x) = \alpha x$, $\alpha \in \mathbb{R}$, si ottiene

$$f_{SN}(x; \alpha) = 2 \varphi(x) \Phi\{\alpha x\}, \quad x \in \mathbb{R} \quad (1.2)$$

che rappresenta la formula della distribuzione Normale Asimmetrica.

Generalizzando la funzione di densità con l'introduzione di parametri di posizione ξ e di scala ω , per $x \in \mathbb{R}$ e Z variabile casuale continua con funzione di densità (1.2), per

$$Y = \xi + \omega Z, \quad \xi \in \mathbb{R}, \omega \in \mathbb{R}^+ \quad (1.3)$$

si ottiene la seguente funzione di densità:

$$f_{SN}(x; \xi, \omega, \alpha) = \frac{2}{\omega} \varphi\left(\frac{x - \xi}{\omega}\right) \Phi\left(\alpha \frac{x - \xi}{\omega}\right) \quad (1.4)$$

da cui si può scrivere

$$Y \sim SN(\xi, \omega^2, \alpha).$$

Di seguito si riportano le principali proprietà che rendono interessante l'utilizzo di questa distribuzione, sottolineando i forti legami con la distribuzione Normale:

- a) $SN(\xi, \omega^2, \alpha) = N(\xi, \omega^2)$ per $\alpha = 0$

La distribuzione Normale Asimmetrica include, come caso particolare, la distribuzione Normale. Questa proprietà sottolinea come la famiglia SN sia una naturale estensione della Normale.

- b) Se $Z \sim SN(0, 1, \alpha)$, allora $-Z \sim SN(0, 1, -\alpha)$.

La distribuzione è simmetrica in senso parametrico. Le densità ottenute per valori positivi o negativi di α sono dunque simmetriche rispetto all'origine.

- c) $\lim_{\alpha \rightarrow +\infty} SN(\xi, \omega^2, \alpha) = HN(\xi, \omega^2)$, HN è la distribuzione Mezza Normale.

All'aumentare del parametro di forma, la distribuzione Normale Asimmetrica converge alla distribuzione Mezza Normale, sottolineando il legame tra la famiglia SN e le distribuzioni troncate.

- d) Se $Z \sim SN(\xi, \omega^2, \alpha)$, allora $\forall \xi \in \mathbb{R}, \omega > 0$ si ha che
 $Y = \xi + \omega Z \sim SN(\xi, \omega^2, \alpha)$.

La SN è chiusa rispetto a trasformazioni lineari, presentando caratteristiche pari alla Normale per la costruzione di modelli parametrici e la loro manipolazione.

- e) Momenti e limiti dell'asimmetria

Per semplificare lo studio dei momenti, invece di utilizzare il parametro α si utilizza

$$\delta = \frac{\alpha}{\sqrt{1 + \alpha^2}}, \quad \delta \in (-1, 1)$$

e utilizzando i momenti $\mu_k = \mathbb{E}[(Y - \mu)^k]$ si ottengono

$$\text{Media } \mathbb{E}[Y] = \mu = \xi + \omega \delta \sqrt{\frac{2}{\pi}}$$

$$\text{Varianza } \mathbb{V}[Y] = \sigma^2 = \omega^2 \left(1 - \frac{2\delta^2}{\pi}\right)$$

e gli indici di

Asimmetria: utilizzato per misurare l'asimmetria della distribuzione

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{4 - \pi}{2} \left(\frac{\delta \sqrt{2/\pi}}{\sqrt{1 - 2\delta^2/\pi}} \right)^3, \quad \gamma_1 \in (-0.995, 0.995)$$

Curtosi: indica il grado di appiattimento della distribuzione rispetto alla normale

$$\gamma_2 = \frac{\mu_4}{\sigma^4} - 3 = 2(\pi - 3) \left(\frac{(\delta \sqrt{2/\pi})^2}{1 - 2\delta^2/\pi} \right)^2, \quad \gamma_2 \in (0, 0.869)$$

Da questi indici si nota che la SN non è molto utile per dati con forti asimmetrie o con code pesanti

f) Se $Z \sim SN(0, 1, \alpha)$, Normale Asimmetrica Standard, allora $Z^2 \sim \chi_1^2$.

Essendo SN simmetrica per riflessione per α e $-\alpha$, l'elevazione al quadrato produce la stessa distribuzione. Si ha che l'effetto di α sparisce ottenendo il quadrato di una distribuzione Normale, perciò di una χ^2 .

Per Y generalizzata si ha che $((Y - \xi)/\omega)^2 \sim \chi_1^2$

g) La famiglia SN è unimodale e log-concava per ogni α .

La funzione mantiene l'unimodalità e la log-concavità, poiché la combinazione della distribuzione normale con la funzione di ripartizione della Normale, monotona crescente, permette di mantenere queste proprietà della distribuzione Normale.

Dal punto di vista inferenziale, la Normale Asimmetrica si tratta in modo molto simile alla Normale simmetrica, tenendo conto delle complessità introdotte dall'aggiunta del parametro di forma.

Proposizione 1.2 *Massima verosimiglianza*

Sia $y = (y_1, \dots, y_n)$ un campione i.i.d. da una $SN(\xi, \omega^2, \alpha)$. La funzione di verosimiglianza si scrive come

$$L(\xi, \omega, \alpha; y) = \prod_{i=1}^n \frac{2}{\omega} \varphi\left(\frac{y_i - \xi}{\omega}\right) \Phi\left(\alpha \frac{y_i - \xi}{\omega}\right) \quad (1.5)$$

da cui deriva la log-verosimiglianza

$$\ell(\xi, \omega, \alpha) = \sum_{i=1}^n \log \left[\frac{2}{\omega} \varphi\left(\frac{y_i - \xi}{\omega}\right) \Phi\left(\alpha \frac{y_i - \xi}{\omega}\right) \right] \quad (1.6)$$

La log-verosimiglianza, scritta con questa parametrizzazione, presenta delle problematiche: per $\alpha = 0$ la matrice di informazione di Fisher è singolare e la verosimiglianza profilo presenta un punto stazionario; inoltre, per valori di α estremi possono presentarsi soluzioni sul bordo dello spazio parametrico. Per migliorare la stabilità numerica si possono utilizzare delle parametrizzazioni alternative, per esempio, sostituendo (ξ, ω, α) con i loro valori derivati dai momenti (μ, σ, γ_1) .

Per $\alpha \neq 0$ e sotto condizioni di regolarità, le stime di massima verosimiglianza risultano consistenti e asintoticamente normali, cioè $\hat{\theta} \sim N(\theta, I(\theta)^{-1})$, con $\theta = (\xi, \omega, \alpha)$ e $I(\theta)$ matrice di informazione di Fisher.

La distribuzione Normale Asimmetrica risulta quindi un'ottima estensione della Normale simmetrica, mantenendo gran parte della proprietà matematiche e migliorandone la flessibilità nella rappresentazione delle asimmetrie.

1.2.2 t Asimmetrica

Dati i limiti della Normale Asimmetrica nel gestire distribuzioni con valori di asimmetria e curtosi elevati e risultando poco adatta a modellare dati con code pesanti, si è ricercata un'estensione che superasse questi limiti. La proposta della distribuzione t Asimmetrica (ST), formulata da Azzalini & Capitanio (2003), combina la capacità di modellare le asimmetrie della Normale Asimmetrica con la flessibilità della t di Student per la gestione delle code pesanti. La t Asimmetrica viene presentata come estensione della Normale Asimmetrica, mantenendo la stessa logica costruttiva, preservando molte proprietà formali ed estendendo il campo applicativo.

Proposizione 1.3 *Se $Z \sim SN(0, 1, \alpha)$ e $V \sim \chi^2_\nu/\nu$ sono indipendenti, con $\alpha \in \mathbb{R}$ e $\nu > 0$, allora*

$$Y = \frac{Z}{\sqrt{V}} \sim ST(\alpha, \nu),$$

cioè Y ha una distribuzione t Asimmetrica con parametri α e ν .

La funzione di densità univariata risulta

$$f(y; \alpha, \nu) = 2 t_\nu(y) T_{\nu+1} \left(\alpha y \sqrt{\frac{\nu+1}{\nu+y^2}} \right), \quad y \in \mathbb{R}, \quad (1.7)$$

dove:

- $t_\nu(y)$ è la densità della t di Student con ν gradi di libertà;
- $T_{\nu+1}(\cdot)$ è la funzione di ripartizione della t di Student con $\nu+1$ gradi di libertà.

Generalizzando, per $z = (y - \xi)/\omega$ con $\xi \in \mathbb{R}$ e $\omega > 0$ si ottiene

$$f(y; \xi, \omega, \alpha, \nu) = \frac{2}{\omega} t_\nu(z) T_{\nu+1} \left(\alpha z \sqrt{\frac{\nu+1}{\nu+z^2}} \right) \quad (1.8)$$

si scrive $Y \sim ST(\xi, \omega^2, \alpha, \nu)$.

Per quanto riguarda le proprietà, la t Asimmetrica è stata costruita per mantenere le proprietà della Normale Asimmetrica tra cui:

- Per $\alpha = 0$ si ottiene la distribuzione simmetrica;

- Se $Y \sim ST(\alpha, \nu)$, allora $-Y \sim ST(-\alpha, \nu)$. Mantiene la simmetria parametrica;
- Se $Y \sim ST(\xi, \omega^2, \alpha, \nu)$, allora per $\tilde{Y} = a + bY$ con $b > 0$, si ottiene $\tilde{Y} \sim ST(a + b\xi, b\omega^2, \alpha, \nu)$. Mantiene la chiusura per trasformazioni lineari;
- ST è unimodale ma non log-concava.

Mentre guadagna le seguenti proprietà:

- Gestione delle code pesanti tramite il parametro ν . Per $\nu \rightarrow \infty$ ST converge alla SN;
- I momenti esistono per determinate condizioni di ν :
 - Per $\nu > 1$ esiste il momento primo;
 - Per $\nu > 2$ esiste il momento secondo;
 - Per $\nu > 3$ esiste l'indice di asimmetria;
 - Per $\nu > 4$ esiste l'indice di curtosi;
- Se i momenti esistono gli indici di asimmetria e curtosi possono assumere valori arbitrariamente grandi.

La t Asimmetrica si presenta come una distribuzione molto più flessibile della Normale Asimmetrica per trattare dati con forte asimmetria e code pesanti anche se introduce delle complicazioni, come l'esistenza dei momenti in funzione dei gradi di libertà, che richiedono un'attenta valutazione prima dell'effettiva scelta tra SN e ST.

Dal punto di vista inferenziale, la distribuzione t Asimmetrica, come per la Normale Asimmetrica, presenta caratteristiche molto simili alla sua versione simmetrica. Si ha un incremento di complessità nel trattarla, anche rispetto alla Normale Asimmetrica, per l'aggiunta del parametro ν per la gestione delle code pesanti. A differenza della Normale Asimmetrica, per $\alpha = 0$ la matrice di informazione di Fisher non è singolare, pertanto è possibile utilizzare il classico test del rapporto di verosimiglianza $H_0 : \alpha = 0$ contro $H_1 : \alpha \neq 0$ assumendo $2\{\ell(\hat{\theta}) - \ell(\theta_0)\} \sim \chi_1^2$.

Proposizione 1.4 *Massima verosimiglianza*

Sia $y = (y_1, \dots, y_n)$ un campione i.i.d. da una $ST(\xi, \omega^2, \alpha, \nu)$. La funzione di verosimiglianza si scrive come

$$L(\xi, \omega, \alpha, \nu; y) = \prod_{i=1}^n \left[\frac{2}{\omega} t_\nu \left(\frac{y_i - \xi}{\omega} \right) T_{\nu+1} \left(\alpha \frac{y_i - \xi}{\omega} \sqrt{\frac{\nu+1}{\nu + \left(\frac{y_i - \xi}{\omega} \right)^2}} \right) \right] \quad (1.9)$$

dove $t_\nu(\cdot)$ è la densità della t di Student univariata con ν gradi di libertà e $T_{\nu+1}(\cdot)$ la funzione di ripartizione della t con $\nu+1$ gradi di libertà.

La log-verosimiglianza risulta

$$\ell(\xi, \omega, \alpha, \nu) = \sum_{i=1}^n \log \left[\frac{2}{\omega} t_\nu \left(\frac{y_i - \xi}{\omega} \right) T_{\nu+1} \left(\alpha \frac{y_i - \xi}{\omega} \sqrt{\frac{\nu+1}{\nu + \left(\frac{y_i - \xi}{\omega} \right)^2}} \right) \right] \quad (1.10)$$

L'ottimizzazione numerica risulta più onerosa rispetto alla Normale Asimmetrica, oltre alla presenza di massimi locali, il parametro ν rende più complessa la convergenza degli algoritmi.

Poiché i momenti della t Asimmetrica non sempre esistono, la stabilità inferenziale dipende molto dal valore stimato di ν .

Sotto condizioni di regolarità e per ν sufficientemente grande le stime di massima verosimiglianza risultano consistenti e asintoticamente normali.

1.2.3 Estensioni multivariate

Subito dopo l'introduzione delle forme univariate si è presentato il problema di trattare modelli che presentano variabili correlate affette da asimmetria. Sono state proposte le versioni della distribuzione Normale Asimmetrica multivariata da Azzalini & Dalla Valle nel 1996 e la versione della distribuzione t Asimmetrica multivariata da Azzalini & Capitanio nel 2003.

Proposizione 1.5 *Data una variabile casuale $Z \in \mathbb{R}^k$, si dice Normale Asimmetrica multivariata la distribuzione*

$$f_{SN_k}(z; \bar{\Omega}, \alpha) = 2\varphi_k(z; \bar{\Omega})\Phi(\alpha^T z), \quad z \in \mathbb{R}^k \quad (1.11)$$

dove

- $\varphi_k(z; \bar{\Omega})$ è la densità di una distribuzione multivariata $N_k(0, \bar{\Omega})$ con $\bar{\Omega}$ matrice di correlazione;
- $\Phi(\cdot)$ è la funzione di ripartizione della normale standard univariata;

- $\alpha \in \mathbb{R}^k$ è il vettore dei parametri di forma.

Introducendo i parametri di posizione $\xi \in \mathbb{R}^k$ e scala $\Omega = \omega \bar{\Omega} \omega$ matrice di varianza e covarianza di rango pieno, si ottiene $Y = \xi + \omega Z$ con distribuzione:

$$f_Y(y) = 2 \varphi_k(y; \xi, \Omega) \Phi(\alpha^\top \omega^{-1}(y - \xi)). \quad (1.12)$$

da cui si ottiene che $Y \sim SN_k(\xi, \Omega, \alpha)$

Proposizione 1.6 *Data una variabile casuale $Y \in \mathbb{R}^d$, si ha una distribuzione del tipo t Asimmetrica Multivariata se Y ha densità del tipo:*

$$f_Y(y) = 2t_d(y; \xi, \Omega, \nu) T_{\nu+d} \left(\alpha^\top \omega^{-1}(y - \xi) \sqrt{\frac{\nu + d}{Q_y + \nu}} \right), \quad y \in \mathbb{R}^d \quad (1.13)$$

dove:

- $t_d(y; \xi, \Omega, \nu)$ è la densità della distribuzione t di Student multivariata con
 - $\xi \in \mathbb{R}^d$ parametro di posizione
 - Ω matrice di scala definita positiva
 - $\nu > 0$ gradi di libertà
- $T_{\nu+d}(\cdot)$ è la funzione di ripartizione della T di Student univariata con $\nu + d$ gradi di libertà
- $\alpha \in \mathbb{R}^d$ è il vettore dei parametri di forma
- ω è la matrice diagonale associata alla scala delle variabili
- $Q_y = (y - \xi)^\top \Omega^{-1}(y - \xi)$

Si può sinteticamente scrivere $Y \sim ST_d(\xi, \Omega, \alpha, \nu)$.

Le versioni multivariate della Normale Asimmetrica e della t Asimmetrica si presentano come le naturali estensioni delle rispettive forme univariate, costruite secondo il principio di perturbazione della distribuzione simmetrica.

In entrambe le famiglie, l'introduzione del vettore di forma $\alpha \in \mathbb{R}^d$ consente di modellare direzioni multiple di asimmetria, descrivendo situazioni complesse di dipendenza tra le variabili.

Le proprietà di base, come la riduzione alla distribuzione simmetrica per $\alpha = 0$ e la chiusura rispetto a trasformazioni lineari, restano valide. Tuttavia, l'utilizzo di una

matrice di covarianza Ω e di α in forma vettoriale, rende la parametrizzazione più complessa e la superficie di log-verosimiglianza più irregolare. Anche in questo caso, per la Normale Asimmetrica multivariata con $\alpha = 0$ si ha la matrice di informazione attesa singolare.

Dal punto di vista inferenziale, le metodologie di stima rimangono basate sul metodo della Massima Verosimiglianza, nonostante la massimizzazione per queste funzioni risulti molto onerosa. Le proprietà asintotiche sono analoghe alle versioni unidimensionali: sotto opportune condizioni di regolarità gli stimatori sono consistenti e asintoticamente normali, con matrice di covarianza ottenibile dall'informazione di Fisher.

Dal punto di vista pratico, le versioni multivariate ampliano il campo applicativo, ma richiedono modelli più complessi con un incremento significativo del costo computazionale.

1.2.4 Normale Asimmetrica Unificata (SUN)

Anche se già presentavano ottime proprietà, le distribuzioni SN e ST rimanevano ancorate a costruzioni semplici e non sempre riconducibili a una formulazione generalizzata.

Arellano-Valle & Azzalini (2006) introducono la distribuzione Normale Asimmetrica Unificata per riunire, sotto un'unica struttura parametrica, diverse estensioni asimmetriche della Normale, tra cui la versione estesa e quella ottenuta tramite selezione di variabili latenti. Sostituendo l'assunzione di Normalità con una distribuzione ellittica si ottiene la corrispondente classe unificata a base ellittica, che comprende le versioni asimmetriche a base ellittica inclusa la variante t Asimmetrica. La SUN si presenta come un'estensione della Normale Asimmetrica (o t Asimmetrica), fondata sullo stesso principio di perturbazione di una distribuzione simmetrica mediante una funzione di ripartizione Normale. La struttura è però arricchita da una componente multivariata in grado di rappresentare coerentemente diverse fonti di asimmetria e dipendenza.

Proposizione 1.7 *Si considera un vettore Normale congiunto*

$$\begin{pmatrix} U_0 \\ U_1 \end{pmatrix} \sim N_{m+d} \left(0, \begin{pmatrix} \Gamma & \Delta^T \\ \Delta & \Omega \end{pmatrix} \right),$$

dove $U_0 \in \mathbb{R}^m$ rappresenta la componente multivariata e $U_1 \in \mathbb{R}^d$ la variabile osservata. Data una variabile casuale $Z \in \mathbb{R}^d$, si dice Normale Asimmetrica

Unificata (SUN) la distribuzione definita dalla seguente funzione di densità

$$f_Z(z) = \phi_d(z - \xi; \Omega) \frac{\Phi_m(\tau + \Delta^T \bar{\Omega}^{-1} \omega^{-1}(z - \xi); \Gamma - \Delta^T \bar{\Omega}^{-1} \Delta)}{\Phi_m(\tau; \Gamma)}, \quad z \in \mathbb{R}^d \quad (1.14)$$

dove:

- $\phi_d(\cdot; \Omega)$ è la densità della distribuzione normale multivariata $N_d(0, \Omega)$;
- $\Phi_m(\cdot; \Gamma)$ è la funzione di ripartizione della normale multivariata standard di dimensione m e matrice di correlazione Γ ;
- $\xi \in \mathbb{R}^d$ è il vettore dei parametri di posizione;
- Ω è la matrice di varianza e covarianza $d \times d$, simmetrica e definita positiva;
- Δ è la matrice dei parametri di forma $d \times m$, che controlla direzione e intensità dell'asimmetria;
- $\tau \in \mathbb{R}^m$ è il vettore di soglia che definisce la regione di troncamento latente;
- Γ è la matrice di correlazione $m \times m$ associata alla componente latente;
- $\bar{\Omega}$ è la matrice di correlazione (definita positiva) associata a Ω ;
- $\omega = \text{diag}(\sqrt{\Omega_{11}}, \dots, \sqrt{\Omega_{dd}})$ è la matrice diagonale delle deviazioni standard e vale $\Omega = \omega \bar{\Omega} \omega$.

Si indica sinteticamente

$$Z \sim SUN_{d,m}(\xi, \Omega, \Delta, \tau, \Gamma).$$

Questa scrittura rappresenta la distribuzione di una Normale multivariata condizionata all'evento $U_0 + \tau > 0$ (condizionamento applicato componente per componente), in presenza di correlazione tra le componenti U_0 e quelle osservate, e include come casi particolari:

- la Normale Asimmetrica multivariata per $m = 1$, $\tau = 0$, $\Gamma = 1$;
- la Normale multivariata per il caso limite $\Delta = 0$;
- la t multivariata cambiando da base Normale a base ellittica;

Le proprietà fondamentali della SUN, per la sua costruzione, estendono quelle già viste per SN e ST. La distribuzione mantiene la simmetria parametrica per cui $Z \sim SUN_{d,m}(\xi, \Omega, \Delta, \tau, \Gamma)$ implica $-Z \sim SUN_{d,m}(-\xi, \Omega, -\Delta, -\tau, \Gamma)$ e risulta chiusa rispetto a trasformazioni lineari: se $Y = a + BZ$, allora

$$Y \sim SUN_{q,m}(a + B\xi, B\Omega B^T, B\Delta, \tau, \Gamma).$$

Un aspetto molto importante per questa distribuzione riguarda la chiusura rispetto al condizionamento multiplo e alla marginalizzazione. Nelle distribuzioni simmetriche classiche, come la Normale, il condizionamento su una parte del vettore casuale produce una distribuzione appartenente alla stessa famiglia. Questa proprietà di chiusura non viene rispettata appieno dalle distribuzioni asimmetriche come SN e ST.

La SUN rispetta pienamente questa caratteristica. Se si considerano più variabili casuali congiuntamente distribuite secondo una SUN e si condiziona su un sottoinsieme di esse, la distribuzione condizionata risultante rimane una SUN.

Un altro aspetto importante è dato dalla maggior flessibilità della struttura parametrica. Dove SN e ST descrivono l'asimmetria tramite un singolo parametro vettoriale, che agisce in una sola direzione, la SUN utilizza una matrice di parametri di forma, Δ , e una matrice di correlazione, Γ . La prima gestisce l'asimmetria in più direzioni, la seconda regola la correlazione tra le variabili che determinano l'asimmetria. In questo modo la SUN riesce a gestire configurazioni di asimmetria molto più complesse rispetto alle distribuzioni precedenti, rendendola una buona generalizzazione delle distribuzioni SN e ST.

Dal punto di vista inferenziale, la SUN conserva molte caratteristiche derivanti da SN e ST. Grazie alla componente $(U_0, U_1)^T$ è possibile esprimere la SUN come una Normale multivariata condizionata e ciò consente di adottare metodi di stima e inferenza basati su algoritmi validi per la Normale.

1.3 Aspetti computazionali

In questo paragrafo si affrontano due aspetti computazionali molto importanti per le distribuzioni asimmetriche: la simulazione dei dati, molto utilizzata in contesti applicativi dove sono presenti dei dati con delle asimmetrie intrinseche nella loro natura; la stima dei parametri, che è il collo di bottiglia a livello numerico soprattutto nei casi multivariati.

La struttura delle famiglie Normale Asimmetrica, t Asimmetrica e Normale Asimmetrica Unificata basa la generazione su variabili latenti per agevolare la simulazione dei dati. Questo approccio però presenta anche delle complicazioni quando si effettua la stima dei parametri: le funzioni di verosimiglianza associate a queste famiglie presentano una complessità computazionale che ne limita la scalabilità.

1.3.1 Simulazione di dati

La generazione di dati dalle famiglie Normale Asimmetrica, t Asimmetrica e SUN si basa sulla perturbazione delle rispettive famiglie simmetriche tramite due meccanismi principali:

- Selezione.
Viene generato un vettore gaussiano (o ellittico) aumentato con vincoli di segno sulle variabili latenti.
- Additivo.
Si esprime la variabile asimmetrica come combinazione lineare di una componente gaussiana e del valore assoluto di una Normale, con pesi che dipendono dai parametri (applicabile anche con componente ellittica).

Anche se entrambe le strade risultano valide, l'efficienza relativa dipende dal contesto numerico e dallo specifico modello, non è possibile selezionare una scelta migliore a priori. È necessario prestare attenzione ai metodi di accetto-rifiuto che, in media, possono scartare la metà dei campioni.

Le tre famiglie di distribuzioni condividono alcune proprietà computazionali utili alla simulazione:

- Chiusura per trasformazioni affini.
Simulata la forma standardizzata, i parametri (ξ, Ω) si applicano alla forma standard per generare la trasformazione d'interesse;
- Forme canoniche.
Per la Normale Asimmetrica esiste una trasformazione lineare che concentra l'asimmetria in al più una componente; risulta utile per ridurre i gradi di libertà effettivi dell'asimmetria;
- Estensioni ellittiche e miscele di scala.
Utilizzando una base ellittica (t di Student), la distribuzione asimmetrica è

spesso ottenibile come miscela di dati di una SN. Questo lega direttamente SN e ST.

Per ogni famiglia si presentano poi degli aspetti peculiari che possono fungere da discriminante nella scelta dell'algoritmo.

Normale Asimmetrica

Facendo riferimento all'equazione (1.2) per la Normale Asimmetrica ottengo i due metodi di selezione:

- Selezione.

Dato $(X_0, X)^\top \sim N_{1+k}(0, \Sigma^*)$, con X_0 scalare e $X \in \mathbb{R}^k$. Definire

$$Z = \begin{cases} X & \text{se } X_0 > 0, \\ -X & \text{altrimenti;} \end{cases} \Rightarrow Z \sim SN_k(\cdot),$$

con parametri $(\bar{\Omega}, \alpha)$ determinati dai blocchi di Σ^* secondo le relazioni fornite da Azzalini & Capitanio (2014). La versione con posizione e scala $Y = \xi + \omega Z$ è adatta alla simulazione multivariata.

- Additiva.

Per il caso univariato si può utilizzare $Z = a U_0 + b |U_1|$ con $U_0, U_1 \stackrel{\text{ind}}{\sim} N(0, 1)$ e $a^2 + b^2 = 1$, b funzione del parametro di forma.

Per il caso multivariato $Z = D_\delta U_0 + \delta |U_1|$, dove $U_0 \sim N_d(0, \Psi)$, $U_1 \sim N(0, 1)$, D_δ è diagonale e $\delta \in \mathbb{R}^d$ regola la direzione e l'intensità dell'asimmetria. Anche in questo caso posso usare $Y = \xi + \omega Z$, questa formulazione evita il rifiuto di troppe unità; è spesso preferibile in simulazione.

Dato che gli algoritmi basati sul sistema di accetto/rifiuto, in media scartano il 50% dei campioni, si preferisce utilizzare forme additive o, in qualche modo, senza la componente di rifiuto.

In generale non esiste una scelta migliore a priori, bisogna valutare l'effettiva complessità di calcolo delle funzioni sulla base del problema da risolvere.

t Asimmetrica

Riprendendo la struttura della t Asimmetrica ottengo, sia per il caso univariato che per quello multivariato, come struttura:

$$Y = \frac{Z}{\sqrt{V}}, \quad Z \sim SN(\cdot), \quad V \sim \chi_\nu^2/\nu, \quad Z \perp V$$

Generando V e Z come descritto, con eventuale trasformazione di posizione e scala, si possono generare dei dati da una distribuzione ST, mantenendo le principali proprietà della SN e le proprietà di gestione delle code della ST. Inoltre si ottiene che esistono i momenti fino al quarto ordine.

Per la generazione di Z si fa riferimento a quello che è stato scritto nel paragrafo precedente per la Normale Asimmetrica.

Normale Asimmetrica Unificata

La SUN, definita in (1.14) come prodotto tra la densità gaussiana e la funzione di ripartizione Normale multivariata, come per la Normale Asimmetrica, presenta due modalità:

- Selezione.

Considerato $\tilde{\beta} \in \mathbb{R}^p$ e $\tilde{z} \in \mathbb{R}^m$ standardizzati con matrice di correlazione a blocchi Σ^* (blocchi $\bar{\Omega}$, Γ e Δ come in (1.14)). Allora

$$\bar{\beta} = (\tilde{\beta} \mid \tilde{z} + \tau > 0) \sim SUN_{p,m}(0, \bar{\Omega}, \Delta, \tau, \Gamma), \quad Z = \xi + \omega \bar{\beta}$$

fornisce campioni dalla SUN. Operativamente, la parte critica è la generazione di $\tilde{z}$ troncata. Per dimensioni moderate la generazione dalla Normale troncata risulta essere il collo di bottiglia, esistono diversi metodi per ridurre l'impatto ma vanno scelti in funzione del problema che si ha di fronte.

- Additiva.

Vengono utilizzate combinazioni lineari di gaussiane e normali troncate, utili per costruire generatori *i.i.d.* e per dare maggiore importanza ai parametri $(\xi, \Omega, \Delta, \tau, \Gamma)$ nel meccanismo generativo.

Data la natura della SUN, che nasce come unificazione delle famiglie gaussiane asimmetriche, conserva molte proprietà tipiche gaussiane che agevolano la simulazione tramite trasformazioni affini e decomposizioni a blocchi. Data la struttura molto complessa, il costo computazionale per utilizzare questa famiglia è molto elevato, rendendone l'utilizzo difficilmente scalabile.

Sostituendo la base normale con una ellittica si giunge alle stesse conclusioni potendo utilizzare anche una distribuzione t Asimmetrica.

1.3.2 Stima dei parametri

L'inferenza per le famiglie Normale Asimmetrica, t Asimmetrica (e Normale Asimmetrica Unificata) riprende la formulazione classica della massima

verosimiglianza introducendo specifiche numeriche legate alla funzione di ripartizione nella densità. Questo aspetto provoca due conseguenze principali: la curvatura locale della log-verosimiglianza è più irregolare rispetto ai modelli simmetrici; la valutazione e la derivazione della funzione di ripartizione, soprattutto nel multivariato, rappresenta un problema computazionale non trascurabile.

In generale, sia $y = (y_1, \dots, y_n)$ un campione *i.i.d.*, indicando con θ il vettore dei parametri (in base alla distribuzione da utilizzare), la log-verosimiglianza ha la classica struttura

$$\ell(\theta) = \sum_{i=1}^n \log f(y_i; \theta),$$

dove $f(\cdot; \theta)$ è la funzione di densità specificata nei paragrafi precedenti ((1.2),(1.7),(1.14)). In tutti i casi, lo score coinvolge derivate di $\log \Phi(\cdot)$, $\log T_\nu(\cdot)$ o $\log \Phi_m(\cdot)$; il loro calcolo efficiente è l'aspetto computazionale più rilevante. Prima di scegliere come affrontare il calcolo di queste funzioni rimane importante definire bene il problema sul quale si sta lavorando per capire quali approcci scegliere, quali scartare e come adattarli; aspetti importanti sono l'inizializzazione dei parametri e l'utilizzo di parametrizzazioni alternative.

Analizzando, brevemente, nello specifico le tre famiglie:

Normale Asimmetrica

Con la parametrizzazione diretta riportata in (1.6) i due aspetti critici fondamentali sono:

- i $\alpha = 0$ che rende la matrice attesa di Fisher singolare e la log-verosimiglianza profilo presenta un punto stazionario;
- ii per valori di α molto grandi si osservano soluzioni sul bordo dello spazio parametrico.

Questi aspetti rendono i metodi basati su Newton sensibili alla scelta della parametrizzazione. Utilizzando parametrizzazioni centrate è possibile regolarizzare la curvatura locale e ottenere maggiore stabilità numerica.

t Asimmetrica

La parametrizzazione riportata in (1.10), estendibile al caso multivariato, risolve i problemi legati ad $\alpha = 0$. Utilizza un parametro in più, ν , per la gestione delle code pesanti, che rende l'ottimizzazione più onerosa computazionalmente; una soluzione

per arginare il problema può essere quella di profilare la verosimiglianza rispetto a ν per stabilizzare l'ottimizzazione. L'impatto di ν si estende anche all'esistenza dei momenti, dove solo se $\nu \geq 4$ si possono ricavare media, varianza, asimmetria e curtosi.

Normale Asimmetrica Unificata

Dalla parametrizzazione riportata in (1.14) si nota che sia lo score che la matrice d'informazione coinvolgono valutazioni ripetute di Φ_m e delle sue derivate; questo passaggio risulta essere il più oneroso in termini computazionali limitando la scalabilità. Sono presenti principalmente due approcci:

- Approccio diretto.
Massimizzazione numerica con valutazioni di Φ_m sfruttando metodi Newton e parametrizzazioni stabili per Ω . La qualità delle stime dipende dalla difficoltà di Φ_m .
- Approccio con variabili latenti. Si sfruttano delle variabili latenti per lavorare su distribuzioni più complete; sono richiesti dei troncamenti di distribuzioni multivariate che non riducono sensibilmente il costo computazionale.

Anche in questo caso l'approccio migliore in assoluto non esiste e va formulato un approccio in funzione del problema da affrontare.

La stima dei parametri tramite distribuzioni asimmetriche è un complesso compromesso tra parametrizzazioni stabili, algoritmi di ottimizzazione robusti e la scelta accurata di come trattare la funzione di ripartizione. La scelta di una distribuzione piuttosto che un'altra prevede la conoscenza approfondita del problema in questione in modo da poterlo approcciare con la soluzione ottimale.

Capitolo 2

Implementazioni software

2.1 La situazione generale

In questo capitolo si presenta una rassegna delle principali implementazioni software, in R e in Python, delle distribuzioni Normale Asimmetrica (SN), *t* Asimmetrica (ST) e Normale Asimmetrica Unificata (SUN), con rimando alle definizioni e alle proprietà descritte nel capitolo precedente.

Lo stato dell'arte nella gestione delle distribuzioni asimmetriche è rappresentato dal pacchetto *sn* di R, sviluppato da Adelchi Azzalini, attualmente l'unico ad avere una gestione completa e aggiornata. Per le distribuzioni SN e ST sono presenti implementazioni stabili e robuste sia in ambito multivariato che univariato; per la SUN, pur avendo un'implementazione stabile, risulta meno immediata da utilizzare e molto onerosa computazionalmente.

Per il linguaggio Python non è attualmente presente un pacchetto paragonabile a *sn*; si trovano pacchetti e repository che coprono parzialmente le funzionalità di base per SN e ST, mentre per la SUN il supporto è molto limitato. I criteri di selezione adottati sono stati: l'utilizzo delle formulazioni di Azzalini; la reperibilità in siti web affidabili e di uso comune (pypi.org, github.com); aggiornati recentemente; facilmente integrabili e utilizzabili.

2.2 Stato dell'arte in R: pacchetto *sn*

Sviluppato e pubblicato per la prima volta da Azzalini nel 1998, il pacchetto *sn* rappresenta il riferimento più completo e maturo per la gestione delle famiglie Normale Asimmetrica, *t* Asimmetrica e Normale Asimmetrica Unificata; l'ultima data di aggiornamento risale a marzo 2021, anno in cui è stato introdotto il supporto anche per la SUN. Lo scopo del pacchetto, riportando le parole tradotte dal manuale, è:

"Costruire e manipolare distribuzioni di probabilità della famiglia skew-normale e alcune famiglie correlate, in particolare le famiglie skew-*t* e SUN. Per le distribuzioni skew-normale e skew-*t*, vengono forniti metodi statistici per l'adattamento dei dati e la diagnostica dei modelli, nel caso univariato e multivariato."

I codici contenuti nel pacchetto fanno riferimento direttamente alla teoria del libro di testo *THE SKEW-NORMAL AND RELATED FAMILIES* (Azzalini & Capitanio, 2014), utilizzata anche per la redazione del Capitolo 1 di questo elaborato.

Ai fini di presentare una proposta per la creazione di una solida base per un pacchetto Python, vengono riportate, per le tre famiglie SN, ST, SUN (sia in ambito univariato che multivariato), solo le funzioni fondamentali: densità; ripartizione; quantile; generazione di numeri casuali. L'analisi si concentra sulle funzioni che adottano la parametrizzazione classica (ξ, ω, α) . Si nota che la funzione quantile multivariata non è prevista; si preferisce ricorrere al calcolo dei quantili marginali.

Normale Asimmetrica

- Funzione di densità.

Univariato: **dsn** $(x, \xi, \omega, \alpha, \tau = 0)$, con riferimento alla (1.4)

Multivariato: **dmsn** $(x, \xi, \Omega, \alpha, \tau = 0)$, con riferimento alla (1.12)

Ponendo $\tau \neq 0$ è possibile lavorare con la Normale Asimmetrica Estesa.

- Funzione di ripartizione.

psn $(x, \xi, \omega, \alpha, \tau = 0, engine = "T.Owen")$ calcola $F_{SN}(x)$. Per $\tau = 0$ si ha $engine == "T.Owen"$ funzione di Owen $T(h, a)$, che consente di esprimere la funzione di distribuzione in forma chiusa come $\Phi(z) - 2T(z, \alpha)$. Per $\tau \neq 0$ o sono presenti problemi di condizionamento, si utilizza la distribuzione di probabilità bivariata.

pmsn $(x, \xi, \Omega, \alpha, \tau = 0)$ riporta la funzione di ripartizione a una Normale multivariata di dimensione $d + 1$, con matrice a blocchi derivata da Ω e α .

- Funzione quantile.

qsn $(p, \xi, \omega, \alpha, \tau = 0, solver = NR/RFB)$, inverte numericamente **psn** utilizzando l'algoritmo di Newton-Raphson (NR) o, in caso di instabilità di NR, utilizza dei metodi di bisezione (RFB).

- Generazione casuale.

rsn $(n, \xi, \omega, \alpha, \tau = 0)$, versione univariata, utilizza la rappresentazione additiva:

Per $\tau = 0$, si usa $\delta = \alpha / \sqrt{1 + \alpha^2}$ che genera $Z = \delta |U_1| + \sqrt{1 - \delta^2} U_0$, quindi $Y = \xi + \omega Z$;

Per $\tau \neq 0$ si sostituisce $|U_1|$ con una Normale troncata $T \sim \mathcal{N}(0, 1) \mid T > -\tau$.

rmsn $(n, \xi, \Omega, \alpha, \tau = 0)$, versione multivariata, estende la rappresentazione a $U_0 \sim \mathcal{N}_d(0, \Psi)$ (via Cholesky), $Z = D_\delta U_0 + \delta |U_1|$ (o δT), quindi trasformazione affine $Y = \xi + \omega Z$. L'utilizzo di questa strategia a selezione additiva riduce l'inefficienza dei sistemi accetto/rifiuto.

t Asimmetrica

- Funzione di densità.

Univariato: $\text{dst}(x, \xi, \omega, \alpha, \nu)$, con riferimento alla (1.8)

Multivariato: $\text{dmst}(x, \xi, \Omega, \alpha, \nu)$, con riferimento alla (1.13)

- Funzione di ripartizione.

$\text{pst}(x, \xi, \omega, \alpha, \nu)$ calcola $F_{ST}(x)$ scegliendo, in base ai parametri, il metodo migliore tra quelli disponibili. Per ν molto grandi ($> 10^4$) è consigliato utilizzare la SN.

$\text{pmst}(x, \xi, \Omega, \alpha, \nu)$ valuta $F_{ST}(x)$ come una probabilità t multivariata. Il costo computazionale è molto elevato.

- Funzione quantile.

$\text{qst}(p, \xi, \omega, \alpha, \nu)$, inverte numericamente pst .

- Generazione casuale.

$\text{rst}(n, \xi, \omega, \alpha, \nu)$ e $\text{rmst}(n, \xi, \Omega, \alpha, \nu)$ utilizzano uno schema di miscela di scala: viene generata una componente da una Normale Asimmetrica che viene scalata tramite una variabile χ_ν^2/ν alla quale viene applicata, infine, una trasformazione (es. lineare).

Da notare:

- i) La convergenza di ST a SN per $\nu \rightarrow \infty$ è utile, in ambito operativo, per inizializzare i parametri.
- ii) La scelta di ν incide su stabilità e tempi delle varie funzioni.
- iii) Nel multivariato si hanno costi computazionali importanti anche per dimensioni contenute (es. $d > 20$).

Normale Asimmetrica Unificata

- Funzione di densità.

$\text{dsun}(x, \xi, \Omega, \Delta, \tau, \Gamma, dp = \text{NULL}, \log = \text{FALSE})$, con riferimento alla (1.14).

Restituisce la log-densità con $\log = \text{TRUE}$. Vincoli: $\Omega \succ 0$; coerenza dimensionale $d \times m$ per Δ , $m \times m$ per Γ .

- Funzione di ripartizione.

$\text{psun}(x, \xi, \Omega, \Delta, \tau, \Gamma, dp = \text{NULL})$ calcola $F_{SUN}(x)$ riducendo il calcolo dell'integrale della (1.14) a probabilità Normali multivariate.

- Generazione casuale.

`rsun`($n = 1, \xi, \Omega, \Delta, \tau, \Gamma, dp = NULL$) genera da una $Y \sim \text{SUN}_{d,m}(\xi, \Omega, \Delta, \tau, \Gamma)$ tramite metodi di selezione; condizionando una Normale con $U_0 + \tau > 0$ e tramite additività, combinazioni di Normali e Normali Troncate. Il costo computazionale è per la maggior parte dato dal calcolo della Normale troncata multivariata.

2.3 Alternative Python

Di seguito si riportano i pacchetti e i repository selezionati, presentati con una sintesi dei contenuti e delle basi teoriche di riferimento. Per visionare i codici sono presenti i link alle pagine web dedicate.

2.3.1 SciPy

Pacchetto Python che tratta la distribuzione Normale Asimmetrica univariata utilizzando la formulazione teorica di Azzalini & Capitanio (1999); link al sito presente in bibliografia [7]. Al momento della stesura dell'elaborato l'ultima versione disponibile è la **1.16.2** rilasciata in data 11 Settembre 2025.

Coerentemente con quanto scritto per *sn*, riporto solo le funzioni di base propedeutiche allo sviluppo di un pacchetto Python integrato. I parametri utilizzati sono $\alpha = a$, $loc = \xi$ e $scale = \omega$.

- Funzione di densità.

`pdf`($x, a, loc = 0, scale = 1$) l'implementazione fa riferimento direttamente a (1.4).

- Funzione di ripartizione.

`cdf`($x, a, loc = 0, scale = 1$) valuta la probabilità cumulata usando il modulo `Boost.Math` per avere più stabilità numerica anche con probabilità estreme.

- Funzione quantile.

`ppf`($q, a, loc = 0, scale = 1$) calcola i quantili per inversione numerica della `cdf`, usando gli algoritmi di `Boost`. È la funzione più onerosa di quelle prese a confronto: ogni quantile richiede più valutazioni della `cdf`. Per code estreme o $|\alpha|$ molto alti possono servire più passaggi rispetto che ai casi standard

- Generazione casuale.

`rvs(a, loc = 0, scale = 1, size = 1, random_state = None)` implementa la rappresentazione additiva proposta al paragrafo 1.3.1.

Boost.Math è un modulo in linguaggio C++ costruito per funzioni matematiche speciali e distribuzioni di probabilità ad alta accuratezza numerica: utilizzato per il calcolo di funzioni complesse come *T.Owen*.

L'implementazione di questo boost porta maggior accuratezza e robustezza nei calcoli, migliora le prestazioni e risulta molto leggero grazie al linguaggio in C++.

2.3.2 Skewt-scipy

Pacchetto Python che tratta la distribuzione t Asimmetrica univariata utilizzando la formulazione teorica di Azzalini & Capitanio (2014); link al sito presente in bibliografia [8]. Al momento della stesura dell'elaborato, l'ultima versione disponibile è la **0.1.2** rilasciata in data 11 Ottobre 2025.

Per l'utilizzo di questo repository è necessaria l'installazione del pacchetto SciPy; i parametri e le strutture utilizzate sono coerenti con quelle di SciPy.

Riporto le funzioni di principale interesse per l'elaborato.

- Funzione di densità.

`pdf(x, a, df, loc = 0, scale = 1)` l'implementazione fa riferimento direttamente a (1.8).

- Funzione di ripartizione.

`cdf(x, a, df, loc = 0, scale = 1)` valuta $F_{ST}(x)$ sulla base della definizione di ST e tramite la struttura e i Boost di SciPy.

- Funzione quantile.

`ppf(q, a, df, loc = 0, scale = 1)` calcola i quantili invertendo numericamente la `cdf`. È l'operazione più computazionalmente pesante fra quelle riportate: ogni quantile richiede più valutazioni della `cdf`; per probabilità estreme o combinazioni con α elevato e ν piccolo, possono essere necessarie più iterazioni rispetto ai casi classici.

- Generazione casuale.

`rvs(a, df, loc = 0, scale = 1, size = 1, random_state = None)` implementa il meccanismo della miscela di scala proposta al paragrafo 1.3.1.

2.3.3 Multivariate skew distribution

Repository su GitHub che implementa le versioni multivariate delle famiglie Normale Asimmetrica e t Asimmetrica; link al sito presente in bibliografia [9]. I codici utilizzano le convenzioni di SciPy, facilitandone l'integrazione. Al momento della stesura di questo elaborato l'ultimo commit risulta effettuato in data 18 Gennaio 2022.

Tutte le funzioni si basano sulla teoria sviluppata da Azzalini, in coerenza con la teoria esposta nel Capitolo 1. Non sono presenti le funzioni di ripartizione e le funzioni dei quantili.

Le funzioni disponibili grazie a questo repository sono:

- Normale Asimmetrica multivariata
 - Funzione di densità $\text{pdf}(y, \xi, \Omega, \alpha)$
 - Generazione casuale $\text{rvs}(n, \xi, \Omega, \alpha, \text{random_state} = \text{None})$
- t Asimmetrica multivariata
 - Funzione di densità $\text{pdf}(y, \xi, \Omega, \alpha, \nu)$
 - Generazione casuale $\text{rvs}(n, \xi, \Omega, \alpha, \nu, \text{random_state} = \text{None})$

2.4 Analisi della situazione generale

In generale si nota una scarsa distribuzione di soluzioni native Python per la gestione delle distribuzioni asimmetriche. I codici reperibili in rete sono spesso costruiti per rispondere a specifici casi d'uso o sono progetti abbandonati da diversi anni. Alcune soluzioni, non riportate in questo elaborato, prevedono di utilizzare Python per richiamare il pacchetto *sn* di R per effettuare varie analisi; questo comporta diverse criticità legate al controllo delle compatibilità tra le versioni e una maggior pesantezza del codice.

È utile analizzare la situazione per ciascuna famiglia.

Normale Asimmetrica

Per la gestione univariata sono presenti codici ben strutturati e mantenuti; mancano però alcuni casi particolari, come il trattamento della Normale Asimmetrica Estesa per $\tau \neq 0$ (situazione trattata nell'implementazione di R [2]).

La presenza del codice nel pacchetto SciPy ne facilita l'utilizzo e costituisce, in generale, una buona base di partenza.

Per la gestione multivariata sono presenti solo la funzione di densità e generazione dei dati. Anche se i codici si basano sulla struttura di SciPy, questi non ottengono aggiornamenti dal 2022, con conseguenti problematiche di compatibilità e soluzioni computazionali non aggiornate.

t Asimmetrica

Per la gestione univariata è disponibile un pacchetto recente, rilasciato per la prima volta nel 2024 e costruito sulla base di SciPy. Permette una gestione completa, per gli obiettivi dell'elaborato, utilizzando metodi computazionali moderni con un'ottima integrazione con SciPy; ciò lo rende un'ottima base di partenza.

Per la gestione multivariata restano valide le osservazioni fatte per la SN, con l'aggiunta che non ci sono implementazioni multivariate dirette che utilizzino la Skewt-scipy, rendendo le compatibilità ancora più delicate e problematiche.

Normale Asimmetrica Unificata

Al momento della stesura dell'elaborato non sono presenti codici che implementano l'utilizzo di questa distribuzione utili all'elaborato. Sono presenti online delle repository sviluppate per trattare casi specifici o situazioni in cui è implementata la generazione di dati come backend (come in BoTorch [10]).

Esiste dunque una buona base per trattare le distribuzioni SN e ST nei loro casi univariati in Python. Per i casi multivariati occorre valutare se i codici hanno senso di essere mantenuti con eventuali modifiche oppure se sono oramai troppo obsoleti. Per quanto riguarda la SUN, non sono state prese in considerazione repository o pacchetti, dato che nessuno implementa codici utili allo scopo dell'elaborato.

Capitolo 3

Analisi e confronti dei codici: R vs Python

3.1 Introduzione generale

L'obiettivo del capitolo è valutare tre aspetti principali: l'accuratezza delle stime dei parametri $(\xi, \omega, \alpha, \nu)$; la robustezza numerica e i costi computazionali delle implementazioni; l'integrazione tra i codici. I dati vengono generati con seed fisso per garantire la replicabilità delle analisi; vengono generati campioni di dimensione $n = 2000$ e, per il multivariato, viene utilizzata la dimensione $p = 3$.

Per le distribuzioni Normale Asimmetrica e t Asimmetrica, sia versione univariata che multivariata, sono state svolte le stesse analisi sia con R che con Python. Dove i codici Python presentavano delle limitazioni è stato rivisto anche il codice R per fare in modo che tutte le richieste fossero sempre equivalenti.

La qualità delle implementazioni Python rispetto R, è stata verificata con dei test diagnostici: QQ-plot; valutazione Uniformità; verifica dell'inversione CDF-PPF anche nelle code; paris plot per il multivariato. Inoltre è stato implementato uno stress test su α per valutare la stabilità delle stime al variare dell'asimmetria.

In tutte le fasi di test sono stati raccolti i tempi di esecuzione per le varie categorie per confrontare le prestazioni delle implementazioni Python con quella in R.

Per ogni set di analisi il dataset viene generato da R e poi letto anche da Python.

3.2 Normale Asimmetrica univariata

3.2.1 Descrizione delle analisi

Essendo $\alpha = 0$ punto non regolare per SN, viene esplicitamente evitata questa inizializzazione ponendo $\alpha = 1.2$.

La stima dei parametri utilizza la massima verosimiglianza con parametrizzazione (ξ, ω, α) . Per Python viene implementata la log-verosimiglianza utilizzando *scipy.stats.skewnorm.fit*, corrispettivo dell'implementazione in *sn*.

I test diagnostici comprendono: QQ-plot con quantili teorici ed empirici; controllo dell'uniformità tramite istogramma $u_i = F_{SN}(y_i; \hat{\xi}, \hat{\omega}, \hat{\alpha})$; QQ-plot dei residui standardizzati al quadrato $\hat{Z}^2$ rispetto a χ_1^2 . Per il controllo della stabilità numerica, viene valutata l'inversione $CDF \iff PPF$ calcolando $x_q = PPF(q)$ e $CDF(x_q)$. L'errore assoluto $|CDF(PPF(q)) - q|$ deve risultare basso nelle zone centrali, mentre può essere più alto nelle code. Viene valutato il profilo di verosimiglianza in α : fissata una griglia di valori, vengono ottimizzati (ξ, ω) e si traccia la devianza $D(\alpha) = 2\{\ell(\hat{\theta}) - \ell(\alpha, \hat{\xi}(\alpha), \hat{\omega}(\alpha))\}$.

Infine è stato svolto uno stress test facendo variare α ; vengono riportati gli scarti relativi di $\hat{\xi}$, $\hat{\omega}$, $\hat{\alpha}$ e della log-verosimiglianza, oltre alle differenze nei p-value del KS e negli errori di inversione CDF $\leftrightarrow$ PPF (zona centrale e code).

3.2.2 Risultati

Nella Tabella 3.1 si mettono a paragone le stime dei parametri e della log-verosimiglianza. Si nota che i risultati sono considerabili equivalenti, pertanto le implementazioni possono essere considerate equivalenti.

Tabella 3.1: Confronto stime R vs Python

Parametro	R	Python	Δ (Py-R)
$\hat{\xi}$	0.0302507	0.0302526	1.94847e-06
$\hat{\omega}$	0.985827	0.985829	2.28837e-06
$\hat{\alpha}$	1.02404	1.02405	6.88873e-06
$loglik$	-2411.45	-2411.45	-4.56535e-08

Anche nei QQ-plot della Figura 3.1 non si notano differenze tra l'implementazione R e quella Python.

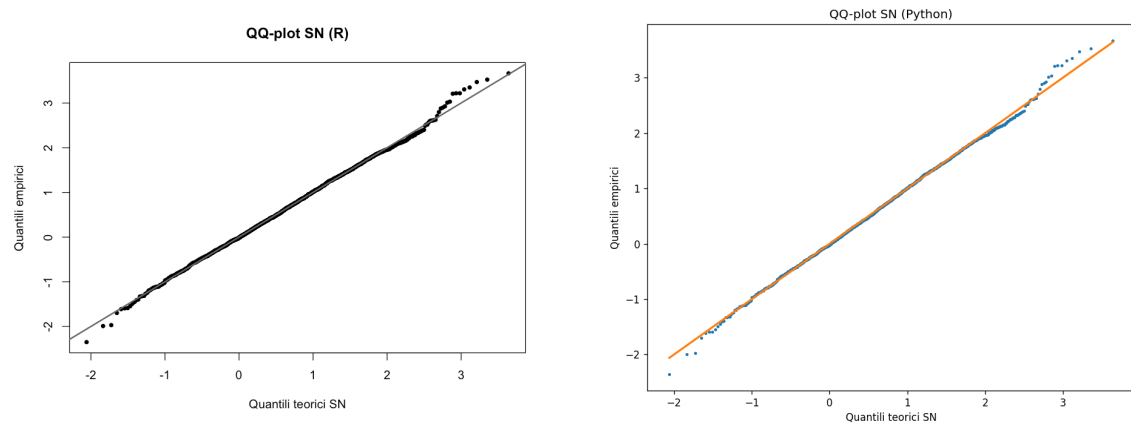


Figura 3.1: Confronto tra i QQ-plot della distribuzione SN in R e Python.

Anche sul controllo dell'uniformità i risultati risultano sovrapponibili. Nei grafici in Figura 3.2 R e Python producono circa gli stessi risultati; non si evidenziano pattern particolari alle estremità. Nella Tabella 3.2 sono riportati i p-value del test di Kolmogorov-Smirnov, in entrambi i casi non si rifiuta l'uniformità; inoltre i p-value risultano quasi identici, con differenze riconducibili ad approssimazioni numeriche.

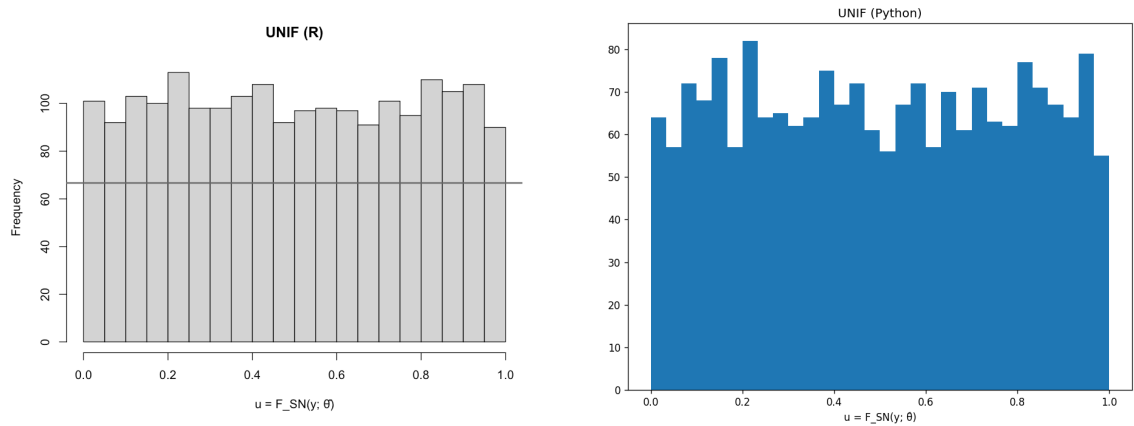


Figura 3.2: Confronto tra gli istogrammi della distribuzione uniforme ottenuti con R e Python.

Tabella 3.2: UNIF: test KS per Uniforme(0,1)

Engine	p -value KS	n
R	0.962244	2000
Python	0.959841	2000

Nella Tabella 3.3 sono riportati: `max_abs_err`, il peggior errore calcolato, e `rmse`, la radice quadrata dell'errore quadratico medio. In entrambi i casi gli errori risultano trascurabili; per Python si nota però un errore molto più piccolo. Anche in questo caso, per fini statistici, le due implementazioni sono equivalenti.

Tabella 3.3: Inversione CDF $\leftrightarrow$ PPF (sintesi errori)

Metrica	R	Python	Δ (Py-R)
<code>max_abs_err</code>	9.99141e-09	2.52206e-14	-9.99138e-09
<code>rmse</code>	3.73806e-09	7.96973e-16	-3.73806e-09

Coerentemente con ciò che è stato riscontrato fino ad ora, i grafici della Figura 3.3 non evidenziano criticità, mostrano dei buoni allineamenti e risultano fra loro sovrapponibili.

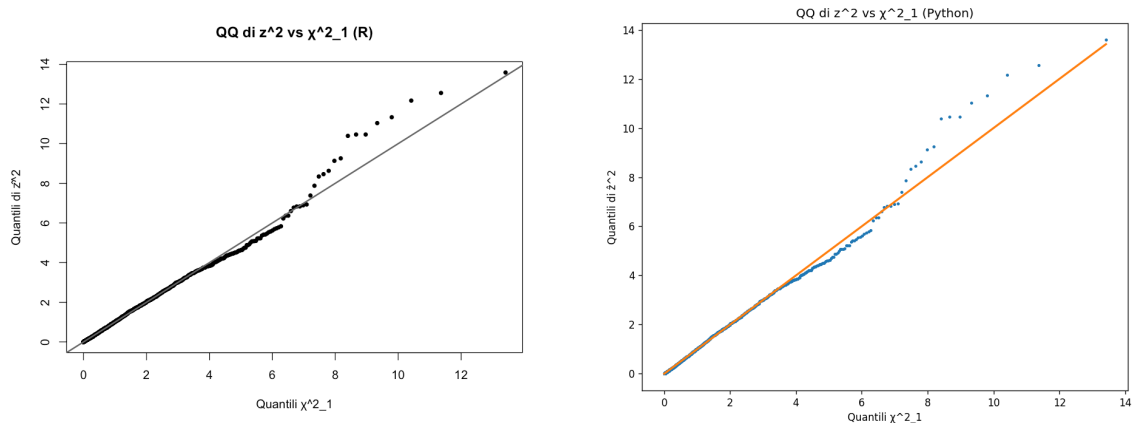


Figura 3.3: Confronto tra i QQ-plot della distribuzione χ^2 ottenuti con R e Python.

Nella Figura 3.4 sono riportati gli andamenti della log-verosimiglianza profilo su α . Si nota che le curve sono sovrapponibili sia per R che per Python. Questo marca il fatto che le due implementazioni concordano sulle stime e sono numericamente robuste.

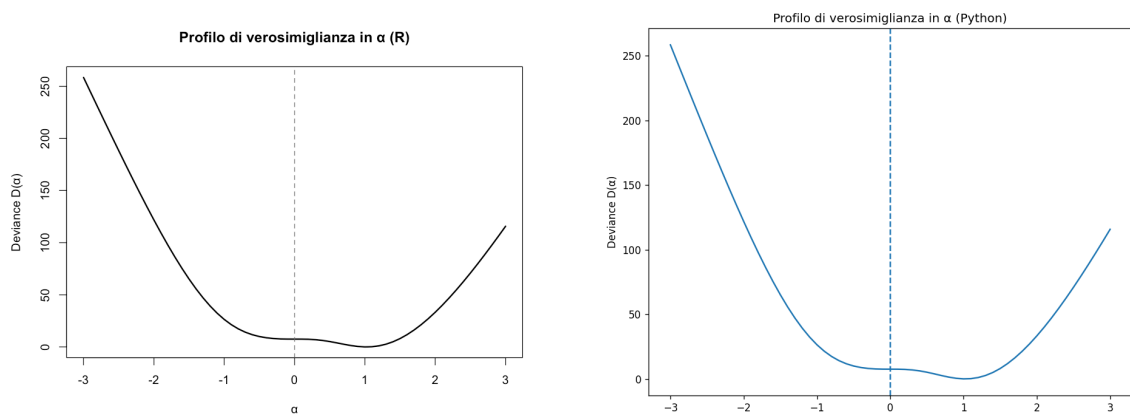


Figura 3.4: Confronto dei profili di log-verosimiglianza rispetto al parametro α ottenuti con R e Python.

Nella Tabella 3.4 sono riportati i tempi di svolgimento di alcune parti del codice. Si può subito notare come nei compiti di carattere numerico (es. fit; profile_alpha) l'implementazione Python risulti sempre nettamente più veloce rispetto R. Solo nelle sezioni dedicate ai plot R batte Python.

Tabella 3.4: Confronto tempi per categoria (secondi)

Categoria	R	Python	Rapporto Py/R
fit	0.038	0.009659	0.2542
profile_alpha	0.834	0.4448	0.5334
ppf	0.028	0.001793	0.06402
cdf	0.007	0.0003145	0.04492
invert_ppf_cdf	0.055	0.001416	0.02574
plot_UNIF	0.008	0.06087	7.609

Infine, è stato svolto uno stress test facendo variare il parametro α . Nella Tabella 3.5 sono riportate le differenze per i valori considerati. Le colonne sono:

- α : Indica i valori di α utilizzati per il test
- $\Delta\xi\%$, $\Delta\omega\%$, $\Delta\alpha\%$, $\Delta\ell\%$: Indicano le differenze relative sulle stime. Il segno indica la direzione dello scarto (positivo: Python > R). Più i valori sono vicini allo zero e più le stime sono sovrapponibili
- $\Delta\text{KS} = \text{KSPy} - \text{KSR}$: Indica la differenza dei p-value relativi ai test di KS per R e Python. Il segno indica la direzione dello scarto (positivo: Python > R).
- $\Delta\text{inv-core}$: Indica la differenza degli errori massimi quando si utilizza l'inversione nella zona centrale.
- $\Delta\text{inv-ext}$: Indica la differenza degli errori massimi di inversione nelle code estreme.

Tabella 3.5: SN univariata — Stress test su α : confronto R vs Python

α	$\Delta\xi\%$	$\Delta\omega\%$	$\Delta\alpha\%$	$\Delta\ell\%$	ΔKS	$\Delta\text{inv-core}$	$\Delta\text{inv-ext}$
-4	-0.0165061	-0.000325774	0.000252372	-8.45912e-10	-0.001786	-2.22044e-16	-6.06822e-10
-1.2	-0.0202012	-0.000877185	0.00258262	-2.32462e-09	-0.002944	-1.66534e-16	-1.72506e-10
-0.2	-68.0332	1.52297	127.321	8.34636e-05	-0.0152	0	-2.9104e-18
0	-90.5366	4.27104	90.723	0.0009561	0.027565	0	1.26319e-14
0.2	78.4152	10.1337	-106.109	0.0128826	0.174775	-1.11023e-16	-3.14419e-18
1.2	-0.0452567	0.000443297	0.00438432	-1.32875e-08	-0.005533	-1.11022e-16	-2.16055e-09
4	0.00462419	0.000434541	0.00108596	-7.91732e-10	-0.006134	0	-7.39033e-10

Dalla tabella si può notare come le implementazioni non presentano risultati significativamente a favore di un'implementazione. Per $\alpha = 0$ si notano delle differenze nei modelli; risultato atteso e giustificato dalla teoria che sconsiglia l'utilizzo di $\alpha = 0$.

3.3 Normale Asimmetrica multivariata

3.3.1 Descrizione delle analisi

Si utilizza una famiglia $SN_p(\xi, \Omega, \alpha)$ con dimensione $p = 3$. Per la generazione dei dati, oltre al seed fisso, assumo:

$$\xi = (0, 1, -1), \quad \Omega = \begin{pmatrix} 1 & 0.4 & 0.2 \\ 0.4 & 1 & 0.3 \\ 0.2 & 0.3 & 1 \end{pmatrix}, \quad \alpha = (2, -1, 1.5), \quad n = 2000.$$

Per la stima dei parametri si utilizza la massima verosimiglianza con la parametrizzazione (ξ, Ω, α) . Per l'implementazione Python sono stati imposti dei vincoli numerici per ridurre i problemi di stabilità numerica. Le analisi in R si adattano ai limiti delle implementazioni Python. Si utilizzano delle analisi grafiche con pairs plot per valutare forma, asimmetria marginale e dipendenze bivariate. Per il confronto quantitativo vengono considerati: $\hat{\xi}$ e $\hat{\alpha}$; la diagonale di $\hat{\Omega}$; la log-verosimiglianza $\ell(\hat{\theta})$. Le differenze sono riportate in scarti assoluti come per la sezione univariata.

Infine, è stato svolto uno stress test su α . Vengono riportati gli scarti delle stime dei parametri, delle varianze e delle log-verosimiglianze.

Per entrambe le implementazioni, la verosimiglianza è stata calcolata con Σ diagonale, agevolando l'implementazione in Python; log-verosimiglianze e varianze marginali sono quindi direttamente confrontabili.

3.3.2 Risultati

I grafici della Figura 3.5 mostrano i pairs plot relativi alle marginali. In entrambi i casi le forme delle nuvole di punti sono coerenti con i segni del vettore α : x_1, x_3 hanno pendenza crescente verso destra, coerente con l'asimmetria positiva data dagli α positivi; x_2 ha pendenza decrescente verso destra, coerente con l'asimmetria negativa data da α negativo. I grafici risultano sovrapponibili, questo conferma la coerenza delle implementazioni; serve ricordare che R si è adattato a Python per rendere le analisi confrontabili, quindi la sovrapponibilità è attesa.

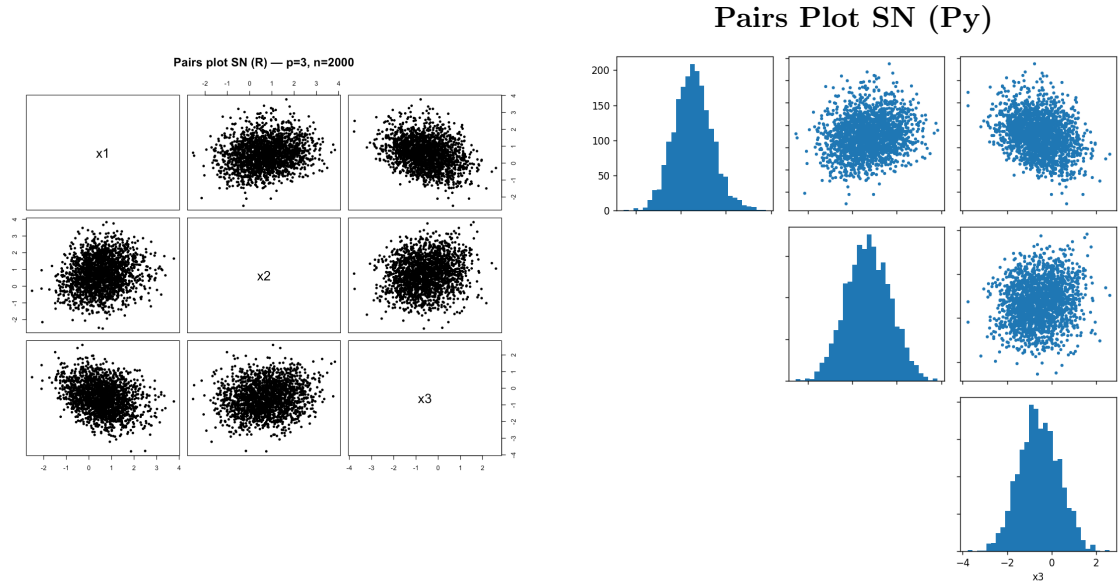


Figura 3.5: Confronto dei pairs plot della distribuzione SN multivariata ottenuti con R e Python.

Nella Tabella 3.6 sono riportati i parametri stimati dalle due implementazioni. Tutti i parametri stimati risultano avere differenze trascurabili fra le implementazioni.

Tabella 3.6: SN multivariata — Confronto stime R vs Python

Parametro	R	Python	Δ (Py–R)
<code>loglik</code>	-7686.33	-7686.33	2.81943e-10
$\hat{\xi}_1$	-0.0483364	-0.0483359	5.08824e-07
$\hat{\xi}_2$	1.00072	1.00072	7.53862e-08
$\hat{\xi}_3$	-0.939499	-0.939499	-9.08807e-08
$\hat{\alpha}_1$	2.10117	2.10117	-2.27841e-06
$\hat{\alpha}_2$	-0.998915	-0.998915	5.25504e-07
$\hat{\alpha}_3$	1.40483	1.40483	-5.79e-07

La Tabella 3.7 confronta gli elementi diagonali della matrice Ω . La sovrapposizione, anche in questo caso, è definibile perfetta. Si nota che, rispetto alla diagonale vera $(1, 1, 1)$, tutte le varianze stimate subiscono una leggera alterazione nell'ordine del 5% o 6%.

Tabella 3.7: SN multivariata — Confronto diagonale di Ω : R vs Python

Dim	$\Omega_{R,ii}$	$\Omega_{Py,ii}$	Δ (Py–R)
$j = 1$	1.0665	1.0665	-8.50389e-07
$j = 2$	1.05219	1.05219	-1.81444e-07
$j = 3$	0.95017	0.95017	-9.46184e-08

Anche in questo caso, come riportato nella Tabella 3.8, la stima dei parametri è nettamente più veloce in Python. I tempi di plotting risultano invece comparabili.

Tabella 3.8: SN multivariata — Confronto tempi per categoria (secondi)

Categoria	R	Python
<code>fit</code>	0.191000	0.095450
<code>plot_pairs</code>	0.073400	0.075250

Infine sono riportate le tabelle relative allo stress test su α .

Nelle successive due tabelle sono riportate le differenze delle stime dei parametri e della log-verosimiglianza: in entrambe le tabelle i valori sono dati da $\Delta = \frac{\text{Python}-\text{R}}{|\text{R}|} \times 100$, quindi un valore positivo implica che la stima fornita da Python è maggiore di quella fornita da R. Nella Tabella 3.9 sono riportate le differenze dei parametri ξ_i e degli elementi della diagonale di Ω . Invece, nella Tabella 3.10 sono riportate le differenze degli α_i e della log-verosimiglianza.

In entrambe le tabelle non ci sono differenze marcate, quindi, nel complesso, le implementazioni risultano equivalenti.

Tabella 3.9: SN multivariata — Stress test su α (Parte I): $\Delta\xi\%$ e $\Delta\Omega\%$ (diag)

α	$\Delta\xi\%$			$\Delta\Omega\%$ (diag)		
	$\Delta\xi_1$	$\Delta\xi_2$	$\Delta\xi_3$	$\Delta\Omega_{1j}$	$\Delta\Omega_{2j}$	$\Delta\Omega_{3j}$
-4.0	-0.6453	0.0040	-0.0039	-0.0031	0.0055	-0.0075
-1.2	-0.0003	-0.0000	-0.0000	-0.0000	-0.0000	-0.0001
-0.2	0.0037	0.0000	-0.0005	0.0000	-0.0000	-0.0002
0.0	-0.0536	-0.0075	-0.0289	0.0004	0.0007	-0.0090
0.2	0.0060	-0.0016	0.0022	-0.0001	0.0006	-0.0009
1.2	0.0004	0.0000	0.0000	-0.0000	-0.0000	-0.0000
4.0	0.0030	-0.0000	0.0000	-0.0001	-0.0000	-0.0001

Tabella 3.10: SN multivariata — Stress test su α (Parte II): $\Delta\alpha\%$ e $\Delta\ell\%$

α	$\Delta\alpha\%$			$\Delta\ell\%$
	$\Delta\alpha_1$	$\Delta\alpha_2$	$\Delta\alpha_3$	$\Delta\ell$
-4.0	0.0114	0.0030	0.0158	6.6e-08
-1.2	0.0001	0.0001	0.0001	2.3e-12
-0.2	-0.0007	-0.0001	0.0013	1.0e-11
0.0	0.0593	0.6749	0.0534	2.8e-08
0.2	-0.0001	0.0100	-0.0117	3.1e-10
1.2	-0.0001	-0.0001	-0.0001	2.2e-12
4.0	-0.0002	-0.0002	-0.0002	8.7e-12

3.4 t Asimmetrica univariata

3.4.1 Descrizione delle analisi

La stima dei parametri utilizza la massima verosimiglianza con parametrizzazione $(\xi, \omega, \alpha, \nu)$. Per la generazione del dataset, oltre al seed fisso, assumo: $\xi = 0$, $\omega = 1$, $\alpha = 1.2$, $\nu = 6$, $n = 2000$.

Per R la massimizzazione è implementata con *optim* utilizzando variabili trasformate e imponendo vincoli numerici per ridurre i casi di problemi di stabilità.

Per Python si implementa *skewt_scipy.skewt.fit* per stimare direttamente i parametri $(\xi, \omega, \alpha, \nu)$; la log-verosimiglianza viene calcolata come somma di *logpdf* nel punto di massimo trovato.

La diagnostica utilizzata è:

- QQ-plot con quantili teorici vs empirici.
- Uniformità con istogramma e test KS.
- Profilo di verosimiglianza con verosimiglianza sia per α , sia per ν .

Infine viene eseguito uno stress test su α .

3.4.2 Risultati

Nella Tabella 3.11 si mettono a confronto le stime dei parametri e della log-verosimiglianza. Si nota che le stime ottenute dalle due implementazioni presentano delle differenze trascurabili.

Tabella 3.11: ST univariata — Confronto stime R vs Python

Parametro	R	Python	Δ (Py-R)
$\hat{\xi}$	0.01486	0.01486	0.0000053
$\hat{\omega}$	0.99984	0.99983	-0.0000040
$\hat{\alpha}$	1.05032	1.05032	-0.0000046
$\hat{\nu}$	6.23673	6.23666	-0.0000713
ℓ	-2779.36	-2779.36	-5.0e ⁻⁹

Nella Figura 3.6 i QQ-plot non mostrano grandi differenze tra l'implementazione in R e quella in Python.

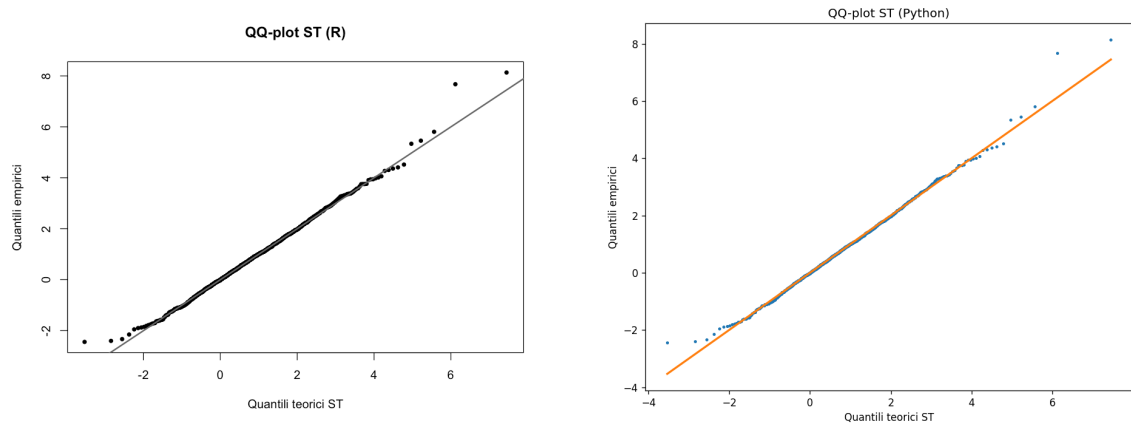


Figura 3.6: Confronto dei QQ-plot della distribuzione ST ottenuti con R e Python.

Anche per l'uniformità i risultati sono sovrapponibili. Nei grafici della Figura 3.7 le implementazioni R e Python producono circa lo stesso risultato; non si evidenziano pattern particolari alle estremità. Nella Tabella 3.12 sono riportati i p-value del test di Kolmogorov-Smirnov, in entrambi i casi non si rifiuta l'uniformità; inoltre i p-value risultano quasi identici.

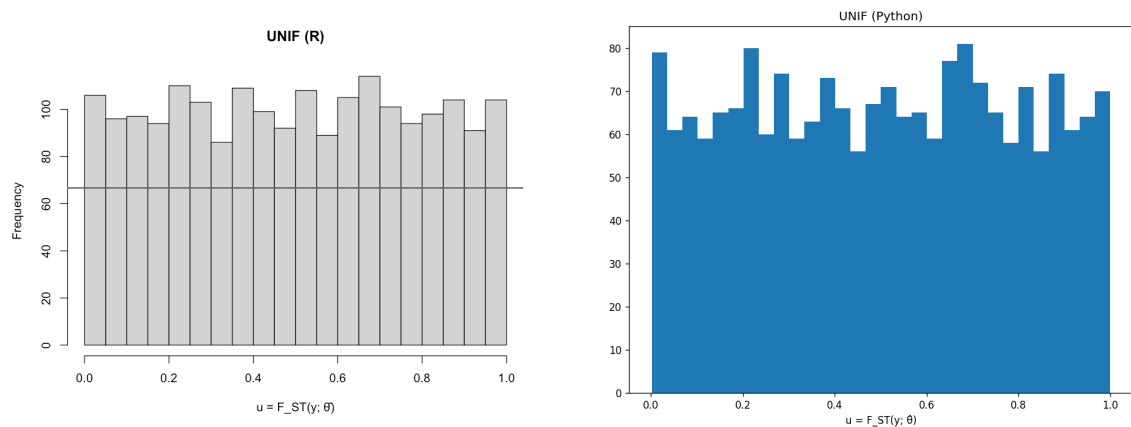


Figura 3.7: Confronto degli istogrammi della distribuzione uniforme ottenuti con R e Python.

Tabella 3.12: Test KS per Uniforme(0,1) — confronto tra engine R e Python

Motore	p -value (KS)	n
R	0.9837	2000
Python	0.9823	2000

Nella Tabella 3.13 sono riportati: `max_abs_err`, il peggior errore calcolato, e RMSE, la radice quadrata dell'errore quadratico medio. In entrambi i casi gli errori risultano trascurabili e pressoché identici. Anche in questo caso, per fini statistici, le due implementazioni sono equivalenti.

Tabella 3.13: Inversione CDF $\leftrightarrow$ PPF — sintesi degli errori numerici

Metrica	R	Python	Δ (Py-R)
<code>max_abs_error</code>	9.99e^{-9}	1.12e^{-9}	-8.87e^{-9}
RMSE	2.87e^{-9}	2.18e^{-10}	-2.66e^{-9}

La Figura 3.8 mostra il profilo di verosimiglianza per α . Non sono presenti differenze marcate, pertanto le due implementazioni sono equivalenti. La stima di α coincide con il minimo del grafico ed è coerente con i valori utilizzati per la generazione dei dati.

La Figura 3.9 mostra il profilo di verosimiglianza per ν . Anche in questo caso non sono presenti differenze marcate, pertanto le due implementazioni sono equivalenti. La stima di ν coincide con il minimo del grafico ed è coerente con i valori utilizzati per la generazione dei dati.

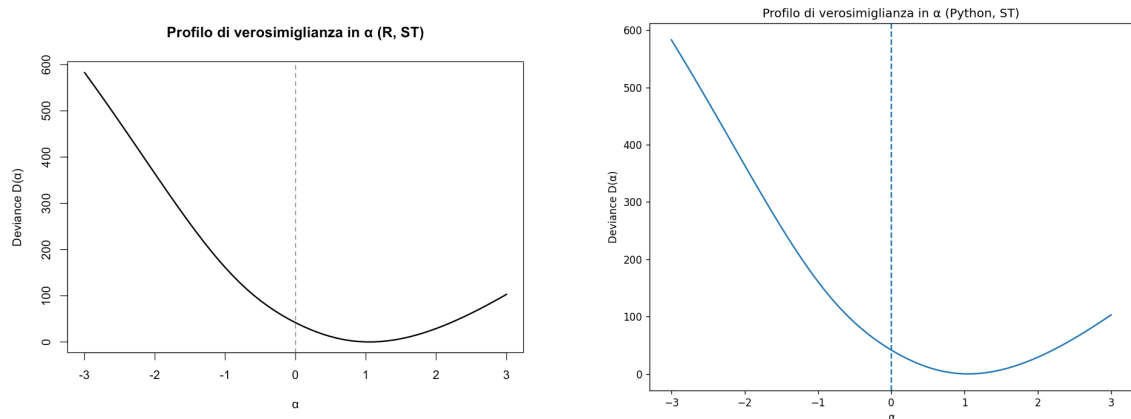


Figura 3.8: Confronto dei profili di log-verosimiglianza rispetto al parametro α ottenuti con R e Python.

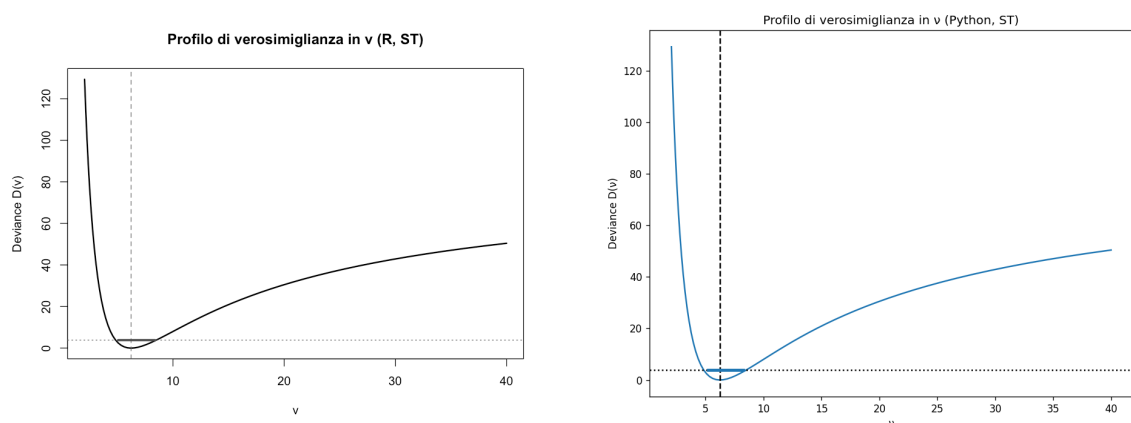


Figura 3.9: Confronto dei profili di log-verosimiglianza rispetto al parametro ν ottenuti con R e Python.

Nella Tabella 3.14 sono rappresentati i tempi per l'esecuzione dei codici. In generale R si presenta sempre più veloce di Python, sia nelle parti numeriche che in quelle grafiche. Il collo di bottiglia dell'implementazione risulta essere principalmente la funzione ppf e in secondo luogo la funzione cdf, risultato in accordo con la teoria del Capitolo 1.

Tabella 3.14: ST univariata — Confronto tempi per categoria (secondi)

Categoria	R	Python	Rapporto Py/R
fit	0.106	0.442	4.17
profile_alpha	3.220	22.36	6.95
profile_nu	3.084	15.52	5.03
ppf	0.695	244.40	351.7
cdf	0.057	25.22	442.5
invert_ppf_cdf	0.367	136.40	371.8
plot_QQ	0.051	0.137	2.68
plot_UNIF	0.009	0.118	13.15
plot_profile_alpha	0.009	0.063	7.01
plot_profile_nu	0.008	0.064	8.06

Infine, è stato svolto uno stress test facendo variare il parametro α . Nella Tabella 3.15 sono riportate le differenze per i valori considerati. Le colonne sono:

- α : Indica i valori di α utilizzati per il test
- $\Delta\xi\%$, $\Delta\omega\%$, $\Delta\alpha\%$, $\Delta\nu\%$, $\Delta\ell\%$: Indicano le differenze relative sulle stime. Il segno indica la direzione dello scarto (positivo: Python $>$ R). Più i valori sono vicini allo zero e più le stime sono sovrapponibili
- $\Delta\text{KS} = \text{KSPy} - \text{KSR}$: Indica la differenza dei p-value relativi ai test di KS per R e Python. Il segno indica la direzione dello scarto (positivo: Python $>$ R).
- $\Delta\text{inv-core}$: Indica la differenza degli errori massimi quando si utilizza l'inversione nella zona centrale.
- $\Delta\text{inv-ext}$: Indica la differenza degli errori massimi di inversione nelle code estreme.

Tabella 3.15: Stress test su α : differenze percentuali R vs Python

α	$\Delta\xi\%$	$\Delta\omega\%$	$\Delta\alpha\%$	$\Delta\nu\%$	$\Delta\ell\%$	$\Delta\text{UNIF(KS)}$	$\Delta\text{inv-core}$	$\Delta\text{inv-ext}$
-4.0	0.0135	0.0001	-0.0018	-0.0027	3.7e^{-9}	-0.0061	9.97e^{-9}	9.45e^{-9}
-1.2	0.1367	0.0065	-0.0146	0.0076	3.4e^{-8}	-0.0049	9.95e^{-9}	6.39e^{-9}
-0.2	-0.7227	-0.0018	0.0341	-0.0038	5.7e^{-9}	-0.0062	9.99e^{-9}	6.94e^{-9}
0.0	-10.333	0.0024	17.1625	-0.0072	-4.8e^{-6}	-0.0013	9.95e^{-9}	6.23e^{-9}
0.2	0.1554	0.0093	-0.4547	0.0417	2.8e^{-7}	-0.0052	9.99e^{-9}	9.42e^{-9}
1.2	0.0095	-0.0018	-0.0033	-0.0090	5.7e^{-9}	-0.0057	9.98e^{-9}	4.99e^{-9}
4.0	0.0117	-0.0013	-0.0002	-0.0072	7.3e^{-9}	-0.0048	9.99e^{-9}	8.03e^{-9}

Dalla tabella si nota che non c'è un'implementazione nettamente migliore dell'altra. Si notano differenze maggiori per $\alpha = 0$, che risulta comunque un punto delicato da trattare anche se si utilizza la t Asimmetrica.

3.5 t Asimmetrica multivariata

3.5.1 Descrizione delle analisi

Si utilizza una famiglia $ST_p(\xi, \Omega, \alpha, \nu)$ con dimensione $p = 3$. Per la generazione dei dati, oltre al seed fisso, assumo:

$$\xi = (0, 1, -1), \quad \Omega = \begin{pmatrix} 1 & 0.4 & 0.2 \\ 0.4 & 1 & 0.3 \\ 0.2 & 0.3 & 1 \end{pmatrix}, \quad \alpha = (2, -1, 1.5), \quad \nu = 8, \quad n = 2000.$$

Per la stima dei parametri si utilizza la massima verosimiglianza nella parametrizzazione $(\xi, \Omega, \alpha, \nu)$. Oltre all'inclusione dell'analisi del parametro ν , le verifiche sono le stesse della SN multivariata, con R adattato a Python. Come in precedenza, in Python si assume Ω diagonale e si applicano gli stessi vincoli numerici (varianze > 0 , $\nu > 2$) per migliorare la stabilità. A differenza della SN multivariata, R non è vincolato all'utilizzo della stessa struttura della matrice perché non risultano problemi nella gestione dei dati.

3.5.2 Risultati

I grafici della Figura 3.10 mostrano i pairs plot relativi alle marginali. In entrambi i casi le forme delle nuvole di punti sono coerenti con i segni del vettore α : x_1, x_3 hanno pendenza crescente verso destra, coerente con l'asimmetria positiva data dagli α positivi; x_2 ha pendenza decrescente verso destra, coerente con l'asimmetria negativa data da α negativo. I grafici risultano sovrapponibili, questo conferma la coerenza delle implementazioni anche con la diversa gestione di Ω : il confronto visivo resta comunque informativo.

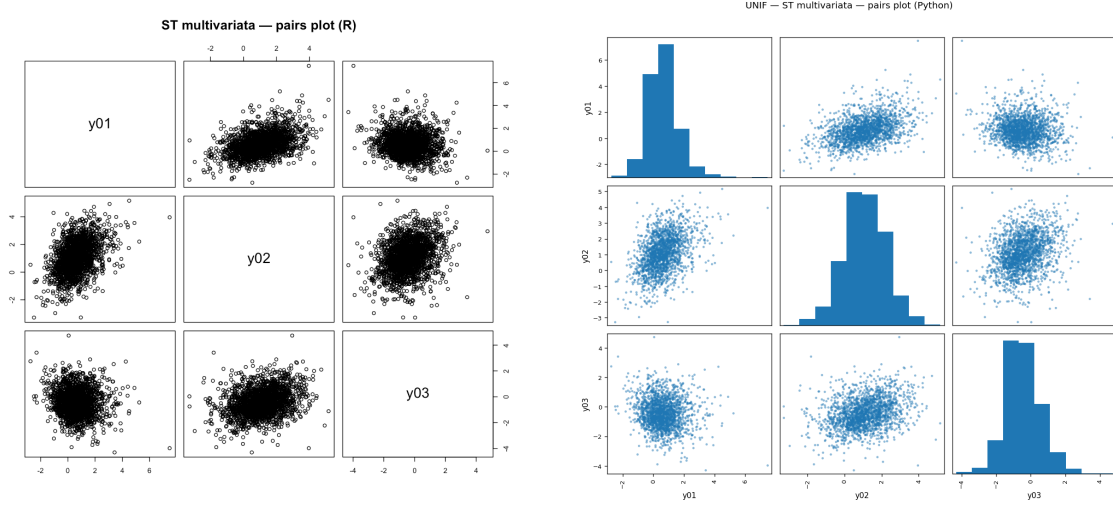


Figura 3.10: Confronto dei pairs plot della distribuzione ST multivariata ottenuti con R e Python.

Nella Tabella 3.16 sono riportati i parametri stimati dalle due implementazioni. Rispetto alle analisi precedenti, le differenze tra le stime risultano più marcate. In particolare, in Python la log-verosimiglianza non raggiunge il massimo globale, risultando quindi peggiore rispetto R; probabilmente una delle cause è la diversa gestione di Ω .

Tabella 3.16: ST multivariata: confronto stime R vs Python

Parametro	R	Python	Δ (Py–R)
loglik	-8207.71	-8288.56	-80.844
$\hat{\nu}$	9.2093	9.4863	0.277
$\hat{\xi}_1$	-0.0636	0.1179	0.181
$\hat{\alpha}_1$	2.0923	1.9071	-0.185
Ω_{11}	1.1105	0.9190	-0.191
$\hat{\xi}_2$	0.9500	1.5501	0.600
$\hat{\alpha}_2$	-0.9265	-1.6628	-0.736
Ω_{22}	1.0857	1.2898	0.204
$\hat{\xi}_3$	-0.9394	-0.8022	0.137
$\hat{\alpha}_3$	1.3613	1.3125	-0.049
Ω_{33}	0.9934	0.8992	-0.094

Nella Tabella 3.17 sono riportati i valori della diagonale di Ω . Anche in questo caso le differenze sono più marcate rispetto alle analisi precedenti; non c'è ancora una netta differenza che possa far preferire un'implementazione all'altra.

Tabella 3.17: ST multivariata: confronto diagonale di Ω (R vs Python)

j	$\Omega_{R,ii}$	$\Omega_{Py,ii}$	Δ (Py–R)
1	1.11045	0.91902	-0.19143
2	1.08573	1.28980	0.20407
3	0.99342	0.89921	-0.09421

Nella Tabella 3.18 sono riportati i tempi di esecuzione per la stima dei parametri e per il plot dei grafici. Dal punto di vista numerico, Python mostra tempi di esecuzione nettamente inferiori rispetto a R, ma non raggiunge la convergenza, producendo risultati meno accurati. Per la parte grafica, invece, R risulta migliore.

Tabella 3.18: ST multivariata: confronto tempi per categoria (secondi)

Categoria	R	Python	Rapporto Py/R
<code>fit</code>	0.937	0.4445	0.4744
<code>plot_pairs</code>	0.111	0.3309	2.981

Le successive tabelle fanno riferimento allo stress test su α .

Nella Tabella 3.19 sono riportate le stime dei parametri per le due implementazioni per i diversi α . Per ogni α , le conclusioni risultano essere le medesime fatte per la Tabella 3.16.

Nella Tabella 3.20 sono riportate le stime dei valori sulla diagonale di Ω per i diversi α . Per ogni α , le conclusioni risultano essere le medesime fatte per la Tabella 3.17.

Tabella 3.19: ST multivariata — Stress test in α : confronto stime R vs Python

α scale = -4.0				α scale = -1.2			
Parametro	R	Python	Δ	Parametro	R	Python	Δ
ℓ	-7937.160	-8006.600	-69.440	ℓ	-8547.650	-8623.260	-75.610
$\hat{\nu}$	7.500	7.138	-0.361	$\hat{\nu}$	7.016	7.443	0.426
$\hat{\xi}_1$	0.009	-0.108	-0.117	$\hat{\xi}_1$	0.058	-0.006	-0.064
$\hat{\xi}_2$	0.917	0.475	-0.442	$\hat{\xi}_2$	0.948	0.393	-0.555
$\hat{\xi}_3$	-1.085	-1.220	-0.135	$\hat{\xi}_3$	-1.009	-1.194	-0.185
$\hat{\alpha}_1$	-3.205	-3.156	0.049	$\hat{\alpha}_1$	-1.043	-1.261	-0.218
$\hat{\alpha}_2$	1.697	2.460	0.764	$\hat{\alpha}_2$	0.619	1.468	0.849
$\hat{\alpha}_3$	-2.273	-2.152	0.121	$\hat{\alpha}_3$	-0.716	-0.630	0.086
α scale = -0.2				α scale = 0.0			
ℓ	-8867.360	-8970.220	-102.866	ℓ	-9037.770	-9175.300	-137.531
$\hat{\nu}$	8.471	9.776	1.304	$\hat{\nu}$	8.141	9.116	0.976
$\hat{\xi}_1$	0.095	0.291	0.196	$\hat{\xi}_1$	0.179	0.374	0.195
$\hat{\xi}_2$	0.899	0.356	-0.543	$\hat{\xi}_2$	1.235	0.389	-0.846
$\hat{\xi}_3$	-1.050	-0.909	0.141	$\hat{\xi}_3$	-0.802	-0.870	-0.068
$\hat{\alpha}_1$	-0.359	-0.873	-0.514	$\hat{\alpha}_1$	-0.129	-0.777	-0.648
$\hat{\alpha}_2$	0.224	1.180	0.957	$\hat{\alpha}_2$	-0.166	1.132	1.298
$\hat{\alpha}_3$	0.017	-0.294	-0.310	$\hat{\alpha}_3$	-0.174	-0.300	-0.127
α scale = 0.2				α scale = 1.2			
ℓ	-8998.750	-9115.350	-116.602	ℓ	-8432.170	-8503.600	-71.433
$\hat{\nu}$	7.675	8.585	0.909	$\hat{\nu}$	8.670	9.067	0.397
$\hat{\xi}_1$	0.104	0.384	0.280	$\hat{\xi}_1$	-0.083	0.051	0.134
$\hat{\xi}_2$	0.889	0.352	-0.538	$\hat{\xi}_2$	0.846	1.601	0.756
$\hat{\xi}_3$	-0.870	-0.679	0.191	$\hat{\xi}_3$	-1.087	-0.850	0.238
$\hat{\alpha}_1$	-0.085	-0.638	-0.553	$\hat{\alpha}_1$	0.884	1.007	0.123
$\hat{\alpha}_2$	0.235	1.253	1.018	$\hat{\alpha}_2$	-0.269	-1.298	-1.029
$\hat{\alpha}_3$	-0.077	-0.463	-0.387	$\hat{\alpha}_3$	0.835	0.729	-0.105
α scale = 4.0							
ℓ	-8014.600	-8105.280	-90.683				
$\hat{\nu}$	7.869	7.117	-0.752				
$\hat{\xi}_1$	-0.003	0.124	0.127				
$\hat{\xi}_2$	0.965	1.488	0.524				
$\hat{\xi}_3$	-1.049	-0.859	0.190				
$\hat{\alpha}_1$	2.865	2.761	-0.105				
$\hat{\alpha}_2$	-1.442	-2.138	-0.696				
$\hat{\alpha}_3$	2.246	1.994	-0.252				

Tabella 3.20: ST multivariata: diagonale di Ω , confronto R vs Python

α scale = -4.0				α scale = -1.2			
j	ω_R	ω_{Py}	Δ	j	ω_R	ω_{Py}	Δ
1	1.02453	0.88742	-0.13711	1	0.98277	0.94325	-0.03952
2	0.98351	1.13253	0.14902	2	0.96698	1.27021	0.30323
3	0.88462	0.77494	-0.10967	3	0.98494	0.89275	-0.09219
α scale = -0.2				α scale = 0.0			
1	1.00522	1.14246	0.13724	1	1.06068	1.18330	0.12262
2	1.04829	1.42116	0.37287	2	1.07132	1.38291	0.31159
3	0.91693	0.95117	0.03425	3	1.07595	1.07215	-0.00380
α scale = 0.2				α scale = 1.2			
1	0.96701	1.06568	0.09868	1	1.04294	0.93708	-0.10586
2	1.12036	1.55305	0.43269	2	0.99236	1.24506	0.25270
3	0.93428	0.98324	0.04896	3	1.03717	0.87725	-0.15992
α scale = 4.0							
1	0.98686	0.82976	-0.15710				
2	1.00960	1.13588	0.12628				
3	1.06376	0.87079	-0.19297				

3.6 Analisi generale dei risultati

Per la Normale Asimmetrica univariata si ha a disposizione la miglior implementazione Python analizzata nell'elaborato. Utilizzandola si ottengono risultati equivalenti a quelli ottenuti con R senza dover inserire delle specifiche limitazioni. Il pacchetto utilizzato è mantenuto e costantemente aggiornato, questo porta ad avere delle prestazioni migliori anche rispetto ad R; nei test, l'implementazione in Python ha prodotto risultati equivalenti a quelli di R, ma con tempi di esecuzione inferiori.

Per la t Asimmetrica univariata si ha a disposizione un pacchetto molto recente come implementazione Python. Si basa su SciPy, quindi ha delle basi di sviluppo solide. Sono serviti dei piccoli accorgimenti per implementarlo ma risulta molto potente ed è stato possibile confrontarlo senza limitare R. I risultati delle stime sono tutti paragonabili a R con uno scarto sempre accettabile. Il problema più grosso riguarda i tempi di esecuzione delle funzioni. In generale i problemi sono attribuibili al fatto che il pacchetto ha meno di 1 anno di sviluppo alle spalle e non è stato ancora ottimizzato.

Per la Normale e la t Asimmetrica multivariata, l'implementazione Python è vecchia e problematica. Utilizza SciPy come motore, ma mancando di aggiornamenti non è più ben ottimizzato per l'utilizzo. Le funzioni implementate sono poche e non ottimizzate, per utilizzarlo è stato necessario adattare R ai codici Python. Nei test eseguiti sopra non sembra neanche avere delle brutte prestazioni, però queste sono viziato dal fatto che R è stato messo allo stesso livello; per SN multivariato inoltre sono state riscontrate maggiori problematiche nell'implementazione principalmente nella gestione delle matrici quasi singolari.

Capitolo 4

Proposta di un pacchetto integrato

4.1 Proposta di integrazione

Le analisi svolte nel capitolo precedente mostrano che, in Python, per le versioni univariate di SN e ST sono disponibili implementazioni affidabili, mentre per le corrispondenti multivariate non sono attualmente presenti soluzioni valide. Come visto nel Capitolo 2, per la SUN, non sono presenti implementazioni utilizzabili native per Python.

In questo capitolo è presentata una possibile implementazione della SUN in codice Python. Dati i risultati del capitolo precedente, l'implementazione prevede il solo utilizzo di SciPy; la struttura delle funzioni è la più simile possibile a quella proposta nel pacchetto *sn*. Per questo primo approccio si è scelto di valutare l'implementazione delle funzioni R: `dsun`, `psun` e `rsun`.

Purtroppo, allo stato attuale del pacchetto, non è possibile implementarle tutte e tre. Per costruire la funzione di generazione di numeri casuali (`rsun`), secondo la teoria descritta nel Capitolo 1, è necessario un generatore di dati da una Normale multivariata troncata. In SciPy è disponibile un generatore di dati da una Normale univariata troncata, non sufficiente a gestire i vincoli di troncamento di tutte le variabili. Essendo inoltre questo generatore il collo di bottiglia della funzione si è preferito non trattare l'implementazione di questa funzione in questo elaborato per dare spazio a funzioni meno delicate ma comunque utili all'integrazione.

Le funzioni di cui si propone una prima implementazione nei paragrafi successivi sono:

- Funzione di densità (`dsun`), con riferimento all'equazione 1.14
- Funzione di ripartizione (`psun`)

L'implementazione di queste funzioni mette le basi per sviluppare altre funzioni come: la verosimiglianza, per fare stime di massima verosimiglianza e confronti; calcolare probabilità cumulate e p-value.

4.2 Schemi di struttura dei codici

4.2.1 Funzione di densità

I passi seguiti, sulla base dell'implementazione in R, per costruire il codice sono:

- Ingresso dei parametri e controlli. $x \in \mathbb{R}^d$, $\xi \in \mathbb{R}^d$, $\Omega \in \mathbb{R}^{d \times d}$ matrice simmetrica e definita positiva, $\Delta \in \mathbb{R}^{d \times m}$, $\tau \in \mathbb{R}^m$, $\Gamma \in \mathbb{R}^{m \times m}$ matrice

simmetrica e definita positiva; serve anche controllare la coerenza dimensionale e generare un warning per $m=1$.

- Standardizzazione di $\omega = \text{diag}(\sqrt{\Omega_{jj}})$ e $\bar{\Omega} = \omega^{-1}\Omega\omega^{-1}$.
- Risolvere $\bar{\Omega}M = \Delta$ tramite Cholesky.
- Derivare $\Sigma_m = \Gamma - \Delta^\top M$ con verifica se è matrice simmetrica e definita positiva.
- Trasformazione dei dati, per ogni riga x , $z = \omega^{-1}(x - \xi)$ risolvendo $\bar{\Omega}u = z$
- Utilizzare SciPy per generare i blocchi Gaussiani:
 - $\log \phi_d(x; \xi, \Omega)$ utilizzando `scipy.stats.multivariate_normal.logpdf`
 - $\log \Phi_m(s; \Sigma_m)$ e $\log \Phi_m(\tau; \Gamma)$ tramite `multivariate_normal.cdf`
- Combinazione logaritmica per evitare problemi di underflow o overflow.
 $\log f_{\text{SUN}}(x) = \log \phi_d(x; \xi, \Omega) + \log \Phi_m(s; \Sigma_m) - \log \Phi_m(\tau; \Gamma)$
- Restituzione di un vettore di lunghezza n

Per la soluzione della di $\bar{\Omega}$ tramite Cholesky è stato implementato un metodo che forza la fattorizzazione. Il metodo dato di default con SciPy è troppo sensibile alle matrici singolari e quasi singolari, rendendo il suo utilizzo poco pratico.

Codice completo presente in appendice.

4.2.2 Funzione di ripartizione

I passi seguiti, sulla base dell'implementazione in R, per costruire il codice sono:

- Ingresso dei parametri e controlli. $x \in R^d$, $x_i \in R^d$, $\Omega \in R^{dd}$ simmetrica e definita positiva, $\Delta \in R^{dm}$, $\tau \in R^m$, $\Gamma \in R^{mm}$ simmetrica e definita positiva; serve anche verificare la coerenza delle dimensioni e un warning per $m = 1$.
- Come per R si imposta la soglia pratica su $(d + m) < 20$.
- Standardizzazione $\omega = \text{diag}(\sqrt{\Omega_{jj}})$ e $\Omega = \omega^{-1}\Omega\omega^{-1}$. Implementata anche se la stima tramite Monte Carlo non la utilizza in maniera diretta.
- Stimatore Monte Carlo. Campionamento di $Y^{(s)} \sim N_d(\xi, \Omega)$ con valutazione dei pesi tramite `dsun`.
- Come output viene dato un vettore di lunghezza n con $F_{\text{SUN}}(x_i)$.

Per implementare questo codice si utilizzano le funzioni già implementate per *dsun*. Siccome SciPy non ha a disposizione una distribuzione Normale multivariata troncata, risulta più utile implementare un metodo Monte Carlo piuttosto che utilizzare SciPy direttamente.

Codice completo presente in Appendice.

4.3 Considerazioni finali

Sebbene in particolare in ambito bayesiano, la funzione di maggior interesse sia quella per la generazione di numeri casuali, in questa fase si è scelto di non implementare *rsun* a causa della sua complessità. Sono stati invece sviluppati i prototipi di *dsun* e *psun* per verificare la fattibilità di un'implementazione in Python basata su SciPy. L'implementazione descritta va considerata un prototipo esplorativo; pertanto, al momento non si presentano confronti con le corrispondenti funzioni disponibili in R.

I codici non sono ottimizzati e presentano grosse criticità da studiare e correggere in futuro. Nonostante alcuni accorgimenti numerici, la gestione delle matrici non risulta ottimale: la fattorizzazione di Cholesky è grezza e poco efficace nel trattare casi di matrici quasi singolari. Inoltre, i costi computazionali non sono stati considerati, pertanto sono presenti passaggi non essenziali e aspetti computazionali non ottimizzati.

Pur essendo SciPy solido e ben ottimizzato, per implementare la SUN serve integrarlo con componenti aggiuntive. Uno dei principali limiti è l'assenza di un generatore per la Normale multivariata troncata, che impedisce di implementare completamente tutte le funzioni necessarie.

Le scelte adottate sono coerenti con i colli di bottiglia presentati in letteratura: la SUN, anche in ambito bayesiano, richiede valutazioni di $\Phi_n(\cdot)$ e generatori efficienti di Normali multivariate troncate. Data l'assenza non c'era motivo di implementare inutilmente *rsun* con un generatore non ottimale; per tale motivazione si ha dato spazio a *dsun* e *psun*, utilizzando soluzioni basate su combinazioni logaritmiche per migliorare la stabilità numerica (Durante, 2019).

Conclusioni

Lo scopo di questo elaborato è quello di fare il punto della situazione sui codici Python realmente utilizzabili per lavorare con le famiglie Normale Asimmetrica, t Asimmetrica e Normale Asimmetrica Unificata. Si è scelto uno stile volto più agli aspetti operativi, utilizzando come riferimento il pacchetto *sn* di R.

Per la Normale Asimmetrica univariata, il pacchetto SciPy offre una base solida e facilmente utilizzabile; risulta essere equivalente a R per la stima dei parametri e migliore negli aspetti computazionali. Per la t Asimmetrica univariata, il pacchetto Skewt-sciPy è molto recente e richiede ancora delle ottimizzazioni, però risulta essere un ottimo strumento da integrare e utilizzare.

Per quanto riguarda le versioni multivariate, le soluzioni Python sono parziali e datate. Le repository presenti sono incomplete e poco ottimizzate; considerato che non vengono aggiornate da anni, potrebbe essere meglio riscrivere da capo le nuove implementazioni.

Per quanto riguarda la SUN, non sono presenti repository Python pronte all'uso. Nel Capitolo 4 viene proposta una prima implementazione allo scopo di testare SciPy: il risultato mostra che il pacchetto ha delle basi solide su cui lavorare ma necessita di alcune integrazioni.

Una proposta di integrazione potrebbe essere sviluppata secondo questa roadmap:

- Consolidare le implementazioni univariate ottimizzando ST
- Costruire delle implementazioni multivariate per SN e ST solide
- Costruire un generatore di numeri da una Normale troncata multivariata
- Costruire una repository per trattare la SUN utilizzando come motore SciPy
- Generalizzare e ottimizzare i codici

Le conclusioni sono basate su campioni di dimensioni moderate e scenari controllati: sufficienti per confrontare coerenza, stabilità e costi computazionali. Restano da verificare la scalabilità su n e per dimensioni elevate di d .

La costruzione di un pacchetto Python che permetta di utilizzare agevolmente queste distribuzioni, potrebbe portare benefici concreti in ambito industriale (grazie all'utilizzo di strumenti più precisi, es. per la gestione delle code) e uno sviluppo più rapido del pacchetto stesso, grazie al lavoro di molte più persone in molti ambiti diversi.

Appendice

I codici completi e aggiornati sono visionabili nella repository GitHub:
github.com/mattiacasetta/skew_distribution_python_tests.git.

Codici Capitolo 3

Normale Asimmetrica univariata

```
# SN univariata (dataset, stima, diagnostica)
# Primitive: dsn/psn/qsn/rsn del pacchetto 'sn'.
# Note: evitare start con alpha=0 (non regolare in alfa=0, manuale sn).
library(sn)
set.seed(20250303)
# Parametri "veri" per la generazione
n <- 2000
xi <- 0
omega <- 1
alpha <- 1.2
# Log-verosimiglianza SN (DP: xi, omega, alpha)
loglik_sn <- function(par, y) {
  xi <- par[1]; om <- abs(par[2]); al <- par[3]
  sum(log(dsn(y, xi = xi, omega = om, alpha = al)))
}
# Dati: generazione + salvataggio
y <- rsn(n, xi = xi, omega = omega, alpha = alpha)
write.csv(data.frame(y = y), paths$data, row.names = FALSE)
# Stima ML
start <- c(median(y), IQR(y)/1.349, 0.5)
fit <- optim(
  par = start,
  fn = function(p) -loglik_sn(p, y),
  method = "BFGS",
  control = list(reltol = 1e-10, maxit = 2000)
)
parhat <- c(xi = fit$par[1], omega = abs(fit$par[2]), alpha = fit$par[3])
llhat <- -fit$value
fevals <- if (!is.null(fit$counts["function"])) as.integer(fit$counts["function"]) else NA_integer_
conv <- fit$convergence
msg <- if (!is.null(fit$message)) as.character(fit$message) else ""
# QQ teorico SN
p <- (1:n - 0.5) / n
q.th <- qsn(p, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"])
q.emp <- sort(y)
png(paths$fig_QQ_sn, width = 900, height = 700, res = 120)
plot(q.th, q.emp, pch = 20, main = "QQ-plot SN (R)",
      xlab = "Quantili teorici SN", ylab = "Quantili empirici")
abline(0, 1, lwd = 2, col = "gray40")
dev.off()
# Diagnostica: UNIF
u <- psn(y, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"])
png(paths$fig_UNIF, width = 900, height = 700, res = 120)
hist(u, breaks = 30, freq = TRUE, main = "UNIF (R)", xlab = "u = F_SN(y; )")
abline(h = length(u)/30, lwd = 2, col = "gray40")
dev.off()
# Test KS
ks_p <- suppressWarnings(ks.test(u, "punif")$p.value)
write.csv(data.frame(ks_pvalue = ks_p, n = n), paths$res_unif, row.names = FALSE)
# Inversione CDF-PPF
p.grid <- c(1e-6, 1e-5, 1e-4, seq(0.001, 0.999, by = 0.001), 1 - 1e-4, 1 - 1e-5, 1 - 1e-6)
q_inv <- qsn(p.grid, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"], solver = "RFB")
p_back <- psn(q_inv, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"])
inv_err <- data.frame(p = p_grid, p_back = p_back, abs_err = abs(p_back - p_grid))
write.csv(inv_err, paths$res_inversion, row.names = FALSE)
# QQ su z^2 vs z^2_1
z_hat <- (y - parhat["xi"]) / parhat["omega"]
z2 <- sort(z_hat^2)
chi2.q <- qchisq((1:n - 0.5) / n, df = 1)
png(paths$fig_QQ_chi2, width = 900, height = 700, res = 120)
plot(chi2.q, z2, pch = 20, main = "QQ di z^2 vs z^2_1 (R)",
      xlab = "Quantili z^2_1", ylab = "Quantili di z^2")
abline(0, 1, lwd = 2, col = "gray40")
dev.off()
# Profilo di verosimiglianza in
alpha_grid <- seq(-3, 3, length.out = 61)
profile <- lapply(alpha_grid, function(a) {
  f <- function(pr) -sum(log(dsn(y, xi = pr[1], omega = abs(pr[2]), alpha = a)))
  o <- optim(c(parhat["xi"], parhat["omega"]), f, method = "BFGS",
             control = list(reltol = 1e-10, maxit = 2000))
  c(alpha = a, xi_hat = o$par[1], omega_hat = abs(o$par[2]), loglik = -o$value)
})
prof_df <- as.data.frame(do.call(rbind, profile))
prof_df$deviance <- 2 * (llhat - prof_df$loglik)
write.csv(prof_df, paths$res_profile, row.names = FALSE)
png(paths$fig_profile, width = 900, height = 700, res = 120)
plot(prof_df$alpha, prof_df$deviance, type = "l", lwd = 2,
      main = "Profilo di verosimiglianza in (R)",
      xlab = "", ylab = "Deviance D()")
```

```

abline(v = 0, lty = 2, col = "gray40")
dev.off()

# SN univariata (dataset, stima, diagnostica)
# Primitive: scipy.stats.skewnorm {pdf, cdf, ppf, rvs, logpdf}
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg") # backend non interattivo
import matplotlib.pyplot as plt
from scipy import stats
from scipy.optimize import minimize

# Carica dataset condiviso
df = pd.read_csv(paths["data"])
y = df["y"].to_numpy()
n = y.size
# Stima ML (skewnorm.fit)
a_hat, loc_hat, scale_hat = stats.skewnorm.fit(y) # a alpha, loc xi, scale omega
# Log-verosimiglianza al massimo
loglik = float(np.sum(stats.skewnorm.logpdf(y, a=a_hat, loc=loc_hat, scale=scale_hat)))
# Diagnostica: QQ teorico SN
p = (np.arange(1, n + 1) - 0.5) / n
q_th = stats.skewnorm.ppf(p, a=a_hat, loc=loc_hat, scale=scale_hat)
q_emp = np.sort(y)
plt.figure(figsize=(7.5, 6))
plt.plot(q_th, q_emp, ".", ms=4)
plt.plot([q_th.min(), q_th.max()], [q_th.min(), q_th.max()], "-", lw=2)
plt.title("QQ-plot SN (Python)")
plt.xlabel("Quantili teorici SN"); plt.ylabel("Quantili empirici")
plt.tight_layout(); plt.savefig(paths["fig-QQ-sn"], dpi=120); plt.close()
# Diagnostica: UNIF + test di uniformità
u = stats.skewnorm.cdf(y, a=a_hat, loc=loc_hat, scale=scale_hat)
plt.figure(figsize=(7.5, 6))
plt.hist(u, bins=30)
plt.title("UNIF (Python)")
plt.xlabel("u = FSN(y); ")
plt.tight_layout(); plt.savefig(paths["fig-UNIF"], dpi=120); plt.close()
ks_p = float(stats.kstest(u, "uniform", args=(0, 1)).pvalue)
pd.DataFrame([{"ks_pvalue": ks_p, "n": int(n)}]).to_csv(paths["res-unif"], index=False)
# Inversione CDF-PPF
p_grid = np.concatenate([[1e-6, 1e-5, 1e-4], np.linspace(0.001, 0.999, 999), [1 - 1e-4, 1 - 1e-5, 1 - 1e-6]])
q_inv = stats.skewnorm.ppf(p_grid, a=a_hat, loc=loc_hat, scale=scale_hat)
p_back = stats.skewnorm.cdf(q_inv, a=a_hat, loc=loc_hat, scale=scale_hat)
inv_err = pd.DataFrame({"p": p_grid, "p_back": p_back, "abs_err": np.abs(p_back - p_grid)})
inv_err.to_csv(paths["res-inversion"], index=False)
# QQ su z^2 vs z^2_1
z_hat = (y - loc_hat) / scale_hat
z2 = np.sort(z_hat ** 2)
chi2_q = stats.chi2(df=1).ppf((np.arange(1, n + 1) - 0.5) / n)
plt.figure(figsize=(7.5, 6))
plt.plot(chi2_q, z2, ".", ms=4)
plt.plot([chi2_q.min(), chi2_q.max()], [chi2_q.min(), chi2_q.max()], "-", lw=2)
plt.title("QQ di z^2 vs z^2_1 (Python)")
plt.xlabel("Quantili z^2_1"); plt.ylabel("Quantili di z^2")
plt.tight_layout(); plt.savefig(paths["fig-QQ-chi2"], dpi=120); plt.close()
# Profilo di verosimiglianza in alpha
alpha_grid = np.linspace(-3.0, 3.0, 61)
prof = []
def nll_cond_alpha(a, par):
    """NLL condizionata su alpha, con parametrizzazione (xi, log(omega))."""
    xi = par[0]
    om = np.exp(par[1]) # vincolo omega>0
    return -np.sum(stats.skewnorm.logpdf(y, a=a, loc=xi, scale=om))
for a in alpha_grid:
    x0 = np.array([loc_hat, np.log(scale_hat)])
    res = minimize(lambda v: nll_cond_alpha(a, v), x0,
                  method="BFGS", options={"maxiter": 2000, "gtol": 1e-8})
    xi_hat_a = float(res.x[0])
    om_hat_a = float(np.exp(res.x[1]))
    ll_a = -float(res.fun)
    prof.append((a, xi_hat_a, om_hat_a, ll_a))
prof_df = pd.DataFrame(prof, columns=["alpha", "xi_hat", "omega_hat", "loglik"])
prof_df["deviance"] = 2.0 * (loglik - prof_df["loglik"])
prof_df.to_csv(paths["res-profile"], index=False)
plt.figure(figsize=(7.5, 6))
plt.plot(prof_df["alpha"], prof_df["deviance"], "-")
plt.axvline(0.0, ls="--")
plt.title("Profilo di verosimiglianza in (Python)")
plt.xlabel(""); plt.ylabel("Deviance D()")
plt.tight_layout(); plt.savefig(paths["fig-profile"], dpi=120); plt.close()

# Stress test alpha
library(sn)
alpha_grid <- c(-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0)
n <- 2000
xi <- 0
omega <- 1

all_fit <- list()
for (a in alpha_grid) {
    # (A) Dati condivisi
    set.seed(20250303 + as.integer(round((a + 10)*1000)))
    y <- rsn(n, xi = xi, omega = omega, alpha = a)

```

```

fdata <- sprintf("%s/sn_uni_alpha_%s.csv", dirs$data, gsub("\\.", "p", sprintf("%.1f", a)))
write.csv(data.frame(y=y), fdata, row.names = FALSE)

# (B) Stima ML
loglik_sn <- function(par, y) {
  xi <- par[1]; om <- abs(par[2]); al <- par[3]
  sum(log(dsn(y, xi=xi, omega=om, alpha=al)))
}
start <- c(median(y), IQR(y)/1.349, sign(a)*max(0.1, min(2, abs(a)))) # start non-degenere
fit <- optim(start, fn=function(p) -loglik_sn(p, y),
  method="BFGS", control=list(reltol=1e-10, maxit=5000))
parhat <- c(xi=fit$par[1], omega=abs(fit$par[2]), alpha=fit$par[3])
llhat <- -fit$value

# (C) Diagnostiche sintetiche: inversione CDF-PPF (core + code)
p_core <- seq(0.001, 0.999, by=0.001)
p_ext <- c(1e-6, 1e-5, 1e-4, 1-1e-4, 1-1e-5, 1-1e-6)
p_grid <- c(p_ext[1:3], p_core, p_ext[4:6])

q_inv <- qsn(p_grid, xi=parhat["xi"], omega=parhat["omega"], alpha=parhat["alpha"])
p_back <- psn(q_inv, xi=parhat["xi"], omega=parhat["omega"], alpha=parhat["alpha"])
inv_core <- max(abs(p_back[4:(length(p_back)-3)] - p_core))
inv_ext <- max(abs(p_back[c(1:3, (length(p_back)-2):length(p_back))] -
  c(p_ext[1:3], p_ext[4:6])))

# (D) UNIF uniformità (KS)
u <- psn(y, xi=parhat["xi"], omega=parhat["omega"], alpha=parhat["alpha"])
ks_p <- suppressWarnings(ks.test(u, "punif")$p.value)

# (E) Salvataggi
all_fit[[length(all_fit)+1]] <- data.frame(
  engine="R", alpha_true=a, xi_hat=parhat["xi"], omega_hat=parhat["omega"], alpha_hat=parhat["alpha"],
  loglik=llhat, n=n, inv_core_maxerr=inv_core, inv_ext_maxerr=inv_ext, ks_pvalue=ks_p
)
}

fit_out <- do.call(rbind, all_fit)
write.csv(fit_out, file.path(dirs$tab, "sn_uni_grid_R.csv"), row.names=FALSE)
cat("R grid: scritto\n")

# stress test alpha
import os, numpy as np, pandas as pd
from scipy import stats
alpha_grid = [-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0]
DATA_DIR = "common/data"
TAB_DIR = "common/tables"
os.makedirs(TAB_DIR, exist_ok=True)
def read_y(a):
  tag = str(round(a,1)).replace(".", "p")
  fpath = os.path.join(DATA_DIR, f"sn_uni_alpha_{tag}.csv")
  return pd.read_csv(fpath)["y"].to_numpy()
rows = []
for a in alpha_grid:
  y = read_y(a); n = y.size

  # Stima SciPy (DP: a, loc=xi, scale=omega)
  a_hat, loc_hat, scale_hat = stats.skewnorm.fit(y)

  # Loglik
  loglik = float(np.sum(stats.skewnorm.logpdf(y, a=a_hat, loc=loc_hat, scale=scale_hat)))

  # Inversione cdf<->ppf
  p_core = np.linspace(0.001, 0.999, 999)
  p_ext = np.array([1e-6, 1e-5, 1e-4, 1-1e-4, 1-1e-5, 1-1e-6])
  p_grid = np.concatenate([p_ext[:3], p_core, p_ext[3:]])
  q_inv = stats.skewnorm.ppf(p_grid, a=a_hat, loc=loc_hat, scale=scale_hat)
  p_back = stats.skewnorm.cdf(q_inv, a=a_hat, loc=loc_hat, scale=scale_hat)
  inv_core = float(np.max(np.abs(p_back[3:-3] - p_core)))
  inv_ext = float(np.max(np.abs(np.concatenate([p_back[:3], p_back[-3:]]) - p_ext)))

  # UNIF uniformità
  u = stats.skewnorm.cdf(y, a=a_hat, loc=loc_hat, scale=scale_hat)
  ks_p = float(stats.kstest(u, "uniform", args=(0,1)).pvalue)

  rows.append({
    "engine": "Python", "alpha_true": a,
    "xi_hat": float(loc_hat), "omega_hat": float(scale_hat), "alpha_hat": float(a_hat),
    "loglik": loglik, "n": int(n),
    "inv_core_maxerr": inv_core, "inv_ext_maxerr": inv_ext, "ks_pvalue": ks_p
  })
pd.DataFrame(rows).to_csv(os.path.join(TAB_DIR, "sn_uni_grid_Py.csv"), index=False)
print("Python grid: scritto")

```


Normale Asimmetrica multivariata

```
# SN multivariata
library(sn)
set.seed(20250303)
# Caricamento dati
load_or_simulate <- function() {
  if (file.exists(paths$data_in)) {
    df <- read.csv(paths$data_in)
    X <- as.matrix(df)
    colnames(X) <- if (is.null(colnames(X))) paste0("x", seq_len(ncol(X))) else colnames(X)
    return(list(X = X, from_file = TRUE))
  }
}
ld <- load_or_simulate()
Y <- ld$X
n <- nrow(Y)
p <- ncol(Y)
# Parametri in un unico vettore: theta = (xi_1..xi_p, eta_1..eta_p, alpha_1..alpha_p)
# con Omega = diag(exp(eta)) -> var_i = exp(eta_i) > 0.
unpack_theta <- function(theta) {
  stopifnot(length(theta) == 3L * p)
  xi <- theta[1:p]
  eta <- theta[(p + 1):(2 * p)]
  alpha <- theta[(2 * p + 1):(3 * p)]
  list(xi = xi, eta = eta, alpha = alpha)
}
nll_diag <- function(theta, Y) {
  pa <- unpack_theta(theta)
  Omega <- diag(exp(pa$eta), nrow = p, ncol = p) # varianze marginali
  -sum(sn::dmsn(Y, xi = pa$xi, Omega = Omega, alpha = pa$alpha, log = TRUE))
}
# Inizializzazione robusta
col_skew <- function(y) {
  yc <- sweep(y, 2L, colMeans(y), FUN = "-")
  s <- sqrt(colMeans(yc^2))
  z <- sweep(yc, 2L, s, FUN = "/" )
  colMeans(z^3)
}
xi0 <- colMeans(Y)
eta0 <- log(pmax(1e-6, diag(var(Y)))) # log-varianze iniziali
sgn <- sign(col_skew(Y)); sgn[sgn == 0] <- 1
alpha0 <- 0.5 * sgn # lontano da 0, ma moderato
th0 <- c(xi0, eta0, alpha0)
# Ottimizzazione ML
cat(sprintf("[INFO] Avvio ottimizzazione (p=%d, n=%d) ...\n", p, n))
ctrl <- list(reltol = 1e-10, maxit = 2000)
set.seed(20250303)
fit <- optim(par = th0, fn = nll_diag, Y = Y, method = "BFGS", control = ctrl)
stopifnot(is.list(fit), is.finite(fit$value))
par_hat <- unpack_theta(fit$par)
Omega_hat <- diag(exp(par_hat$eta), nrow = p, ncol = p)
loglik <- -fit$value
conv <- fit$convergence
fevals <- if (!is.null(fit$counts[["function"]])) as.integer(fit$counts[["function"]]) else NA_integer_
cat(sprintf("[OK] Ottimizzazione terminata (conv=%d, loglik=%.6f)\n", conv, loglik))
# Pairs plot
png(paths$fig_pairs, width = 1100, height = 900, res = 120)
par(mfrow = c(1,1))
pairs(Y, pch = 20, main = sprintf("Pairs plot SN (R) - p=%d, n=%d", p, n))
dev.off()
# Salvataggi risultati
# (1) Stime sintetiche in un'unica riga (compatibile con merge LEAN)
omega_diag_vals <- as.numeric(diag(Omega_hat)) # varianze marginali
omega_diag_str <- paste(format(omega_diag_vals, digits = 10), collapse = ";")
out <- data.frame(
  engine = "R",
  p = p,
  loglik = loglik,
  fevals = fevals,
  convergence = conv,
  omega_hat_diag = omega_diag_str,
  stringsAsFactors = FALSE
)
for (j in seq_len(p)) out[[sprintf("xi_hat_%d", j)]] <- par_hat$xi[j]
for (j in seq_len(p)) out[[sprintf("alpha_hat_%d", j)]] <- par_hat$alpha[j]
write.csv(out, paths$fit_csv, row.names = FALSE)
# (2) Omega in formato long (i,j,value)
long_Omega <- do.call(rbind, lapply(seq_len(p), function(i)
  do.call(rbind, lapply(seq_len(p), function(j)
    data.frame(i = i, j = j, value = Omega_hat[i, j])
  ))
))
write.csv(long_Omega, paths$fit_Omega, row.names = FALSE)

# SN multivariata
from __future__ import annotations
import os
import math
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg") # backend non interattivo per esecuzioni batch
import matplotlib.pyplot as plt
from scipy.optimize import minimize
```

```

import importlib
import importlib.util
from types import SimpleNamespace
def _load_msn_module():
    # 1) try common import names
    candidates = [
        "multivariate_skew_normal",
        "multivariate_skew_distribution.multivariate_skew_normal",
        "multivariate_skew_distribution.multivariate_skew_normal.multivariate_skew_normal",
    ]
    for name in candidates:
        try:
            mod = importlib.import_module(name)
            if hasattr(mod, "rvs") and hasattr(mod, "logpdf"):
                print(f"[INFO] Loaded MSN module '{name}' via import")
                return SimpleNamespace(rvs=mod.rvs, logpdf=mod.logpdf)
            if hasattr(mod, "multivariate_skew_normal"):
                sub = getattr(mod, "multivariate_skew_normal")
                if hasattr(sub, "rvs") and hasattr(sub, "logpdf"):
                    print(f"[INFO] Loaded MSN from '{name}.multivariate_skew_normal'")
                    return SimpleNamespace(rvs=sub.rvs, logpdf=sub.logpdf)
        except Exception:
            continue

    # 2) try to load from plausible file locations
    here = os.path.dirname(__file__) if '__file__' in globals() else os.getcwd()
    cand_files = [
        os.path.join(here, "multivariate_skew_normal.py"),
        os.path.join(here, "multivariate_skew_distribution", "multivariate_skew_normal.py"),
        os.path.join(here, "..", "multivariate_skew_distribution", "multivariate_skew_normal.py"),
        os.path.join(os.getcwd(), "multivariate_skew_distribution", "multivariate_skew_normal.py"),
    ]
    for p in cand_files:
        p = os.path.normpath(p)
        if os.path.exists(p):
            try:
                spec = importlib.util.spec_from_file_location("multivariate_skew_normal_local", p)
                mod = importlib.util.module_from_spec(spec)
                spec.loader.exec_module(mod) # type: ignore
                if hasattr(mod, "rvs") and hasattr(mod, "logpdf"):
                    print(f"[INFO] Loaded MSN from file: {p}")
                    return SimpleNamespace(rvs=mod.rvs, logpdf=mod.logpdf)
                if hasattr(mod, "multivariate_skew_normal"):
                    sub = getattr(mod, "multivariate_skew_normal")
                    if hasattr(sub, "rvs") and hasattr(sub, "logpdf"):
                        print(f"[INFO] Loaded MSN (sub) from file: {p}")
                        return SimpleNamespace(rvs=sub.rvs, logpdf=sub.logpdf)
            except Exception as e:
                print(f"[WARN] Failed loading MSN from {p}: {e}")
                continue

    # 3) fallback: independent-marginals Skew-Normal (requires scipy)
    try:
        from scipy.stats import skewnorm
    except Exception:
        raise ModuleNotFoundError(
            "multivariate_skew_normal module not found and scipy.stats.skewnorm unavailable. "
            "Install scipy or add multivariate_skew_distribution/multivariate_skew_normal.py to the repo."
        )

class FallbackMSN:
    @staticmethod
    def rvs(loc, scale, shape, size, random_state=None):
        loc = np.asarray(loc)
        shape = np.asarray(shape)
        if np.ndim(size) == 0:
            n = int(size)
        else:
            n = int(size)
        p = loc.size
        if np.ndim(scale) == 2:
            sd = np.sqrt(np.diag(scale))
        else:
            sd = np.asarray(scale)
        rng = np.random.default_rng() if random_state is None else random_state
        out = np.zeros((n, p))
        for j in range(p):
            a = float(shape[j])
            out[:, j] = skewnorm.rvs(a, loc=loc[j], scale=sd[j], size=n, random_state=rng)
        return out

    @staticmethod
    def logpdf(X, loc, scale, shape):
        X = np.atleast_2d(X)
        loc = np.asarray(loc)
        shape = np.asarray(shape)
        if np.ndim(scale) == 2:
            sd = np.sqrt(np.diag(scale))
        else:
            sd = np.asarray(scale)
        lp = np.zeros(X.shape[0])
        for j in range(loc.size):
            a = float(shape[j])
            lp += skewnorm.logpdf(X[:, j], a, loc=loc[j], scale=sd[j])
        return lp

```

```

print(["WARN"] 'multivariate_skew_normal' not found: using independent-marginals fallback (skewnorm).")
return SimpleNamespace(rvs=_FallbackMSN.rvs, logpdf=_FallbackMSN.logpdf)
# expose msn with expected API (rvs, logpdf)
msn = _load_msn_module()
def _load_or_simulate() -> tuple[np.ndarray, int]:
    """Carica common/data/sn_mv.csv se presente; altrimenti simula un dataset standard.
    Ritorna (Y, p) con Y di shape (n, p).
    """
    if os.path.exists(PATHS["data"]):
        df = pd.read_csv(PATHS["data"]) # atteso: colonne x1,...,xp
        Y = df.to_numpy(dtype=float)
        if Y.ndim != 2 or Y.shape[1] < 1:
            raise ValueError("Il CSV dei dati deve avere almeno una colonna di numeri.")
        return Y, Y.shape[1]
# Log-verosimiglianza SN_d
def _neg_loglik(theta: np.ndarray, Y: np.ndarray) -> float:
    """Negativa della log-verosimiglianza SN_d con diagonale.

    Parametrizzazione (per stabilità):
    theta = [ _1..p, _1..p, _1..p ], con _ii = exp(_i)
    (notare: qui usiamo come **matrice di varianza**, non la radice; quindi diag() = exp()).
    """
    n, p = Y.shape
    xi = theta[:p]
    eta = theta[p:2*p]
    alpha = theta[2*p:2*p+p]

    Omega_diag = np.exp(eta) # varianze > 0
    Omega = np.diag(Omega_diag)

    ll = msn.logpdf(Y, loc=xi, scale=Omega, shape=alpha)
    return -float(np.sum(ll))
# Stima ML
def _fit_ml(Y: np.ndarray) -> tuple[dict, np.ndarray]:
    """Stima ML"""
    n, p = Y.shape
    # Momenti campionari di base
    mu = np.mean(Y, axis=0)
    var = np.var(Y, axis=0, ddof=1)

    # Skewness campionaria per il segno dell'offset iniziale
    m3 = np.mean((Y - mu) ** 3, axis=0)
    s = np.sqrt(var)
    with np.errstate(divide='ignore', invalid='ignore'):
        skew = np.where(s > 0, m3 / np.maximum(s, 1e-12) ** 3, 0.0)
    alpha0 = 0.5 * np.sign(skew) # evita =0 (non regolare)

    # Parametri iniziali nel dominio unconstrained
    eps = 1e-6
    xi0 = mu
    eta0 = np.log(np.maximum(var, eps))
    theta0 = np.concatenate([xi0, eta0, alpha0])

    # Ottimizzazione
    res = minimize(_neg_loglik, theta0, args=(Y,), method="BFGS",
        options={"gtol": 1e-8, "maxiter": 2000})

    if not res.success:
        print(["WARN"] Ottimizzazione non convergente:", res.message)

    th = res.x
    xi_hat = th[:p]
    eta_hat = th[p:2*p]
    alpha_hat = th[2*p:2*p+p]
    Omega_hat_diag = np.exp(eta_hat)

    out = {
        "engine": "Python",
        "p": int(p),
        "loglik": float(-res.fun),
        **{f"xi_hat_{j+1}": float(xi_hat[j]) for j in range(p)},
        **{f"alpha_hat_{j+1}": float(alpha_hat[j]) for j in range(p)},
        "omega_hat_diag": ";".join(f"{v:.12g}" for v in Omega_hat_diag),
    }
    return out, Omega_hat_diag
# Pairs plot
def _pairs_plot(Y: np.ndarray, path_png: str) -> None:
    """Pairs-plot minimale (triangolo superiore) per diagnostica grafica."""
    n, p = Y.shape
    fig, axes = plt.subplots(p, p, figsize=(2.6*p, 2.6*p))
    for i in range(p):
        for j in range(p):
            ax = axes[i, j]
            if i == j:
                ax.hist(Y[:, j], bins=30)
            elif i < j:
                ax.scatter(Y[:, j], Y[:, i], s=5)
            else:
                ax.axis('off')
            if i == p-1:
                ax.set_xlabel(f"x{j+1}")
            else:
                ax.set_xticklabels([])
            if j == 0 and i != 0:
                ax.set_ylabel(f"x{i+1}")

```

```

        elif j != 0:
            ax.set_yticklabels([])
            plt.tight_layout()
            fig.savefig(path_png, dpi=120)
            plt.close(fig)

# Stress test alpha
library(sn)
alpha_grid <- c(-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0) # griglia alpha (slant)
n <- 2000 # numerosità per scenario
p <- 3 # dimensione
xi_true <- c(0, 1, -1) # location di verità-terreno
Omega_true <- diag(rep(1, p)) # vera: diagonale unità
# Stima ML (Omega diag)
# Parametri da ottimizzare: theta = (xi_1..xi_p, eta_1..eta_p, alpha_1..alpha_p)
# con Omega = diag(exp(eta)), quindi positività assicurata via exp.
nll_msn_diag <- function(theta, X) {
    d <- ncol(X)
    xi <- theta[1:d]
    eta <- theta[(d+1):(2*d)]
    alpha <- theta[(2*d+1):(3*d)]
    Omega <- diag(exp(eta), d, d)

    if (any(!is.finite(Omega))) return(.Machine$double.xmax * 1e-5)

    val <- tryCatch({
        llvec <- sn::dmsn(X, xi = xi, Omega = Omega, alpha = alpha, log = TRUE)
        if (any(!is.finite(llvec))) .Machine$double.xmax * 1e-5 else -sum(llvec)
    }, error = function(e) .Machine$double.xmax * 1e-5)

    if (!is.finite(val)) val <- .Machine$double.xmax * 1e-5
    val
}

# Costruzione start robusti per (xi, eta, alpha) dato X e uno "scale" per alpha
make_start <- function(X, a_scalar) {
    d <- ncol(X)
    xi0 <- apply(X, 2, median)
    sc <- (apply(X, 2, IQR) / 1.349)
    var0 <- sc^2
    var0[!is.finite(var0)] <- 1e-3
    var0 <- pmax(var0, 1e-3)
    eta0 <- log(var0)
    a0 <- sign(a_scalar) * max(0.1, min(2, abs(a_scalar)))
    alpha0 <- rep(a0, d)
    c(xi0, eta0, alpha0)
}

# Loop su griglia alpha
all_fit <- list()
for (a in alpha_grid) {
    # Dati condivisi
    set.seed(20250303 + as.integer(round((a + 10) * 1000)))

    # validate inputs used for generation
    tryCatch({
        validate_finite_matrix(as.matrix(xi_true), "xi_true")
        if (!is.matrix(Omega_true)) Omega_true <- as.matrix(Omega_true)
        validate_finite_matrix(Omega_true, "Omega_true")
        validate_finite_matrix(matrix(rep(a, p), nrow = p, ncol = 1), "alpha (candidate)")
    }, error = function(e) {
        stop(sprintf("Input validation failed before rmsn: %s", e$message))
    })

    X <- tryCatch({
        sn::rmsn(n = n, xi = xi_true, Omega = Omega_true, alpha = rep(a, p))
    }, error = function(e) {
        stop(sprintf("sn::rmsn failed for alpha=%s: %s\nxi_true=%s\nOmega_true(diag)=%s",
            format(a), e$message,
            paste(capture.output(print(xi_true)), collapse = ","),
            paste(diag(Omega_true), collapse = ",")))
    })

    # Salva dataset per consumo Python (colonne xi..xp)
    dfX <- as.data.frame(X)
    colnames(dfX) <- paste0("x", seq_len(p))
    fdata <- file.path(dirs$data, sprintf("sn_mv_alpha_a_%s.csv", alphas(a)))
    write.csv(dfX, fdata, row.names = FALSE)

    # Stima ML trasparente
    start <- make_start(X, a)
    fit <- optim(
        par = start,
        fn = function(th) nll_msn_diag(th, X),
        method = "BFGS",
        control = list(reltol = 1e-10, maxit = 5000)
    )

    # Estrai parametri e logLik
    xi_hat <- fit$par[1:p]
    eta_hat <- fit$par[(p+1):(2*p)]
    alpha_hat <- fit$par[(2*p+1):(3*p)]
    omega_hat_diag <- exp(eta_hat) # varianze diagonali stimate
    loglik <- -fit$value

    # Accumula output
    row <- c(

```

```

        list(engine = "R", p = p, alpha_true = a, n = n, loglik = loglik),
        setNames(as.list(xi_hat), paste0("xi_hat_", seq_len(p))),
        setNames(as.list(omega_hat_diag), paste0("omega_hat_diag_", seq_len(p))),
        setNames(as.list(alpha_hat), paste0("alpha_hat_", seq_len(p)))
    )
    all_fit[[length(all_fit) + 1]] <- as.data.frame(row, check.names = FALSE)
}

# Stress test alpha
# Stessa sezione import di prima
ALPHA_GRID = [-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0]
DATA_DIR = "common/data"
TAB_DIR = "common/tables"
os.makedirs(TAB_DIR, exist_ok=True)
def tag_from_alpha(a: float) -> str:
    """Restituisce il tag stile R ('m1p2' per -1.2)."""
    s = f"{a:.1f}"
    return s.replace("-", "m").replace(".", "p")

def _alternate_tag(a: float) -> str:
    """Formato alternativo con segno esplicito (es. -1.2 -> '-1p2')."""
    return f"{a:.1f}".replace(".", "p")

def read_matrix_for_alpha(a: float) -> np.ndarray | None:
    """Legge X da common/data/sn_mv_alpha_a_{TAG}.csv; None se assente."""
    candidates = [tag_from_alpha(a), _alternate_tag(a)]
    fpath = None
    for tg in candidates:
        cand = os.path.join(DATA_DIR, f"sn_mv_alpha_a_{tg}.csv")
        if os.path.exists(cand):
            fpath = cand
            break
    if fpath is None:
        tried = ", ".join(os.path.join(DATA_DIR, f"sn_mv_alpha_a_{tg}.csv") for tg in candidates)
        print(f"[WARN] Dataset mancante per a={a}. Path provati: {tried}")
        return None

    df = pd.read_csv(fpath)
    X = df.select_dtypes(include=["number"]).to_numpy()
    if X.ndim != 2 or X.shape[0] == 0 or X.shape[1] == 0:
        print(f"[WARN] File privo di colonne numeriche utili: {fpath}")
        return None
    return X

# Verosimiglianza
def nll_msn_diag(params: np.ndarray, X: np.ndarray) -> float:
    """Negative log-likelihood per SN_p con diagonale.
    """
    n, p = X.shape
    xi = params[0:p]
    eta = params[p:2*p]
    alp = params[2*p:3*p]

    diag_var = np.exp(eta)
    if not np.all(np.isfinite(diag_var)):
        return 1e300

    try:
        ll = float(np.sum(multivariate_skew_normal.logpdf(X, loc=xi, scale=np.diag(diag_var), shape=alp)))
        if not np.isfinite(ll):
            return 1e300
        return -ll
    except Exception:
        return 1e300

def fit_msn_diag(X: np.ndarray, a_hint: float) -> Tuple[Dict[str, float], Dict[str, np.ndarray]]:
    """Stima ML per diagonale (BFGS)."""
    n, p = X.shape

    # Start robust
    xi0 = np.nanmedian(X, axis=0)
    v0 = np.nanvar(X, axis=0, ddof=1)
    v0[v0 <= 1e-12] = 1e-12
    eta0 = np.log(v0)
    mag = max(0.1, min(2.0, abs(a_hint)))
    alp0 = np.sign(a_hint) * np.full(p, mag, dtype=float)
    x0 = np.concatenate([xi0, eta0, alp0])

    # Ottimizzazione
    opt = minimize(
        fun=nll_msn_diag,
        x0=x0,
        args=(X,),
        method="BFGS",
        options={"maxiter": 5000, "gtol": 1e-8}
    )

    par = opt.x
    xi_hat = par[0:p]
    eta_hat = par[p:2*p]
    alp_hat = par[2*p:3*p]
    var_hat = np.exp(eta_hat)
    ll = -float(opt.fun)

    scalars = {

```

```

        "n": float(n),
        "p": float(p),
        "loglik": ll,
        "fevals": float(opt.nfev if hasattr(opt, "nfev") else np.nan),
        "convergence": float(opt.status),
    }
    vectors = {"xi_hat": xi_hat, "alpha_hat": alp_hat, "omega_hat_diag": var_hat}
    return scalars, vectors
# Main loop
rows = []

for a in ALPHA_GRID:
    # (A) Read
    X = read_matrix_for_alpha(a)
    if X is None:
        continue

    # (B) Fit ( diagonale)
    scalars, vectors = fit_msn_diag(X, a_hint=a)

    # (C) Riga "wide" per merge
    p = int(scalars["p"]) if "p" in scalars else X.shape[1]
    row = {
        "engine": "Python",
        "alpha_true": float(a),
        "p": p,
        "loglik": float(scalars["loglik"]),
        "n": int(scalars["n"]),
        "fevals": float(scalars["fevals"]),
        "convergence": float(scalars["convergence"]),
    }
    for j in range(p):
        row[f"xi_hat_{j+1}"] = float(vectors["xi_hat"][j])
        row[f"alpha_hat_{j+1}"] = float(vectors["alpha_hat"][j])
        row[f"omega_hat_diag_{j+1}"] = float(vectors["omega_hat_diag"][j])

    rows.append(row)

```

t Asimmetrica univariata

```
#!/usr/bin/env Rscript
# ST univariata
library(sn)
set.seed(20250303)
# Parametri "veri" per la generazione
n <- 2000
xi <- 0
omega <- 1
alpha <- 1.2
nu <- 6 # medio (momenti fino al 4° esistono)
# Soglia per
SEM_NU_MIN <- suppressWarnings(as.numeric(Sys.getenv("ST_SEM_NU_MIN", unset = "2.05")))
if (!is.finite(SEM_NU_MIN) || SEM_NU_MIN <= 0) SEM_NU_MIN <- 2.05
# Bound numerici
LOG_OM_MIN <- log(1e-8) # > 0
LOG_NU_MIN <- log(SEM_NU_MIN) # >= soglia semantica
# Controlli L-BFGS-B
LBFGSB_CTRL <- list(maxit = 5000, factr = 1e7, pgtol = 1e-8)
# NLL
# NLL "safe" con penalizzazione (parametri: xi, log(omega), alpha, log(nu))
nll_st_par <- function(par, y) {
  xi <- par[1]
  lom <- par[2]
  al <- par[3]
  lnu <- par[4]
  if (!is.finite(lom) || !is.finite(lnu)) return(1e50)

  om <- exp(lom); nv <- exp(lnu)
  if (!is.finite(om) || om <= 0) return(1e50)
  if (!is.finite(nv) || nv < SEM_NU_MIN) return(1e50)

  dens <- suppressWarnings(try(dst(y, xi = xi, omega = om, alpha = al, nu = nv), silent = TRUE))
  if (inherits(dens, "try-error") || any(!is.finite(dens)) || any(dens <= 0)) return(1e50)

  -sum(log(dens))
}
# Dati: generazione + salvataggio
y <- rst(n, xi = xi, omega = omega, alpha = alpha, nu = nu)
write.csv(data.frame(y = y), paths$data, row.names = FALSE)
# Stima HL (L-BFGS-B con vincoli)
start <- c(median(y), log(IQR(y)/1.349), 0.5, log(max(SEM_NU_MIN + 0.5, 6)))
fit <- optim(
  par = start,
  fn = function(p) nll_st_par(p, y),
  method = "L-BFGS-B",
  lower = c(-Inf, LOG_OM_MIN, -Inf, LOG_NU_MIN),
  control = LBFGSB_CTRL
)
parhat <- c(
  xi = fit$par[1],
  omega = exp(fit$par[2]),
  alpha = fit$par[3],
  nu = exp(fit$par[4])
)
llhat <- -fit$value
fevals <- if (!is.null(fit$counts["function"])) as.integer(fit$counts["function"]) else NA_integer_
conv <- fit$convergence
msg <- if (!is.null(fit$message)) as.character(fit$message) else ""
# Diagnostica: QQ teorico ST
p <- (1:n - 0.5) / n
q_th <- qst(p, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"], nu = parhat["nu"])
q_emp <- sort(y)
png(paths$fig_QQ_st, width = 900, height = 700, res = 120)
plot(q_th, q_emp, pch = 20, main = "QQ-plot ST (R)",
  xlab = "Quantili teorici ST", ylab = "Quantili empirici")
abline(0, 1, lwd = 2, col = "gray40")
dev.off()
# Diagnostica: UNIF + KS
u <- pst(y, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"], nu = parhat["nu"])
png(paths$fig_UNIF, width = 900, height = 700, res = 120)
hist(u, breaks = 30, freq = TRUE, main = "UNIF (R)", xlab = "u = F_ST(y; )")
abline(h = length(u)/30, lwd = 2, col = "gray40")
dev.off()
ks_p <- suppressWarnings(ks.test(u, "punif")$p.value)
write.csv(data.frame(ks_pvalue = ks_p, n = n), paths$res_unif, row.names = FALSE)
# Inversione CDF-PPF
p_grid <- c(1e-6, 1e-5, 1e-4, seq(0.001, 0.999, by = 0.001), 1 - 1e-4, 1 - 1e-5, 1 - 1e-6)
q_inv <- suppressWarnings(qst(p_grid, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"], nu =
  parhat["nu"]))
p_back <- suppressWarnings(pst(q_inv, xi = parhat["xi"], omega = parhat["omega"], alpha = parhat["alpha"], nu
  = parhat["nu"]))
inv_err <- data.frame(p = p_grid, p_back = p_back, abs_err = abs(p_back - p_grid))
write.csv(inv_err, paths$res_inversion, row.names = FALSE)
# Profilo in alpha
alpha_grid <- seq(-3, 3, length.out = 61)
prof_a <- lapply(alpha_grid, function(a) {
  f <- function(pr) { # pr = (xi, log(omega), log(nu))
    xi <- pr[1]; lom <- pr[2]; lnu <- pr[3]
    nll_st_par(c(xi, lom, a, lnu), y)
  }
  o <- optim(c(parhat["xi"], log(parhat["omega"]), log(parhat["nu"])),
    f, method = "L-BFGS-B",
    lower = c(-Inf, LOG_OM_MIN, LOG_NU_MIN),
    control = LBFGSB_CTRL)
```

```

c(alpha = a, xi_hat = o$par[1], omega_hat = exp(o$par[2]),
  nu_hat = exp(o$par[3]), loglik = -o$value)
})
prof_a_df <- as.data.frame(do.call(rbind, prof_a))
prof_a_df$deviance <- 2 * (llhat - prof_a_df$loglik)
write.csv(prof_a_df, paths$res_profile_a, row.names = FALSE)
png(paths$fig_profile_a, width = 900, height = 700, res = 120)
plot(prof_a_df$alpha, prof_a_df$deviance, type = "l", lwd = 2,
  main = "Profilo di verosimiglianza in (R, ST)", xlab = "", ylab = "Deviance D()")
abline(v = 0, lty = 2, col = "gray40")
dev.off()
# Profilo in nu
nu_grid <- exp(seq(log(max(SEM_NU_MIN, 2.05)), log(40), length.out = 60))
prof_n <- lapply(nu_grid, function(nv) {
  f <- function(pr) { # pr = (xi, log(omega), alpha)
    xi <- pr[1]; lom <- pr[2]; al <- pr[3]
    nll_st_par(c(xi, lom, al, log(nv)), y) # blocca log(nu)=log(nv)
  }
  o <- optim(c(parhat["xi"], log(parhat["omega"])), parhat["alpha"],
    f, method = "L-BFGS-B",
    lower = c(-Inf, LOG_OM_MIN, -Inf),
    control = LBFGS_CTRL)
  c(nu = nv, xi_hat = o$par[1], omega_hat = exp(o$par[2]),
    alpha_hat = o$par[3], loglik = -o$value)
})
prof_n_df <- as.data.frame(do.call(rbind, prof_n))
prof_n_df$deviance <- 2 * (llhat - prof_n_df$loglik)
write.csv(prof_n_df, paths$res_profile_n, row.names = FALSE)
# IC 95% dal profilo: D() D_min + ^2_{1,0.95}
chi2_1_095 <- qchisq(0.95, df = 1)
Dmin <- min(prof_n_df$deviance, na.rm = TRUE)
ok <- which(prof_n_df$deviance <= Dmin + chi2_1_095)
if (length(ok) >= 1) {
  nu_CI_low <- min(prof_n_df$nu[ok]); nu_CI_upp <- max(prof_n_df$nu[ok])
} else {
  nu_CI_low <- NA_real_; nu_CI_upp <- NA_real_
}
write.csv(data.frame(nu_hat = parhat["nu"], nu_CI_low = nu_CI_low, nu_CI_upp = nu_CI_upp,
  Dmin = Dmin, chi2_1_095 = chi2_1_095),
  paths$res_profile_n_CI, row.names = FALSE)
# Plot profilo nu + IC
png(paths$fig_profile_n, width = 900, height = 700, res = 120)
plot(prof_n_df$nu, prof_n_df$deviance, type = "l", lwd = 2,
  main = "Profilo di verosimiglianza in (R, ST)",
  xlab = "", ylab = "Deviance D()")
abline(h = Dmin + chi2_1_095, lty = 3, col = "gray40")
abline(v = parhat["nu"], lty = 2, col = "gray40")
if (!is.na(nu_CI_low) && !is.na(nu_CI_upp)) {
  segments(nu_CI_low, Dmin + chi2_1_095, nu_CI_upp, Dmin + chi2_1_095, lwd = 3, col = "gray30")
}
dev.off()

# ST univariata
import os, math
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg") # backend non interattivo
import matplotlib.pyplot as plt
from scipy import stats
# ----- import skewt-sciPy (nome modulo corretto) -----
try:
  from skewt_sciPy.skewt import skewt
except Exception as e:
  raise ImportError(
    "Impossibile importare 'skewt' dal pacchetto 'skewt-sciPy'. "
    "Installa/aggiorna con: pip install -U skewt-sciPy"
  ) from e
# Soglia per nu
SEM_NU_MIN = float(os.getenv("ST_SEM_NU_MIN", "2.05"))
if not np.isfinite(SEM_NU_MIN) or SEM_NU_MIN <= 0:
  SEM_NU_MIN = 2.05
# Carica dataset condiviso
if not os.path.exists(paths["data"]):
  raise FileNotFoundError(
    f"Dataset mancante: {paths['data']}. Genera prima i dati in R (rst) oppure crea il CSV."
  )
df = pd.read_csv(paths["data"])
y = df["y"].to_numpy()
n = y.size
# Stima ML con skewt.fit
a_hat, df_hat, loc_hat, scale_hat = skewt.fit(y) # a alpha, df nu, loc xi, scale omega
# Log-verosimiglianza al massimo
loglik = float(np.sum(skewt.logpdf(y, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)))
# Diagnostica: QQ teorico ST
p = (np.arange(1, n + 1) - 0.5) / n
q_th = skewt.ppf(p, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
q_emp = np.sort(y)
plt.figure(figsize=(7.5, 6))
plt.plot(q_th, q_emp, "-", ms=4)
mn, mx = float(np.min(q_th)), float(np.max(q_th))
plt.plot([mn, mx], [mn, mx], "-", lw=2)
plt.title("QQ-plot ST (Python)")
plt.xlabel("Quantili teorici ST"); plt.ylabel("Quantili empirici")

```



```

plt.tight_layout(); plt.savefig(paths["fig_QQ_st"], dpi=120); plt.close()
# Diagnostica: UNIF + KS
u = skewt.cdf(y, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
plt.figure(figsize=(7.5, 6))
plt.hist(u, bins=30)
plt.title("UNIF (Python)")
plt.xlabel("u = F_ST(y; )")
plt.tight_layout(); plt.savefig(paths["fig_UNIF"], dpi=120); plt.close()
ks_p = float(stats.kstest(u, "uniform", args=(0, 1)).pvalue)
pd.DataFrame({"ks_pvalue": ks_p, "n": int(n)}).to_csv(paths["res_unif"], index=False)
# Inversione CDF-PPF
p_grid = np.concatenate([[1e-6, 1e-5, 1e-4], np.linspace(0.001, 0.999, 999), [1 - 1e-4, 1 - 1e-5, 1 - 1e-6]])
q_inv = skewt.ppf(p_grid, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
p_back = skewt.cdf(q_inv, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
pd.DataFrame({"p": p_grid, "p_back": p_back, "abs_err": np.abs(p_back - p_grid)}).to_csv(paths["res_inversion"],
↪ index=False)
# Profilo in
alpha_grid = np.linspace(-3.0, 3.0, 61)
prof_a = []
def ll_given(a: float, df_: float, loc_: float, scale_: float) -> float:
    return float(np.sum(skewt.logpdf(y, a=a, df=df_, loc=loc_, scale=scale_)))
for a in alpha_grid:
    # fit con alpha bloccato (fa=a) per ottimizzare df, loc, scale
    a_fix, df_fix, loc_fix, scale_fix = skewt.fit(y, fa=float(a))
    ll = ll_given(a, df_fix, loc_fix, scale_fix)
    prof_a.append((a, loc_fix, scale_fix, df_fix, ll))

prof_a_df = pd.DataFrame(prof_a, columns=["alpha", "xi_hat", "omega_hat", "nu_hat", "loglik"])
prof_a_df["deviance"] = 2.0 * (loglik - prof_a_df["loglik"])
prof_a_df.to_csv(paths["res_profile_a"], index=False)

plt.figure(figsize=(7.5, 6))
plt.plot(prof_a_df["alpha"], prof_a_df["deviance"], "-")
plt.axvline(0.0, ls="--")
plt.title("Profilo di verosimiglianza in (Python, ST)")
plt.xlabel(""); plt.ylabel("Deviance D()")
plt.tight_layout(); plt.savefig(paths["fig_profile_a"], dpi=120); plt.close()
# Profilo in + IC 95%
nu_grid = np.exp(np.linspace(np.log(max(SEM_NU_MIN, 2.05)), np.log(40.0), 60))
prof_n = []

for nv in nu_grid:
    # fit con df bloccato (fdf=nv) per ottimizzare a, loc, scale
    a_fix, df_fix, loc_fix, scale_fix = skewt.fit(y, fdf=float(nv))
    ll = ll_given(a_fix, nv, loc_fix, scale_fix)
    prof_n.append((nv, loc_fix, scale_fix, a_fix, ll))

prof_n_df = pd.DataFrame(prof_n, columns=["nu", "xi_hat", "omega_hat", "alpha_hat", "loglik"])
prof_n_df["deviance"] = 2.0 * (loglik - prof_n_df["loglik"])
prof_n_df.to_csv(paths["res_profile_n"], index=False)

# IC da profilo (df=1)
chi2_1_095 = stats.chi2(df=1).ppf(0.95)
Dmin = float(np.nanmin(prof_n_df["deviance"]))
ok = np.where(prof_n_df["deviance"].to_numpy() <= Dmin + chi2_1_095)[0]
if ok.size >= 1:
    nu_CI_low = float(np.min(prof_n_df["nu"].to_numpy()[ok]))
    nu_CI_upp = float(np.max(prof_n_df["nu"].to_numpy()[ok]))
else:
    nu_CI_low = float("nan"); nu_CI_upp = float("nan")

pd.DataFrame([
    "nu_hat": float(df_hat),
    "nu_CI_low": nu_CI_low,
    "nu_CI_upp": nu_CI_upp,
    "Dmin": Dmin,
    "chi2_1_095": float(chi2_1_095),
]).to_csv(paths["res_profile_n_CI"], index=False)

plt.figure(figsize=(7.5, 6))
plt.plot(prof_n_df["nu"], prof_n_df["deviance"], "-")
plt.axhline(Dmin + chi2_1_095, ls=":", color="black")
plt.axvline(df_hat, ls="--", color="black")
if not (math.isnan(nu_CI_low) or math.isnan(nu_CI_upp)):
    plt.hlines(Dmin + chi2_1_095, nu_CI_low, nu_CI_upp, linewidth=3)
plt.title("Profilo di verosimiglianza in (Python, ST)")
plt.xlabel(""); plt.ylabel("Deviance D()")
plt.tight_layout(); plt.savefig(paths["fig_profile_n"], dpi=120); plt.close()

# stress test alpha
library(sn)
alpha_grid <- c(-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0)
n <- 2000
xi <- 0
omega <- 1
nu_true <- 6 # momenti fino al 4° ben definiti
# Soglia semantica per nu (varianza esistente). Per pretendere anche kurtosi: usa 4.05
SEM_NU_MIN <- suppressWarnings(as.numeric(Sys.getenv("ST_SEM_NU_MIN", unset = "2.05")))
if (!is.finite(SEM_NU_MIN) || SEM_NU_MIN <= 0) SEM_NU_MIN <- 2.05
# Bound
LOG_OM_MIN <- log(1e-8)
LOG_NU_MIN <- log(SEM_NU_MIN)
# Controlli L-BFGS-B
LBFGSB_CTRL <- list(maxit = 5000, factr = 1e7, pgtol = 1e-8)
# Directory

```

```

dirs <- list(
  data = "common/data",
  fig = "common/fig",
  tab = "common/tables"
)
for (d in dirs) dir.create(d, recursive = TRUE, showWarnings = FALSE)
# ----- NLL "safe" (xi, log(omega), alpha, log(nu)) -----
nll_st_par <- function(par, y) {
  xi <- par[1]; lom <- par[2]; al <- par[3]; lnu <- par[4]
  if (!is.finite(lom) || !is.finite(lnu)) return(1e50)
  om <- exp(lom); nv <- exp(lnu)
  if (!is.finite(om) || om <= 0) return(1e50)
  if (!is.finite(nv) || nv < SEM_NU_MIN) return(1e50)

  dens <- suppressWarnings(try(dst(y, xi = xi, omega = om, alpha = al, nu = nv), silent = TRUE))
  if (inherits(dens, "try-error") || any(!is.finite(dens)) || any(dens <= 0)) return(1e50)
  -sum(log(dens))
}
# Loop su alpha
all_fit <- list()
for (a in alpha_grid) {
  # (A) Dati
  set.seed(20250303 + as.integer(round((a + 10)*1000)))
  y <- rst(n, xi = xi, omega = omega, alpha = a, nu = nu_true)

  fdata <- sprintf("%s/st_uni_A_alpha_%s.csv", dirs$data, gsub("\\.", "p", sprintf("%.1f", a)))
  write.csv(data.frame(y = y), fdata, row.names = FALSE)

  # (B) Stima ML (xi, log(omega), alpha, log(nu)) con vincoli
  start_alpha <- if (a == 0) 0 else sign(a) * max(0.1, min(2, abs(a)))
  start <- c(median(y), log(IQR(y)/1.349), start_alpha, log(max(SEM_NU_MIN + 0.5, nu_true)))

  fit <- optim(
    par = start,
    fn = function(p) nll_st_par(p, y),
    method = "L-BFGS-B",
    lower = c(-Inf, LOG_OM_MIN, -Inf, LOG_NU_MIN),
    control = LBFGS_CTRL
  )

  parhat <- c(
    xi = fit$par[1],
    omega = exp(fit$par[2]),
    alpha = fit$par[3],
    nu = exp(fit$par[4])
  )
  llhat <- -fit$value

  # (C) Diagnostiche sintetiche: inversione CDFPPF (core + code) e UNIF (KS)
  p_core <- seq(0.001, 0.999, by = 0.001)
  p_ext <- c(1e-6, 1e-5, 1e-4, 1 - 1e-4, 1 - 1e-5, 1 - 1e-6)
  p_grid <- c(p_ext[1:3], p_core, p_ext[4:6])

  q_inv <- suppressWarnings(qst(p_grid, xi = parhat["xi"], omega = parhat["omega"],
    alpha = parhat["alpha"], nu = parhat["nu"]))
  p_back <- suppressWarnings(pst(q_inv, xi = parhat["xi"], omega = parhat["omega"],
    alpha = parhat["alpha"], nu = parhat["nu"]))
  inv_core <- max(abs(p_back[4:(length(p_back)-3)] - p_core))
  inv_ext <- max(abs(p_back[c(1:3, (length(p_back)-2):length(p_back))] -
    c(p_ext[1:3], p_ext[4:6])))

  u <- suppressWarnings(pst(y, xi = parhat["xi"], omega = parhat["omega"],
    alpha = parhat["alpha"], nu = parhat["nu"]))
  ks_p <- suppressWarnings(ks.test(u, "punif")$p.value) # UNIF: KS su U(0,1)

  # (D) Accumulo risultati
  all_fit[[length(all_fit)+1]] <- data.frame(
    engine="R", alpha_true=a,
    xi_hat=parhat["xi"], omega_hat=parhat["omega"],
    alpha_hat=parhat["alpha"], nu_hat=parhat["nu"],
    loglik=llhat, n=n,
    inv_core_maxerr=inv_core, inv_ext_maxerr=inv_ext, ks_pvalue=ks_p,
    nu_sem_threshold = SEM_NU_MIN
  )
}

# stress test alpha
# stessi import di prima
alpha_grid = [-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0]
DATA_DIR = "common/data"
TAB_DIR = "common/tables"
os.makedirs(TAB_DIR, exist_ok=True)

OUT_TAB = os.path.join(TAB_DIR, "st_uni_A_grid_Py.csv")

def data_path_for_alpha(a: float) -> str:
  tag = str(round(a, 1)).replace(".", "p")
  return os.path.join(DATA_DIR, f"st_uni_A_alpha_{tag}.csv")

def read_y(a: float) -> np.ndarray:
  fpath = data_path_for_alpha(a)
  if not os.path.exists(fpath):
    raise FileNotFoundError(
      f"Dataset mancante per alpha={a} -> {fpath}\n"
      "Generalo prima (RST in R) o crea il CSV con una colonna 'y'."
    )

```

```

    )
    return pd.read_csv(fpath)["y"].to_numpy()
rows = []

for a in alpha_grid:
    print(f"\n[={a:+.1f}] Carico dati...", flush=True)
    y = read_y(a)
    n = y.size
    print(f" → n = {n}", flush=True)

    # --- Fit ST (a, df, loc, scale) ---
    print(" Fit skew-t...", flush=True)
    a_hat, df_hat, loc_hat, scale_hat = skewt.fit(y) # aalpha, dfnu, locxi, scaleomega
    print(f" → stime: alpha={a_hat:.6g}, nu={df_hat:.6g}, xi={loc_hat:.6g}, omega={scale_hat:.6g}", flush=True)

    # --- Log-likelihood ---
    loglik = float(np.sum(skewt.logpdf(y, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)))

    # --- Inversione CDFPPF (core + code) ---
    p_core = np.linspace(0.001, 0.999, 999)
    p_ext = np.array([1e-6, 1e-5, 1e-4, 1-1e-4, 1-1e-5, 1-1e-6])
    p_grid = np.concatenate([p_ext[:3], p_core, p_ext[3:]])
    q_inv = skewt.ppf(p_grid, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
    p_back = skewt.cdf(q_inv, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
    inv_core = float(np.max(np.abs(p_back[3:-3] - p_core)))
    inv_ext = float(np.max(np.abs(np.concatenate([p_back[:3], p_back[-3:]] - p_ext)))

    # --- UNIF (PIT + KS) ---
    u = skewt.cdf(y, a=a_hat, df=df_hat, loc=loc_hat, scale=scale_hat)
    ks_p = float(stats.kstest(u, "uniform", args=(0, 1)).pvalue)

    # --- Accumula risultati ---
    rows.append({
        "engine": "Python",
        "alpha_true": a,
        "xi_hat": float(loc_hat),
        "omega_hat": float(scale_hat),
        "alpha_hat": float(a_hat),
        "nu_hat": float(df_hat),
        "loglik": loglik,
        "n": int(n),
        "inv_core_maxerr": inv_core,
        "inv_ext_maxerr": inv_ext,
        "ks_pvalue": ks_p,
    })

```

t Asimmetrica multivariata

```
# ST multivariata
library(sn)
DO_PAIRS <- TRUE # pairs plot leggero
set.seed(20250303)
n <- 2000; p <- 3
xi_true <- c(0, 1, -1)
Omega_true <- matrix(c(
  1.0, 0.4, 0.2,
  0.4, 1.0, 0.3,
  0.2, 0.3, 1.0
), nrow = p, byrow = TRUE)
alpha_true <- c(2.0, -1.0, 1.5)
nu_true <- 8
# Dati
Y <- sn::rmst(n, xi = xi_true, Omega = Omega_true, alpha = alpha_true, nu = nu_true)
colnames(Y) <- sprintf("%02d", seq_len(ncol(Y)))
write.csv(as.data.frame(Y), paths$data, row.names = FALSE)
# Stima ML via selm (family = "ST")
# Nota: uso di selm con sola intercetta.
Ydf <- as.data.frame(Y)
form <- as.formula(paste0("cbind(", paste(names(Ydf), collapse = ","), ") ~ 1"))
fit <- sn::selm(form, data = Ydf, family = "ST") # MLE per MST con intercetta
# Estrazione parametri DP (xi, Omega, alpha, nu)
cf <- sn::coef(fit, "DP", vector = FALSE)
xi_hat <- as.numeric(cf$beta)
Omega_hat <- as.matrix(cf$Omega)
alpha_hat <- as.numeric(cf$alpha)
nu_hat <- as.numeric(cf$nu)
# Log-verosimiglianza
llhat <- if (!is.null(fit@logL)) as.numeric(fit@logL) else {
  dens <- sn::dmst(as.matrix(Y), xi = xi_hat, Omega = Omega_hat, alpha = alpha_hat, nu = nu_hat)
  sum(log(dens))
}
# Diagnostica: pairs
if (DO_PAIRS) {
  png(paths$fig_pairs, width = 1000, height = 900, res = 120)
  sub_idx <- if (nrow(Y) > 3000) sample.int(nrow(Y), 3000) else seq_len(nrow(Y))
  pairs(as.data.frame(Y[sub_idx, , drop = FALSE]), main = "ST multivariata - pairs plot (R)")
  dev.off()
}

# ST multivariata
import os, sys
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from pandas.plotting import scatter_matrix
from scipy.optimize import minimize
DO_PAIRS = True # pairs plot su sottocampione
# ----- Setup import modulo locale -----
PROJ_ROOT = Path(__file__).resolve().parents[1]
CANDIDATES = [
  PROJ_ROOT / "multivariate_skew_distributions",
  PROJ_ROOT / "multivariate_skew_distribution",
]
found = None
for cand in CANDIDATES:
  if (cand / "multivariate_skew_t.py").exists():
    sys.path.insert(0, str(cand))
    found = cand
    break
if found is None:
  raise FileNotFoundError(
    "Impossibile trovare multivariate_skew_t.py. "
    "Atteso in:\n - multivariate_skew_distributions/\n - multivariate_skew_distribution/\n"
    f"Root progetto: {PROJ_ROOT}"
  )

# Importa SOLO la versione 'generator'
from multivariate_skew_t import multivariate_skew_t as mst # modulo locale
# Dati: leggi se esiste, altrimenti genera
info("=== ST multivariata (Python) - start ===")
if os.path.exists(paths["data"]):
  info(f"[A] Lettura dataset: {paths['data']}")
  df = pd.read_csv(paths["data"])
else:
  ...

Y = df.to_numpy()
n, p = Y.shape
info(f" → dataset: n={n}, p={p}")

# MLE (Sigma diagonale) con mst.logpdf
# Parametrizzazione: theta = (mu[0..p-1], log_var[0..p-1], alpha[0..p-1], nu)
LOG_VAR_MIN = float(os.getenv("ST_MV_LOG_VAR_MIN", "-10.0")) # var exp(-10)
LOG_VAR_MAX = float(os.getenv("ST_MV_LOG_VAR_MAX", "10.0")) # var exp(10)
NU_MIN = float(os.getenv("ST_MV_NU_MIN", "2.05")) # var finita
NU_MAX = float(os.getenv("ST_MV_NU_MAX", "40.0"))
```

```

def pack(mu, log_var, alpha, nu):
    return np.concatenate([mu, log_var, alpha, np.atleast_1d(nu)])

def unpack(theta):
    mu = theta[0:p]
    log_var = theta[p:2*p]
    alpha = theta[2*p:3*p]
    nu = theta[3*p]
    return mu, log_var, alpha, nu

def stable_var(log_var):
    lv = np.clip(log_var, LOG_VAR_MIN, LOG_VAR_MAX)
    return np.exp(lv)

def stable_nu(nu):
    return float(np.clip(nu, NU_MIN, NU_MAX))

def nll(theta):
    mu, log_var, alpha, nu = unpack(theta)
    var = stable_var(log_var)
    if not np.all(np.isfinite(var)) or np.any(var <= 0):
        return np.inf
    Sigma = np.diag(var).astype(float)
    nu = stable_nu(nu)
    try:
        lp = mst.logpdf(Y, df=nu, loc=mu, scale=Sigma, shape=alpha)
        if not np.all(np.isfinite(lp)):
            return np.inf
        ll = float(np.sum(lp))
    except Exception:
        return np.inf
    return -ll

# Start: momenti campionari + alpha=0 + nu iniziale moderato
mu0 = Y.mean(axis=0)
var0 = Y.var(axis=0) + 1e-6
log_var0 = np.log(np.clip(var0, np.exp(LOG_VAR_MIN+1), np.exp(LOG_VAR_MAX-1)))
alpha0 = np.zeros(p)
nu0 = 8.0
x0 = pack(mu0, log_var0, alpha0, nu0)

# Bounds per L-BFGS-B
bounds = (
    [(None, None)] * p +
    [(LOG_VAR_MIN, LOG_VAR_MAX)] * p +
    [(None, None)] * p +
    [(NU_MIN, NU_MAX)]
)

info("[B] Stima MLE (L-BFGS-B) su (mu, log_var, alpha, nu) con Sigma diagonale ...")
opt = minimize(nll, x0, method="L-BFGS-B", bounds=bounds,
    options={"maxiter": 8000, "ftol": 1e-9, "maxfun": 300000})
info(f" → success={opt.success}, fun={opt.fun:.6f}, nit={opt.nit}")

mu_hat, log_var_hat, alpha_hat, nu_hat = unpack(opt.x)
var_hat = stable_var(log_var_hat)
Sigma_hat = np.diag(var_hat)
nu_hat = stable_nu(nu_hat)
loglik = -float(opt.fun)

# Pairs plot
if DO_PAIRS:
    info("[C] Pairs plot ...")
    sub_idx = np.arange(n) if n <= 3000 else np.random.default_rng(123).choice(n, size=3000, replace=False)
    scatter_matrix(
        df.iloc[sub_idx, :],
        figsize=(10, 9),
        diagonal='hist',
        range_padding=0.05
    )
    plt.suptitle("ST multivariata - pairs plot (Python)", y=1.02)
    plt.tight_layout()
    plt.savefig(paths["fig_pairs"], dpi=120, bbox_inches="tight")
    plt.close()
    info(f" → salvato: {paths['fig_pairs']}")

# Salvataggi
def fmt(x, k=6):
    try:
        return f"{float(x):.{k}g}"
    except Exception:
        return str(x)

fit_out = {
    "engine": "Python",
    "p": int(p),
    "n": int(n),
    "loglik": float(loglik),
    "nu_hat": float(nu_hat),
    "omega_hat_diag": ",".join(fmt(v) for v in np.diag(Sigma_hat)) # varianze
}
for j in range(p):
    fit_out[f"xi_hat_{j+1}"] = float(mu_hat[j])
for j in range(p):
    fit_out[f"alpha_hat_{j+1}"] = float(alpha_hat[j])

pd.DataFrame([fit_out]).to_csv(paths["tab_fit"], index=False)

```

```

# Matrice Omega in formato "long" (i,j,value)
Om_long = []
for i in range(p):
    for j in range(p):
        Om_long.append({"i": i+1, "j": j+1, "value": float(Sigma_hat[i, j])})
pd.DataFrame(Om_long).to_csv(paths["tab_fit_0m"], index=False)

# stress test alpha
alpha_grid <- c(-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0)
n <- 2000; p <- 3
# Parametri fissi
xi_true <- c(0, 1, -1)
Omega_true <- matrix(c(
    1.0, 0.4, 0.2,
    0.4, 1.0, 0.3,
    0.2, 0.3, 1.0
), nrow = p, byrow = TRUE)
alpha_dir_base <- c(2.0, -1.0, 1.5) # direzione base
alpha_dir <- alpha_dir_base / sqrt(sum(alpha_dir_base^2)) # normalizzata
nu_true <- 8
all_fit <- list()
# Loop sulla griglia di alpha
for (s in alpha_grid) {
    set.seed(20250303 + as.integer(round((s + 10)*1000)))
    alpha_true <- as.numeric(s * alpha_dir) # s = s · v

    # (A) Generazione dati
    Y <- sn::rmst(n, xi = xi_true, Omega = Omega_true, alpha = alpha_true, nu = nu_true)
    colnames(Y) <- sprintf("y%02d", seq_len(ncol(Y)))
    tag <- gsub("\\.", "p", sprintf("%.1f", s))
    fdata <- file.path(dirs$data, sprintf("st_mv_A_alpha_%s.csv", tag))
    write.csv(as.data.frame(Y), fdata, row.names = FALSE)

    # (B) Fit via selm (family = "ST")
    Ydf <- as.data.frame(Y)
    form <- as.formula(paste0("cbind(", paste(names(Ydf), collapse = ","), ") ~ 1"))
    fit <- sn::selm(form, data = Ydf, family = "ST")

    # Estrazione parametri DP (xi, Omega, alpha, nu)
    cf <- sn::coef(fit, "DP", vector = FALSE)
    xi_hat <- as.numeric(cf$beta)
    Omega_hat <- as.matrix(cf$Omega)
    alpha_hat <- as.numeric(cf$alpha)
    nu_hat <- as.numeric(cf$nu)

    # Log-verosimiglianza
    llhat <- if (!is.null(fit@logL)) as.numeric(fit@logL) else {
        dens <- sn::dmst(as.matrix(Y), xi = xi_hat, Omega = Omega_hat, alpha = alpha_hat, nu = nu_hat)
        sum(log(dens))
    }

    # (C) Riepilogo riga
    row <- data.frame(
        engine = "R",
        alpha_scale = s,
        p = ncol(Y),
        n = nrow(Y),
        loglik = llhat,
        nu_hat = nu_hat,
        omega_hat_diag = paste(fmt(diag(Omega_hat)), collapse = ";"),
        stringsAsFactors = FALSE
    )
    for (j in seq_len(ncol(Y))) row[[paste0("xi_hat_", j)]] <- xi_hat[j]
    for (j in seq_len(ncol(Y))) row[[paste0("alpha_hat_", j)]] <- alpha_hat[j]
    for (j in seq_len(ncol(Y))) row[[paste0("alpha_true_", j)]] <- alpha_true[j]

    all_fit[[length(all_fit) + 1]] <- row

    cat(sprintf(" s=%s + %s | loglik=%.6g | nu=%.6g\n", tag, fdata, llhat, nu_hat))
}

# stress test alpha
# stessi import di prima
alpha_grid = [-4.0, -1.2, -0.2, 0.0, 0.2, 1.2, 4.0] # stessi tag dell'R grid
# Direzione di skew: nu = (2, -1, 1.5)
alpha_dir_base = np.array([2.0, -1.0, 1.5], dtype=float)
alpha_dir = alpha_dir_base / np.linalg.norm(alpha_dir_base)

# Bounds robuste (coerenti con lo script ST mv "lean")
LOG_VAR_MIN = float(os.getenv("ST_MV_LOG_VAR_MIN", "-10.0")) # var >= exp(-10)
LOG_VAR_MAX = float(os.getenv("ST_MV_LOG_VAR_MAX", "10.0")) # var <= exp(+10)
NU_MIN = float(os.getenv("ST_MV_NU_MIN", "2.05")) # >2 per var finita
NU_MAX = float(os.getenv("ST_MV_NU_MAX", "40.0"))

def _format_tag(a: float) -> str:
    """Canonical tag with sign preserved (e.g. -1.2 -> '-1p2')."""
    return f"{a:.1f}".replace(".", "p")

def _candidate_dataset_names(a: float):
    """Possibili nomi file, prima convenzione attuale poi legacy."""
    canonical = _format_tag(a) # es. -1.2 -> "-1p2"
    legacy = canonical.replace("-", "m") # es. -1.2 -> "m1p2"
    seen = set()
    for tag in (canonical, legacy):

```

```

        fname = f"st_mv_A_alpha_{tag}.csv"
        if fname not in seen:
            seen.add(fname)
            yield fname

def read_Y(a: float) -> pd.DataFrame:
    for fname in _candidate_dataset_names(a):
        fp = DATA_DIR / fname
        if fp.exists():
            return pd.read_csv(fp)
        raise FileNotFoundError(
            f"Dataset mancante per alpha_scale={a:.1f}. "
            "Esegui prima lo script R grid. "
            f"Ricercati: {'', '.join(str(DATA_DIR / fn) for fn in _candidate_dataset_names(a))}"
        )

def fmt(x, k=6):
    try:
        return f"{float(x):.{k}g}"
    except Exception:
        return str(x)

# Parametrizzazione e NLL
# theta = (mu[0..p-1], log_var[0..p-1], alpha[0..p-1], nu)
def pack(mu, log_var, alpha, nu):
    return np.concatenate([mu, log_var, alpha, np.atleast_1d(nu)])

def unpack(theta, p):
    mu = theta[0:p]
    log_var = theta[p:2*p]
    alpha = theta[2*p:3*p]
    nu = float(theta[3*p])
    return mu, log_var, alpha, nu

def stable_var(log_var):
    lv = np.clip(log_var, LOG_VAR_MIN, LOG_VAR_MAX)
    return np.exp(lv)

def stable_nu(nu):
    return float(np.clip(nu, NU_MIN, NU_MAX))

def make_nll(Y):
    def nll(theta):
        p = Y.shape[1]
        mu, log_var, alpha, nu = unpack(theta, p)
        var = stable_var(log_var)
        if not np.all(np.isfinite(var)) or np.any(var <= 0):
            return np.inf
        Sigma = np.diag(var).astype(float)
        nu = stable_nu(nu)
        try:
            lp = mst.logpdf(Y, df=nu, loc=mu, scale=Sigma, shape=alpha)
            if not np.all(np.isfinite(lp)):
                return np.inf
            ll = float(np.sum(lp))
        except Exception:
            return np.inf
        return -ll
    return nll

# Loop sulla griglia
rows = []

for a in alpha_grid:
    # (A) Lettura dati
    df = read_Y(a)
    Y = df.to_numpy(dtype=float)
    n, p = Y.shape

    # (B) Start e bounds
    mu0 = Y.mean(axis=0)
    var0 = Y.var(axis=0) + 1e-6
    log_var0 = np.log(var0)
    log_var0 = np.clip(log_var0, LOG_VAR_MIN + 1.0, LOG_VAR_MAX - 1.0)
    alpha0 = np.zeros(p, dtype=float)
    nu0 = 8.0

    x0 = pack(mu0, log_var0, alpha0, nu0)
    bounds = (
        [(None, None)] * p +
        [(LOG_VAR_MIN, LOG_VAR_MAX)] * p +
        [(None, None)] * p +
        [(NU_MIN, NU_MAX)]
    )

    # (C) Fit MLE (L-BFGS-B)
    nll = make_nll(Y)
    opt = minimize(nll, x0, method="L-BFGS-B", bounds=bounds,
        options={"maxiter": 8000, "ftol": 1e-9, "maxfun": 300000})

    mu_hat, log_var_hat, alpha_hat, nu_hat = unpack(opt.x, p)
    var_hat = stable_var(log_var_hat)
    Sigma_hat = np.diag(var_hat)
    nu_hat = stable_nu(nu_hat)
    loglik = -float(opt.fun)

    # (D) Riga output

```

```

row = {
    "engine": "Python",
    "alpha_scale": float(a),
    "p": int(p),
    "n": int(n),
    "loglik": float(loglik),
    "nu_hat": float(nu_hat),
    "omega_hat_diag": " ".join(fmt(v) for v in np.diag(Sigma_hat)),
}
for j in range(p):
    row[f"xi_hat_{j+1}"] = float(mu_hat[j])
    row[f"alpha_hat_{j+1}"] = float(alpha_hat[j])

# true per trasparenza/confronto (stessa direzione dell'R)
alpha_true_vec = float(a) * alpha_dir
for j in range(p):
    row[f"alpha_true_{j+1}"] = float(alpha_true_vec[j])

rows.append(row)
print(f" a={a:.1f}  n={n}  p={p}  loglik={loglik:.6f}  nu_hat={nu_hat:.6g}", flush=True)

```


Codici Capitolo 4 - Implementazione SUN

Funzione dsun (Python)

```
# Implementazione della funzione di densità della SUN
# Vengono utilizzate solo le librerie SciPy e NumPy
# Basata sulla funzione del pacchetto "sn" di R
from __future__ import annotations
import warnings
import numpy as np
import scipy
from typing import Optional, Tuple
from scipy.stats import multivariate_normal
from scipy.linalg import LinAlgError

def _chol_solve(L: np.ndarray, B: np.ndarray) -> np.ndarray:
    """
    Risolve il sistema (L L) x = B usando forward e backward substitution,
    dato il fattore di Cholesky inferiore L (ovvero: A = L L).

    Parametri
    -----
    L : np.ndarray
        Fattore di Cholesky inferiore (triangolare) di A.
    B : np.ndarray
        Termine noto (vettore o matrice). Le ultime dimensioni devono
        essere compatibili con L.

    Ritorna
    -----
    np.ndarray
        Soluzione x che soddisfa (L L) x = B.
    """
    # forward solve: L Y = B
    Y = scipy.linalg.solve(L, B, assume_a='pos', lower=True, check_finite=False)
    # backward solve: L x = Y
    return scipy.linalg.solve(L.T, Y, assume_a='pos', lower=False, check_finite=False)

def _is_simdefpos(A: Optional[np.ndarray], tol: float = 1e-12) -> bool:
    """
    Verifica se una matrice è simmetrica (entro tolleranza) e definita positiva.

    Note:
    - Usa eigvalsh (autovalori reali) perché A è simmetrica.
    - Consente un piccolo margine negativo (~ -1e-12) per tolleranze numeriche.

    Parametri
    -----
    A : np.ndarray | None
        Matrice candidata.
    tol : float
        Tolleranza per la simmetria.

    Ritorna
    -----
    bool
        True se la matrice è simmetrica e definita positiva entro tolleranza.
    """
    if A is None:
        return False
    A = np.asarray(A)
    if A.ndim != 2 or A.shape[0] != A.shape[1]:
        return False
    # simmetria numerica
    if not np.allclose(A, A.T, atol=max(tol, 1e-15)):
        return False
    # positività (entro piccola tolleranza negativa)
    try:
        vals = np.linalg.eigvalsh(A)
    except Exception:
        return False
    return np.min(vals) > -1e-12

def make_PD(A: np.ndarray, eps_start: float = 1e-8, max_iter: int = 10) -> np.ndarray:
    """
    Rende una matrice simmetrica e (quasi) definita positiva aggiungendo un jitter
    diagonale crescente fino a ottenere una fattorizzazione di Cholesky.

    Strategia:
    - Simmetrizza A.
    - Prova cholesky; se fallisce, sposta gli autovalori minimi verso la positività
      con un incremento diagonale crescente.
    - Come fallback, forza positività diagonale minima.

    Parametri
    -----
    A : np.ndarray
        Matrice di ingresso.
    eps_start : float
    """
```

```

        Jitter diagonale iniziale.
max_iter : int
        Numero massimo di tentativi di regolarizzazione.

Ritorna
-----
np.ndarray
    Matrice simmetrica (quasi) definita positiva.
"""
A = 0.5 * (np.asarray(A) + np.asarray(A).T)
eps = eps_start

for _ in range(max_iter):
    try:
        # Se Cholesky riesce, A è già PD
        scipy.linalg.cholesky(A, check_finite=False)
        return A
    except LinAlgError:
        # Stima lo shift necessario a rendere A PD
        try:
            min_ev = np.min(scipy.linalg.eigvalsh(A))
        except Exception:
            min_ev = -abs(eps)
        shift = max(eps, -min_ev + eps)
        A = A + np.eye(A.shape[0]) * shift
        eps *= 10

# Fallback: garantisce positività minima sulla diagonale
diag = np.diag(A).copy()
diag[diag <= 0] = eps
A = A + np.eye(A.shape[0]) * eps
return A

# Densità della SUN
def dsun(
    x: np.ndarray,
    xi: np.ndarray,
    Omega: np.ndarray,
    Delta: np.ndarray,
    tau: np.ndarray,
    Gamma: np.ndarray,
    log: bool = False
) -> np.ndarray:
    """
    Calcola la (log-)densità della distribuzione SUN nel/i punto/i x.

    Parametri
    -----
    x : np.ndarray
        Punto di valutazione. Accetta forma (p,) per un singolo punto o (n, p) per n punti.
    xi : np.ndarray
        Vettore di localizzazione (dimensione p,).
    Omega : np.ndarray
        Matrice di covarianza (p x p).
    Delta : np.ndarray
        Matrice di skewness (p x m).
    tau : np.ndarray
        Vettore di soglia (m,).
    Gamma : np.ndarray
        Matrice di covarianza aggiuntiva (m x m).
    log : bool
        Se True, restituisce il logaritmo della densità.

    Ritorna
    -----
    np.ndarray
        Vettore (n,) di densità o log-densità della SUN in corrispondenza delle righe di x.
    """
    # --- Validazioni e conversioni ---
    X = np.asarray(x)
    xi = np.asarray(xi)
    Omega = np.asarray(Omega)
    Delta = np.asarray(Delta)
    tau = np.asarray(tau)
    Gamma = np.asarray(Gamma)

    # Consenti x singolo (p,) oppure matrice (n, p)
    if X.ndim == 1:
        X = X.reshape(1, -1)
    n, p = X.shape
    m = tau.size

    # Controlli dimensionali coerenti
    if xi.size != p:
        raise ValueError("Dimensione di xi non coerente con X.")
    if Omega.shape != (p, p):
        raise ValueError("Omega deve essere p x p.")
    if Delta.shape[0] != p or Delta.shape[1] != m:
        raise ValueError("Delta deve essere p x m.")
    if Gamma.shape != (m, m):
        raise ValueError("Gamma deve essere m x m.")

    # Verifica simmetria/PD per Omega e Gamma (entro tolleranza)
    if not _is_simdefpos(Omega):
        raise ValueError("Omega non simmetrica/PD.")
    if not _is_simdefpos(Gamma):

```

```

        raise ValueError("Gamma non simmetrica/PD.")

# --- Standardizzazione marginale ---
# omega: deviazioni standard marginali; inv_omega: inv(diag(omega))
omega = np.sqrt(np.diag(Omega))
if np.any(omega == 0):
    raise ValueError("Elementi diagonali di Omega nulli.")
inv_omega = np.diag(1.0 / omega)

# Omega_bar = D^{-1} Omega D^{-1} con D = diag(omega)
Omega_bar = inv_omega @ Omega @ inv_omega
Omega_bar = 0.5 * (Omega_bar + Omega_bar.T) # simmetrizza numericamente
Omega_bar = make_PD(Omega_bar)

# Fattorizzazione di Cholesky di Omega_bar
L_bar = scipy.linalg.cholesky(Omega_bar, check_finite=False)

# M = Omega_bar^{-1} Delta (risolto tramite Cholesky)
M = _chol_solve(L_bar, Delta)

# Sigma_m = Gamma - Delta Omega_bar^{-1} Delta
Sigma_m = Gamma - Delta.T @ M
Sigma_m = 0.5 * (Sigma_m + Sigma_m.T) # simmetrizza

# Assicurati che Sigma_m sia SPD (aggiusta con jitter se necessario)
if not _is_simdefpos(Sigma_m):
    Sigma_m = make_PD(Sigma_m, eps_start=1e-10, max_iter=20)
    if not _is_simdefpos(Sigma_m):
        raise ValueError("Sigma_m non convertibile in SPD.")

# Cholesky di Sigma_m (ridondante per robustezza numerica, come da implementazione originale)
L_Sm = scipy.linalg.cholesky(Sigma_m, check_finite=False)

# Ulteriore salvaguardia: riapplica make_PD e rifai Cholesky
Sigma_m = make_PD(Sigma_m, eps_start=1e-10, max_iter=20)
L_Sm = scipy.linalg.cholesky(Sigma_m, check_finite=False)

# --- Preparazione covarianze regolarizzate per valutazioni PDF/CDF ---
Gamma_pd = make_PD(Gamma, eps_start=1e-10, max_iter=20)
if not np.allclose(Gamma_pd, Gamma):
    warnings.warn("Gamma regularized to be positive-definite for CDF evaluation.")

Omega_pd = make_PD(Omega, eps_start=1e-12, max_iter=20)
if not np.allclose(Omega_pd, Omega):
    warnings.warn("Omega regularized to be positive-definite for PDF evaluation.")

# Distribuzioni normali multivariate per le componenti CDF/PDF
mvn_Gamma = multivariate_normal(mean=np.zeros(m), cov=Gamma_pd, allow_singular=True)
pi = mvn_Gamma.cdf(tau)
pi = float(np.clip(pi, 1e-300, None)) # stabilità log
log_pi = np.log(pi)

# Prealloca output (log-densità) e PDF con Omega regolarizzata
out_log = np.empty(n, dtype=float)
mvn_Omega = multivariate_normal(mean=xi, cov=Omega_pd, allow_singular=True)

# --- Ciclo sui punti X ---
for i in range(n):
    # Scostamento dal centro
    xi_minus = X[i] - xi

    # Standardizzazione z = D^{-1} (x - xi)
    z = inv_omega @ xi_minus

    # u = Omega_bar^{-1} z (via Cholesky solve)
    u = _chol_solve(L_bar, z)

    # s = tau + Delta u (argomento CDF m-dimensionale)
    s = tau + Delta.T @ u

    # log_p(x; xi, Omega)
    log_phi = mvn_Omega.logpdf(X[i])

    # Calcolo CDF_m(s; 0, Sigma_m) con Sigma_m regolarizzata
    Sm_pd = make_PD(Sigma_m, eps_start=1e-10, max_iter=20)
    if not np.allclose(Sm_pd, Sigma_m):
        warnings.warn("Sigma_m regularized prior to CDF(s) evaluation.")
    mvn_Sm = multivariate_normal(mean=np.zeros(m), cov=Sm_pd, allow_singular=True)
    pi2 = mvn_Sm.cdf(s)
    pi2 = float(np.clip(pi2, 1e-300, None))
    log_pi2 = np.log(pi2)

    # log-densità SUN (formula: log + log - log (tau))
    out_log[i] = log_phi + log_pi2 - log_pi

# Ritorna log-densità o densità
return out_log if log else np.exp(out_log)

```

Funzione psum (Python)

```
# Implementazione della funzione di ripartizione della SUN
# Vengono utilizzate solo le librerie SciPy e NumPy
# Utilizzo la funzione di densità SUN definita in densita_sun.py
# Basata sulla funzione del pacchetto "sn" di R
from __future__ import annotations
import os
import sys
import warnings
import importlib.util
import numpy as np
from typing import Optional
from scipy.stats import multivariate_normal
from scipy.linalg import LinAlgError # noqa: F401 (import coerente con il file originale)
from scipy.special import ndtr # noqa: F401 (import coerente con il file originale)

# Caricamento esplicito di densita_sun.py dalla stessa cartella
here = os.path.dirname(__file__)
densita_path = os.path.join(here, "densita_sun.py")
if not os.path.exists(densita_path):
    # Se il file non esiste, prova comunque un import "normale" per un messaggio più informativo
    try:
        from densita_sun import dsun, make_PD, _is_simdefpos # pragma: no cover
    except Exception as e:
        raise ImportError(
            f"densita_sun.py non trovato in {here} e import semplice fallito: {e}"
        )
else:
    spec = importlib.util.spec_from_file_location("densita_sun_local", densita_path)
    mod = importlib.util.module_from_spec(spec)
    assert spec is not None and spec.loader is not None
    spec.loader.exec_module(mod) # type: ignore[attr-defined]
    dsun = getattr(mod, "dsun")
    make_PD = getattr(mod, "make_PD")
    _is_simdefpos = getattr(mod, "_is_simdefpos")

# Funzione di ripartizione SUN (stima Monte Carlo importance sampling)
def psum(
    x: np.ndarray,
    xi: np.ndarray,
    Omega: np.ndarray,
    Delta: np.ndarray,
    tau: np.ndarray,
    Gamma: np.ndarray,
    log: bool = False,
    max_dim: int = 20,
    nsamples: int = 20000,
    seed: Optional[int] = None
) -> np.ndarray:
    """
    Calcola la funzione di ripartizione (CDF) della distribuzione SUN nel/i punto/i x.

    Metodo:
    - Stima Monte Carlo con importance sampling usando come proposta _d(·; xi, Omega).
    - Il peso è w(y) = f_SUN(y) / _d(y; xi, Omega).
    - Per ogni x_i si stima P(Y ≤ x_i) media( w(Y_j) * 1{Y_j ≤ x_i} ).

    Parametri
    -----
    x : (n,d) oppure (d,)
        Punti in cui valutare la CDF. Se (d,), viene trattato come un singolo punto.
    xi : (d,)
        Vettore di posizione.
    Omega : (d,d)
        Matrice di covarianza (come in dsun).
    Delta : (d,m)
        Matrice di selezione/skewness.
    tau : (m,)
        Vettore di soglia.
    Gamma : (m,m)
        Matrice di covarianza aggiuntiva.
    log : bool
        Se True, restituisce log(CDF) (con clipping numerico).
    max_dim : int
        Soglia massima su d+m (default 20), come in "sn" di R: evita CDF multivariate instabili.
    nsamples : int
        Numero di campioni Monte Carlo per la stima (proposta _d).
    seed : int | None
        Semina del generatore pseudo-casuale (per riproducibilità).

    Ritorna
    -----
    np.ndarray
        Vettore (n,) con le stime della CDF in [0,1] per ciascuna riga di x
        (o log-CDF se log=True).
    """
    # -----
    # Validazioni e conversioni input
    # -----
    X = np.asarray(x)
    xi = np.asarray(xi)
    Omega = np.asarray(Omega)
    Delta = np.asarray(Delta)
```

```

tau = np.asarray(tau)
Gamma = np.asarray(Gamma)

# Accetta (d,) + (1,d)
if X.ndim == 1:
    X = X.reshape(1, -1)
n, d = X.shape
m = tau.size

# Controlli dimensionali
if xi.size != d:
    raise ValueError("Dimensione di xi non coerente con X.")
if Omega.shape != (d, d):
    raise ValueError("Omega deve essere d x d.")
if Delta.shape != (d, m):
    raise ValueError("Delta deve essere d x m.")
if Gamma.shape != (m, m):
    raise ValueError("Gamma deve essere m x m.")

# Soglia pratica su d+m (come in 'sn' di R) per stabilità numerica
if d + m > max_dim:
    raise ValueError(
        f"Dimensione complessiva d+m={d+m} supera la soglia max_dim={max_dim}."
    )

# Verifica simmetria/PD su Omega e Gamma
if not _is_simdefpos(Omega):
    raise ValueError("Omega non simmetrica/PD.")
if not _is_simdefpos(Gamma):
    raise ValueError("Gamma non simmetrica/PD.")

# Standardizzazione e regolarizzazione SPD

# Diagonale delle varianze marginali e matrice standardizzata Omega_bar
omega = np.sqrt(np.diag(Omega))
if np.any(omega <= 0):
    raise ValueError("Omega non definita positiva.")
inv_omega = np.diag(1.0 / omega)
Omega_bar = inv_omega @ Omega @ inv_omega
Omega_bar = 0.5 * (Omega_bar + Omega_bar.T) # simmetria numerica
# Garantisce SPD (per coerenza con densità e stabilità)
Omega_bar = make_PD(Omega_bar, eps_start=1e-10, max_iter=20)

# Proposta Monte Carlo: _d(.)

Omega_pd = make_PD(Omega, eps_start=1e-12, max_iter=20)
if not np.allclose(Omega_pd, Omega):
    warnings.warn("Omega regularized to be positive-definite for sampling.")

rng = np.random.default_rng(seed)
# Campioni dalla proposta _d(y; xi, Omega_pd)
Y = rng.multivariate_normal(mean=xi, cov=Omega_pd, size=int(nsamples))

# Densità della proposta (Y) (normale multivariata)
mvn_prop = multivariate_normal(mean=xi, cov=Omega_pd, allow_singular=True)
phi_vals = mvn_prop.pdf(Y)

# Densità bersaglio f(Y) = dsun(Y, ...)
f_vals = np.asarray(dsun(Y, xi, Omega, Delta, tau, Gamma, log=False))

# Evita divisioni per zero e calcola i pesi d'importanza
phi_vals = np.clip(phi_vals, 1e-300, None)
weights = f_vals / phi_vals

# Stima CDF per ciascun punto x_i richiesto

out = np.empty(n, dtype=float)
stderr = np.empty(n, dtype=float) # stima dell'errore standard (informativa)

for i in range(n):
    x_i = X[i]
    # Indicatore 1{Y x_i} (confronto componente per componente)
    inds = np.all(Y <= x_i.reshape(1, -1), axis=1).astype(float)

    # Stimatore d'importanza: E[ w(Y) * 1{Y x_i} ]
    vals = weights * inds
    est = float(np.mean(vals))

    # Varianza Monte Carlo (solo informativa; non restituita)
    var = float(np.var(vals, ddof=1)) if vals.size > 1 else 0.0
    se = float(np.sqrt(var / max(1, len(vals))))

    # Proietta entro [0,1] per robustezza numerica
    out[i] = min(max(est, 0.0), 1.0)
    stderr[i] = se # disponibile per diagnostica, non usato in output

# Output (log o no)

if log:
    with np.errstate(divide="ignore"):
        return np.log(np.clip(out, 1e-300, None))
return out

```


Bibliografia

- [1] A. Azzalini e A. Capitanio (2014). *The Skew-Normal and Related Families*. Cambridge University Press, Cambridge. ISBN 978-1-107-02927-9.
- [2] A. Azzalini (2021). *Package `sn`: The Skew-Normal and Related Distributions Such as the Skew-t and the SUN*. Manuale R, versione 2.0.0, 25 marzo 2021.
- [3] D. Durante (2019). Conjugate Bayes for probit regression via unified skew-normals. *arXiv:1802.09565v5*.
- [4] A. Fasano e D. Durante (2022). A Class of Conjugate Priors for Multinomial Probit Models which Includes the Multivariate Normal One. *Journal of Machine Learning Research*
- [5] N. Anceschi, A. Fasano, D. Durante e G. Zanella (2023). Bayesian Conjugacy in Probit, Tobit, Multinomial Probit and Extensions: A Review and New Results. *Journal of the American Statistical Association*
- [6] A. Fasano e F. Denti (2024). Multivariate Gaussian cumulative distribution functions as the marginal likelihood of their dual Bayesian probit models. *Biometrika*
- [7] *SciPy*, <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.skewnorm.html>,
- [8] *tScipi*, <https://pypi.org/project/skewt-sciPy/#description>,
- [9] *mskewdisp*, https://github.com/spc90/multivariate_skew_distributions,
- [10] *botorch*, https://github.com/meta-pytorch/botorch/blob/main/test/utis/probability/test_unified_skew_normal.py,
- [11] *stackoverflow* <https://survey.stackoverflow.co/2024/technology#most-popular-technologies-language>

Ringraziamenti

Chiudo questo elaborato con poche ma sincere parole di gratitudine: pur trattandosi di un elaborato triennale, per me segna la conclusione di un lungo e importante periodo.

Ringrazio innanzitutto il Prof. Antonio Canale per i consigli nello studio: le sue lezioni mi hanno fatto riscoprire la passione per la statistica.

Desidero ringraziare NEXTER S.r.l., che ha sostenuto questo traguardo mettendomi a disposizione tempo e risorse.

Un ringraziamento speciale a Margherita Casetta e Pierpaolo Cavalli per le lunghe chiacchierate e per avermi spronato a non mollare.

Alla Dott.ssa Elisa Galesso va un grazie di cuore: la sua presenza nei momenti più critici mi ha aiutato a portare a termine questo percorso senza perdere la fiducia e il sorriso.

Grazie agli amici Bartolomeo, Caterina, Elisa, Emanuele, Erika, Nicole e Sebastiano per tutti i momenti di leggerezza passati insieme e per aver sopportato le mie mille lamentele.

Grazie anche alla famiglia, Madre, Padre, Fratello e Nonni, dove ognuno, a modo suo, fra alti e bassi, ha fatto parte di questo percorso.

A Te, che non ci sei più, grazie per avermi guidato e ora, *stay tuned...*

*“Keep smiling, because life is a beautiful thing
and there’s so much to smile about.”*

M.M.